

THE SPAWN OF CHAOS
Game Dev OS and Unity Script Guide

Purpose: a ranked list of 40 project files that are easiest to explain.

Python note

No project-owned Python files were found in the workspace. The Game Dev OS logbook describes Python/Flask work, but the backend source is not present here.

The project documentation is in: `game_dev_and_ai_training_logbook.md`
The production plan is in: `The_Spawn_of_Chaos_Demo_Roadmap.md`

Frontend

FILE 1: `index.html`

PURPOSE: Drift Up page structure.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Drift Up</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <main class="app">
      <h1>Drift Up</h1>
      <p>Tap or click to flip the drift. Survive as long as possible.</p>
      <canvas id="game" width="480" height="720"></canvas>
      <div class="hud">
        <div class="score">Score: <span id="score">0</span></div>
        <button id="restart">Restart</button>
      </div>
    </main>
    <script src="game.js"></script>
  </body>
</html>
```

FILE 2: `game.js`

PURPOSE: Canvas game loop, movement, obstacles, and collisions.

```
const canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');
const scoreEl = document.getElementById('score');
const restartButton = document.getElementById('restart');

const playerSize = 16;
const baseSpeed = 260;
const maxSpeed = 420;

let state = createInitialState();

function createInitialState() {
  return {
    player: {
      x: canvas.width / 2,
      y: canvas.height - 90,
      vx: 0,
      vy: -baseSpeed,
      dir: 0,
    },
    obstacles: [],
    spawnCooldown: 0.8,
    time: 0,
    gameOver: false,
    message: 'Tap to start drifting',
  };
};

function resetGame() {
  state = createInitialState();
  state.message = 'Tap to change direction';
}
```

```

function toggleDirection() {
  if (state.gameOver) {
    resetGame();
    return;
  }

  state.player.dir = state.player.dir === 0 ? -1 : -state.player.dir;
  state.player.vx = state.player.dir * 170;
  state.message = 'Keep going!';
}

canvas.addEventListener('pointerdown', toggleDirection);
window.addEventListener('keydown', (event) => {
  if (event.code === 'Space' || event.code === 'ArrowLeft' || event.code === 'ArrowRight') {
    event.preventDefault();
    toggleDirection();
  }
});
restartButton.addEventListener('click', resetGame);

let lastTimestamp = performance.now();

function loop(timestamp) {
  const delta = Math.min(0.032, (timestamp - lastTimestamp) / 1000);
  lastTimestamp = timestamp;

  update(delta);
  draw();

  requestAnimationFrame(loop);
}

function update(delta) {
  if (!state.gameOver) {
    state.time += delta;
    state.spawnCooldown -= delta;
    if (state.spawnCooldown <= 0) {
      spawnObstacle();
      state.spawnCooldown = Math.max(0.28, 0.8 - state.time * 0.01);
    }

    const player = state.player;
    player.x += player.vx * delta;
    player.y += player.vy * delta;

    if (player.x < playerSize || player.x > canvas.width - playerSize) {
      player.x = Math.max(playerSize, Math.min(canvas.width - playerSize, player.x));
      player.dir = 0;
      player.vx = 0;
      state.message = 'Bounced off the wall!';
    }

    state.obstacles.forEach((obstacle) => {
      obstacle.y += obstacle.speed * delta;
      obstacle.x += obstacle.sway * delta;
      if (obstacle.x < -120) obstacle.x = -120;
      if (obstacle.x > canvas.width - obstacle.w + 120) obstacle.x = canvas.width - obstacle.w
+ 120;
    });

    state.obstacles = state.obstacles.filter((obstacle) => obstacle.y < canvas.height + 60);

    const collision = state.obstacles.some((obstacle) => intersects(player, obstacle));
    if (collision) {
      state.gameOver = true;
      state.message = 'Crash! You lasted ${Math.floor(state.time)}s';
    }
  }

  scoreEl.textContent = Math.floor(state.time);
}

function spawnObstacle() {
  const width = 60 + Math.random() * 110;
  const height = 16 + Math.random() * 24;
  const x = Math.random() * (canvas.width - width);

```

```

const speed = 220 + Math.random() * 80 + Math.min(90, state.time * 6);

state.obstacles.push({
  x,
  y: -height,
  w: width,
  h: height,
  speed,
  sway: (Math.random() > 0.5 ? 1 : -1) * (40 + Math.random() * 30),
});
}

function intersects(player, obstacle) {
  return (
    player.x - playerSize < obstacle.x + obstacle.w &&
    player.x + playerSize > obstacle.x &&
    player.y - playerSize < obstacle.y + obstacle.h &&
    player.y + playerSize > obstacle.y
  );
}

function draw() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  const gradient = ctx.createLinearGradient(0, 0, 0, canvas.height);
  gradient.addColorStop(0, '#071129');
  gradient.addColorStop(0.45, '#0d1832');
  gradient.addColorStop(1, '#060814');
  ctx.fillStyle = gradient;
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  drawStars();
  drawGuideLines();

  state.obstacles.forEach((obstacle) => {
    ctx.fillStyle = '#ff5d7b';
    roundRect(ctx, obstacle.x, obstacle.y, obstacle.w, obstacle.h, 10);
    ctx.fill();
  });

  const player = state.player;
  ctx.save();
  ctx.translate(player.x, player.y);
  ctx.fillStyle = '#7bffc9';
  ctx.beginPath();
  ctx.moveTo(0, -playerSize);
  ctx.lineTo(playerSize, playerSize);
  ctx.lineTo(-playerSize, playerSize);
  ctx.closePath();
  ctx.fill();
  ctx.restore();

  ctx.fillStyle = 'rgba(255,255,255,0.72)';
  ctx.font = '700 18px Inter, sans-serif';
  ctx.textAlign = 'center';
  ctx.fillText(state.message, canvas.width / 2, 40);
}

function drawStars() {
  ctx.save();
  ctx.fillStyle = 'rgba(255,255,255,0.75)';
  for (let i = 0; i < 60; i += 1) {
    const x = (i * 97) % canvas.width;
    const y = (i * 31) % canvas.height;
    const radius = (i % 3) + 1;
    ctx.beginPath();
    ctx.arc(x, y, radius, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawGuideLines() {
  ctx.save();
  ctx.strokeStyle = 'rgba(255,255,255,0.12)';
  ctx.lineWidth = 1;

```

```

    for (let y = 0; y < canvas.height; y += 80) {
      ctx.beginPath();
      ctx.moveTo(0, y);
      ctx.lineTo(canvas.width, y);
      ctx.stroke();
    }
    ctx.restore();
  }

function roundRect(context, x, y, width, height, radius) {
  context.beginPath();
  context.moveTo(x + radius, y);
  context.arcTo(x + width, y, x + width, y + height, radius);
  context.arcTo(x + width, y + height, x, y + height, radius);
  context.arcTo(x, y + height, x, y, radius);
  context.arcTo(x, y, x + width, y, radius);
  context.closePath();
}

requestAnimationFrame(loop);

```

FILE 3: style.css

PURPOSE: Drift Up styling.

```

:root {
  color-scheme: dark;
  font-family: Inter, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: grid;
  place-items: center;
  background: radial-gradient(circle at top, #22184b 0%, #09030f 65%, #040208 100%);
  color: #f7f1ff;
}

.app {
  width: min(92vw, 560px);
  text-align: center;
}

h1 {
  margin: 0 0 0.35rem;
  font-size: clamp(1.9rem, 3vw, 2.6rem);
}

p {
  margin: 0 0 1rem;
  color: #d5c2ff;
}

canvas {
  width: 100%;
  border: 2px solid rgba(255, 255, 255, 0.18);
  border-radius: 20px;
  box-shadow: 0 18px 60px rgba(0, 0, 0, 0.4);
  background: linear-gradient(180deg, #0e162f 0%, #090b16 100%);
}

.hud {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 0.9rem;
}

.score {
  font-size: 1.1rem;
  font-weight: 700;
}

```

```

    letter-spacing: 0.04em;
  }

  button {
    border: none;
    padding: 0.7rem 1rem;
    border-radius: 999px;
    background: linear-gradient(135deg, #f35d8a, #7356ff);
    color: white;
    font-weight: 700;
    cursor: pointer;
    box-shadow: 0 8px 24px rgba(243, 93, 138, 0.25);
  }

  button:hover {
    transform: translateY(-1px);
  }

```

FILE 4: menu.html

PURPOSE: Main menu page structure.

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>The Spawn of Chaos ??? Main Menu</title>
  <link rel="stylesheet" href="menu.css">
</head>
<body>
  <canvas id="bg"></canvas>
  <div class="ui-wrap">
    <h1 id="title">The Spawn of Chaos</h1>
    <div class="subtitle">An immersive dark fantasy adventure</div>

    <div class="buttons">
      <button class="btn" id="startBtn">Start Game</button>
      <button class="btn" id="continueBtn">Continue</button>
      <button class="btn" id="optionsBtn">Options</button>
      <button class="btn" id="quitBtn">Quit</button>
    </div>

    <footer class="credit">Designed by you ?? Animated by JS/CSS</footer>
  </div>

  <script src="menu.js"></script>
</body>
</html>

```

FILE 5: menu.js

PURPOSE: Menu buttons, particles, and interactions.

```

(() => {
  const canvas = document.getElementById('bg');
  const ctx = canvas.getContext('2d');
  let w = canvas.width = innerWidth;
  let h = canvas.height = innerHeight;
  window.addEventListener('resize', () => { w = canvas.width = innerWidth; h = canvas.height = innerHeight; });

  // Particles for ambient chaos
  const particles = [];
  const colors = ['#0ff6ff', '#b88cff', '#ffd97b', '#ff8fa3', '#5effc7'];
  for (let i=0; i<120; i++) particles.push({x:Math.random()*w, y:Math.random()*h, vx:(Math.random()-0.5)*0.2, vy:(Math.random()-0.5)*0.2, size:Math.random()*2+0.4, color:colors[i%colors.length], alpha:Math.random()*0.6+0.15});

  function update(){
    ctx.clearRect(0,0,w,h);
    // soft radial gradient
    const g = ctx.createLinearGradient(0,0,w,h);
    g.addColorStop(0, '#06060a'); g.addColorStop(1, '#040308');
    ctx.fillStyle = g; ctx.fillRect(0,0,w,h);

    // draw particles

```

```

    for (let p of particles){
      p.x += p.vx; p.y += p.vy;
      if (p.x < -10) p.x = w+10; if (p.x > w+10) p.x = -10;
      if (p.y < -10) p.y = h+10; if (p.y > h+10) p.y = -10;
      ctx.globalAlpha = p.alpha;
      ctx.fillStyle = p.color;
      ctx.beginPath(); ctx.arc(p.x,p.y,p.size,0,Math.PI*2); ctx.fill();
    }

    // subtle noise overlay
    ctx.globalAlpha = 0.035;
    for (let i=0;i<35;i++){
      ctx.fillStyle = colors[(i+7)%colors.length];
      ctx.fillRect((Math.random()*w)|0,(Math.random()*h)|0,1,1);
    }
    ctx.globalAlpha = 1;

    requestAnimationFrame(update);
  }
  update();

  // Title shimmer using CSS class toggles for performance
  const title = document.getElementById('title');
  setInterval(()=>{
    title.style.filter = 'drop-shadow(0 22px 40px rgba(0,0,0,0.85))';
    title.style.opacity = 0.98;
    setTimeout(()=>{title.style.filter='drop-shadow(0 8px 30px rgba(0,0,0,0.8))';},240);
  },3000);

  // Letter wave when hovering title
  title.addEventListener('mousemove', (e)=>{
    const rect = title.getBoundingClientRect();
    const x = (e.clientX - rect.left)/rect.width;
    title.style.transform = 'translateY(${(x-0.5)*-10}px) rotate(${(x-0.5)*1}deg)';
  });
  title.addEventListener('mouseleave', ()=>{ title.style.transform = ''; });

  // Buttons
  const startBtn = document.getElementById('startBtn');
  const continueBtn = document.getElementById('continueBtn');
  const optionsBtn = document.getElementById('optionsBtn');
  const quitBtn = document.getElementById('quitBtn');

  function pulse(btn){ btn.animate([transform:'scale(1)',transform:'scale(1.04)'],{duration:260,iterations:1,easing:'ease-out'}); }
  startBtn.addEventListener('click', ()=>{ pulse(startBtn); console.log('Start pressed ??? hook into game bootstrap'); startBtn.disabled=true; startBtn.textContent='Launching...'; setTimeout(()=>startBtn.textContent='Start Game',1200); });
  continueBtn.addEventListener('click', ()=>{ pulse(continueBtn); console.log('Continue pressed ??? load save'); });
  optionsBtn.addEventListener('click', ()=>{ pulse(optionsBtn); alert('Options ??? placeholder'); });
  quitBtn.addEventListener('click', ()=>{ pulse(quitBtn); console.log('Quit pressed ??? close window if applicable'); if (navigator.userAgent.indexOf('Electron')>-1) window.close(); });

  // small accessibility: keyboard control
  document.addEventListener('keydown',e)=>{ if (e.key==='Enter') startBtn.click(); if (e.key==='o') optionsBtn.click(); });
})();

```

FILE 6: menu.css

PURPOSE: Main menu styling.

```

*{box-sizing:border-box;margin:0;padding:0}html,body,#bg{height:100%}body{font-family:Inter,system-ui,Segoe UI,Roboto,Helvetica,Arial,sans-serif;background:#06060a;color:#e6eef6;overflow:hidden}
#bg{position:fixed;left:0;top:0;width:100%;height:100%;z-index:0}
.ui-wrap{position:relative;z-index:2;min-height:100vh;display:flex;flex-direction:column;align-items:center;justify-content:center;padding:4vh 6vw;text-align:center}
#title{font-size:clamp(40px,7vw,120px);letter-spacing:0.02em;margin-bottom:6px;background:linear-gradient(90deg,#9be7ff,#b8cfff 30%,#ffd97b 60%,#ff8fa3);-webkit-background-clip:text;background-clip:text;color:transparent;filter:drop-shadow(0 8px 30px rgba(0,0,0,0.8));}
#title::after{content:"";position:absolute;left:50%;transform:translateX(-50%);top:40%;width:60%;height:2px;background:linear-gradient(90deg,transparent,rgba(255,255,255,0.08),transparent);mix-blend-mode:overlay;opacity:0.7}
.subtitle{color:#98a6b8;opacity:0.85;margin-bottom:24px}

```

```

.buttons{display:flex;flex-direction:column;gap:14px;width:min(420px,80vw);margin-top:12px}
.btn{background:linear-gradient(180deg,rgba(255,255,255,0.03),rgba(255,255,255,0.01));border:1
px solid rgba(255,255,255,0.06);padding:14px 22px;border-radius:12px;color:#eaf6ff;font-weig
ht:700;font-size:18px;cursor:pointer;backdrop-filter:blur(6px);box-shadow:0 8px 30px rgba(0,
0,0,0.6);transition:transform .28s cubic-bezier(.2,.9,.2,1),box-shadow .28s,filter .28s}
.btn:hover{transform:translateY(-6px) scale(1.01);box-shadow:0 18px 48px rgba(0,175,255,0.12);
filter:brightness(1.05)}
.btn:active{transform:translateY(-2px) scale(.99)}
.btn:nth-child(1){border-left:3px solid #6de0ff}
.btn:nth-child(2){border-left:3px solid #b88cff}
.btn:nth-child(3){border-left:3px solid #ffd97b}
.btn:nth-child(4){border-left:3px solid #ff8fa3}
.credit{margin-top:28px;color:#7d8a9b;font-size:13px;opacity:0.9}

/* subtle vignette */
body:before{content:"";position:fixed;inset:0;background:radial-gradient(60% 60% at 50% 40%,tr
ansparent,rgba(0,0,0,0.6));pointer-events:none;z-index:1}

/* title wobble + scanline */
@keyframes wobble{0%{transform:translateY(0)}50%{transform:translateY(-6px) rotate(-0.5deg)}10
0%{transform:translateY(0)}}
#title{animation:wobble 4.5s ease-in-out infinite}

/* small screen tweaks */
@media (max-width:520px){#title{font-size:36px} .btn{font-size:16px;padding:12px}}

```

FILE 7: main-menu-preview.html
PURPOSE: Canvas menu preview page.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>The Spawn of Chaos ??? Main Menu Preview</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      background: #020105;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      min-height: 100vh;
      font-family: Georgia, 'Times New Roman', serif;
      overflow: hidden;
    }
    h1 {
      color: rgba(140, 135, 175, 0.5);
      font-size: 14px;
      font-weight: normal;
      letter-spacing: 4px;
      text-transform: uppercase;
      margin-bottom: 16px;
    }
    canvas {
      border: 1px solid rgba(120, 60, 200, 0.2);
      box-shadow: 0 0 80px rgba(80, 30, 160, 0.15);
      max-width: 95vw;
      max-height: 80vh;
    }
    .hint {
      color: rgba(100, 95, 130, 0.4);
      font-size: 12px;
      margin-top: 16px;
      font-style: italic;
    }
  </style>
</head>
<body>
  <h1>Menu Preview ??? JavaScript + C# Shared Design</h1>
  <canvas id="menuCanvas" width="1920" height="1080"></canvas>
  <p class="hint">Same visual language as the Unity in-game menu. Click anywhere for chaos b
ursts.</p>

```

```

<script src="Assets/Scenes/scripts/Main Menu/main-menu-effects.js"></script>
<script>
    var menu = SpawnOfChaosMenu.initCanvasMenu('menuCanvas');
    var canvas = document.getElementById('menuCanvas');

    canvas.addEventListener('click', function (e) {
        var rect = canvas.getBoundingClientRect();
        var scaleX = canvas.width / rect.width;
        var scaleY = canvas.height / rect.height;
        var x = (e.clientX - rect.left) * scaleX;
        var y = (e.clientY - rect.top) * scaleY;

        for (var i = 0; i < 12; i++) {
            var angle = Math.random() * Math.PI * 2;
            var speed = 100 + Math.random() * 250;
            var r = 4 + Math.random() * 12;
            // Burst drawn inline for preview interactivity
        }
    });
</script>
</body>
</html>

```

FILE 8: Assets/Scenes/scripts/style.css
PURPOSE: Additional web-style presentation rules.

```

body {
    font-family: sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
}

.container {
    width: 80%;
    margin: 0 auto;
    padding: 20px;
}

/* Header Styles */
header {
    background-color: #333;
    color: #fff;
    padding: 20px 0;
}

header h1 {
    margin: 0;
    font-size: 2.5em;
}

header p {
    margin-bottom: 0;
    font-size: 1.2em;
}

/* Hero Section Styles */
.hero {
    background-color: #e9f7ff;
    padding: 50px 0;
    text-align: center;
}

.hero-content {
    background-color: #fff;
    padding: 30px;
    border-radius: 5px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

.hero h2 {
    font-size: 2em;
    margin-bottom: 20px;
}

```

```

}

.button {
  display: inline-block;
  background-color: #4CAF50;
  color: white;
  padding: 10px 20px;
  text-decoration: none;
  border-radius: 5px;
  font-size: 1.1em;
}

/* Features Section Styles */
.features {
  background-color: #fff;
  padding: 50px 0;
}

.feature-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}

.feature {
  text-align: center;
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 5px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

.feature img {
  max-width: 100%;
  height: auto;
  margin-bottom: 10px;
}

.feature h3 {
  font-size: 1.5em;
  margin-bottom: 10px;
}

/* Testimonials Section Styles */
.testimonials {
  background-color: #fff;
  padding: 50px 0;
  text-align: center;
}

.testimonial-slider {
  display: flex;
  justify-content: center;
  gap: 20px;
}

.testimonial {
  border: 1px solid #ddd;
  border-radius: 5px;
  padding: 20px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

/* Footer Styles */
.footer {
  background-color: #333;
  color: #fff;
  padding: 20px 0;
  text-align: center;
}

/* Media Queries (Responsive Design) */
.container {
  width: 95%;
}

```

```
.hero h2 {
    font-size: 1.8em;
}
```

Unity C#

FILE 9: Assets/Scenes/scripts/IDamageable.cs

PURPOSE: Shared damage interface.

```
public interface IDamageable
{
    /// <summary>
    /// Apply the requested amount of damage to this object.
    /// </summary>
    /// <param name="amount">Damage to apply.</param>
    void TakeDamage(int amount);
}
```

FILE 10: Assets/Scenes/scripts/Main Camera/CameraFollow.cs

PURPOSE: Smooth camera tracking.

```
using UnityEngine;

/// <summary>
/// CameraFollow - Smooth framerate-independent camera tracking script with boundary clamping.
/// Automatically detects and binds to the player across scene transitions.
/// </summary>
public class CameraFollow : MonoBehaviour
{
    [Header("Tracking Target")]
    [Tooltip("Player transform to follow. Auto-detected if unassigned.")]
    public Transform player;

    [Header("Follow Settings")]
    [Tooltip("Camera offset relative to player. Y=0.5f places player in the vertical center of screen.")]
    public Vector3 offset = new Vector3(0f, 0.5f, -10f);
    public float smoothSpeed = 6f;

    [Header("Boundary Clamping")]
    [Tooltip("Enable bounding box to clamp camera within level limits.")]
    public bool useBounds = false;
    public Vector2 minBounds = new Vector2(-10f, -20f);
    public Vector2 maxBounds = new Vector2(300f, 20f);

    private Camera cam;

    void Awake()
    {
        cam = GetComponent<Camera>();
    }

    void Start()
    {
        if (player == null)
        {
            FindPlayer();
        }
    }

    void LateUpdate()
    {
        if (player == null)
        {
            FindPlayer();
            if (player == null) return;
        }

        Vector3 shake = (CameraShakeManager.Instance != null) ? CameraShakeManager.Instance.CurrentShakeOffset : Vector3.zero;
        Vector3 targetPosition = player.position + offset + shake;

        if (useBounds)
```

```

        {
            targetPosition = ClampPositionToBounds(targetPosition);
        }

        // Exponential decay for framerate-independent smooth tracking
        float t = 1f - Mathf.Exp(-smoothSpeed * Time.deltaTime);
        transform.position = Vector3.Lerp(transform.position, targetPosition, t);
    }

    public void FindPlayer()
    {
        if (move.Instance != null)
        {
            player = move.Instance.transform;
        }
        else
        {
            GameObject pObj = GameObject.FindGameObjectWithTag("Player");
            if (pObj != null) player = pObj.transform;
        }

        if (player != null)
        {
            SnapToTarget();
        }
    }

    public void SnapToTarget()
    {
        if (player != null)
        {
            Vector3 pos = player.position + offset;
            if (useBounds)
            {
                pos = ClampPositionToBounds(pos);
            }
            transform.position = pos;
        }
    }

    private Vector3 ClampPositionToBounds(Vector3 target)
    {
        float clampedX = target.x;
        float clampedY = target.y;

        if (cam != null && cam.orthographic)
        {
            float vertExtent = cam.orthographicSize;
            float horzExtent = vertExtent * cam.aspect;

            float minX = minBounds.x + horzExtent;
            float maxX = maxBounds.x - horzExtent;
            float minY = minBounds.y + vertExtent;
            float maxY = maxBounds.y - vertExtent;

            if (minX <= maxX) clampedX = Mathf.Clamp(target.x, minX, maxX);
            else clampedX = (minBounds.x + maxBounds.x) * 0.5f;

            if (minY <= maxY) clampedY = Mathf.Clamp(target.y, minY, maxY);
            else clampedY = (minBounds.y + maxBounds.y) * 0.5f;
        }
        else
        {
            clampedX = Mathf.Clamp(target.x, minBounds.x, maxBounds.x);
            clampedY = Mathf.Clamp(target.y, minBounds.y, maxBounds.y);
        }

        return new Vector3(clampedX, clampedY, target.z);
    }
}

```

FILE 11: Assets/Scenes/scripts/Player/health.cs
 PURPOSE: Player health and damage handling.

using System;

```

using UnityEngine;

public class Health : MonoBehaviour, IDamageable
{
    [Header("Mage Hurt Voice")]
    [Tooltip("Audio clips for Mage taking damage.")]
    public AudioClip[] hurtVoiceClips;
    private AudioSource hurtVoiceSource;

    public int maxHealth = 14;
    private int currentHealth;

    public int CurrentHealth => currentHealth;
    public int MaxHealth => maxHealth;

    public event Action<int> onDamageTaken;
    public event Action<int> onMaxHealthChanged;
    public event Action onDeath;

    private SpriteRenderer playerSpriteRenderer;
    private Coroutine flashCoroutine;
    private Color originalPlayerColor = Color.white;
    private bool hasOriginalPlayerColor = false;

    void Start()
    {
        InitHurtVoice();
        if (CompareTag("Player"))
        {
            maxHealth = 100; // Standard 100% health scale
            playerSpriteRenderer = GetComponentInChildren<SpriteRenderer>();
            if (playerSpriteRenderer != null)
            {
                originalPlayerColor = playerSpriteRenderer.color;
                hasOriginalPlayerColor = true;
            }
        }
        currentHealth = maxHealth;
    }

    /// <summary>
    /// Inflicts damage calculated as a percentage of maximum health (e.g. 25f for 25%).
    /// </summary>
    public void TakeDamagePercent(float percent)
    {
        int damageAmount = Mathf.Max(1, Mathf.RoundToInt(maxHealth * (percent / 100f)));
        TakeDamage(damageAmount);
    }

    public void TakeDamage(int damage)
    {
        // Check if invulnerable (e.g. player is dashing)
        move playerMove = GetComponent<move>();
        if (playerMove != null && playerMove.IsInvulnerable)
        {
            Debug.Log($"{name} is invulnerable (blob/dash)! Ignored damage.");
            return;
        }

        // Check if Lumi's Orb Shield is active (holding 'H')
        if (CompareTag("Player") && LightOrbCompanion.Instance != null && LightOrbCompanion.In
stance.isShieldActive)
        {
            float reduction = LightOrbCompanion.Instance.shieldDamageReduction;
            damage = Mathf.RoundToInt(damage * (1f - reduction));
            Debug.Log($"Lumi's Shield mitigated damage! Reduced to {damage}.");
            if (damage <= 0) return;
        }

        currentHealth -= damage;
        currentHealth = Mathf.Max(currentHealth, 0);
        Debug.Log($"{name} took {damage} damage. Health now {currentHealth}/{maxHealth}.");

        if (CompareTag("Player"))
        {
            PlayRandomHurtVoice();
        }
    }
}

```

```

        if (playerSpriteRenderer != null)
        {
            if (flashCoroutine != null) StopCoroutine(flashCoroutine);
            flashCoroutine = StartCoroutine(FlashPlayerRed());
        }
    }

    EnemyPatrol2D enemy = GetComponent<EnemyPatrol2D>();
    if (enemy != null)
    {
        enemy.TriggerKnockback();
    }

    if (!CompareTag("Player"))
    {
        HitFeedbackManager.TriggerHitFeedback(transform, transform.position, damage, damage >= 25, EnemyHitType.PhysicalMelee);
    }

    onDamageTaken?.Invoke(damage);

    if (currentHealth <= 0)
    {
        // 1-in-3 Luck Cheat-Death (DarkBladeSmall)
        if (CompareTag("Player") && SpawnOfChaos.Weapons.WeaponManager.Instance != null && SpawnOfChaos.Weapons.WeaponManager.Instance.TryTriggerLuckCheatDeath(this))
        {
            return;
        }

        Die();
    }
}

private System.Collections.IEnumerator FlashPlayerRed()
{
    if (playerSpriteRenderer == null) yield break;
    if (!hasOriginalPlayerColor)
    {
        originalPlayerColor = playerSpriteRenderer.color;
        hasOriginalPlayerColor = true;
    }

    playerSpriteRenderer.color = Color.red;
    yield return new WaitForSeconds(hitFlashDuration);
    playerSpriteRenderer.color = originalPlayerColor;
    flashCoroutine = null;
}

private void OnEnable()
{
    UnityEngine.SceneManagement.SceneManager.sceneLoaded += OnSceneLoaded;
}

private void OnDisable()
{
    UnityEngine.SceneManagement.SceneManager.sceneLoaded -= OnSceneLoaded;
    if (playerSpriteRenderer != null && hasOriginalPlayerColor)
    {
        playerSpriteRenderer.color = originalPlayerColor;
    }
}

private void OnSceneLoaded(UnityEngine.SceneManagement.Scene scene, UnityEngine.SceneManagement.LoadSceneMode mode)
{
    if (CompareTag("Player") && (currentHealth <= 0 || PlayerSpawnPointManager.isRespawning))
    {
        Resurrect();
    }
}

[Header("Hit Flash (Player)")]
public float hitFlashDuration = 0.15f;

```

```

public void IncreaseMaxHealth(int amount)
{
    maxHealth += amount;
    currentHealth = Mathf.Min(currentHealth + amount, maxHealth);
    onMaxHealthChanged?.Invoke(maxHealth);
    onDamageTaken?.Invoke(0); // trigger UI redraw
    Debug.Log($"{name} max health increased by {amount}. Max: {maxHealth}, Current: {currentHealth}");
}

public void Heal(int amount)
{
    currentHealth = Mathf.Min(currentHealth + amount, maxHealth);
    onDamageTaken?.Invoke(0); // trigger UI redraw
    Debug.Log($"{name} healed by {amount}. Health now {currentHealth}/{maxHealth}.");
}

public void Resurrect()
{
    currentHealth = maxHealth;

    move playerMove = GetComponent<move>();
    if (playerMove != null) playerMove.enabled = true;

    MageCombat playerCombat = GetComponent<MageCombat>();
    if (playerCombat != null) playerCombat.enabled = true;

    Rigidbody2D rb = GetComponent<Rigidbody2D>();
    if (rb != null)
    {
        rb.isKinematic = false;
        rb.linearVelocity = Vector2.zero;
    }

    Collider2D col = GetComponent<Collider2D>();
    if (col != null) col.enabled = true;

    Debug.Log($"{name} has been resurrected and fully reset.");
    onDamageTaken?.Invoke(0); // redraw UI
}

void Die()
{
    Debug.Log($"{name} died.");
    onDeath?.Invoke();

    if (CompareTag("Player"))
    {
        // Disable movement and combat to prevent input execution after death
        move playerMove = GetComponent<move>();
        if (playerMove != null) playerMove.enabled = false;

        MageCombat playerCombat = GetComponent<MageCombat>();
        if (playerCombat != null) playerCombat.enabled = false;

        Rigidbody2D rb = GetComponent<Rigidbody2D>();
        if (rb != null)
        {
            rb.linearVelocity = Vector2.zero;
            rb.isKinematic = true;
        }

        Collider2D col = GetComponent<Collider2D>();
        if (col != null) col.enabled = false;

        if (RevivalPromptUI.Instance != null)
        {
            RevivalPromptUI.Instance.PromptRevival(this, () => ExecuteStandardDeathRespawn());
        }
        else
        {
            ExecuteStandardDeathRespawn();
        }
    }
    else

```

```

        {
            Destroy(gameObject);
        }
    }

    public void ExecuteStandardDeathRespawn()
    {
        if (SpawnOfChaos.Systems.DrifterSaveManager.Instance != null && SpawnOfChaos.Systems.D
rifterSaveManager.Instance.HasCheckpoint())
        {
            SpawnOfChaos.Systems.DrifterSaveManager.Instance.RespawnAtDeathAnchor();
        }
        else
        {
            StartCoroutine(ReloadSceneRoutine(1.5f));
        }
    }

    private System.Collections.IEnumerator ReloadSceneRoutine(float delay)
    {
        yield return new WaitForSeconds(delay);
        PlayerSpawnPointManager.isRespawning = true;
        UnityEngine.SceneManagement.SceneManager.LoadScene(UnityEngine.SceneManagement.SceneMa
nager.GetActiveScene().name);
    }

    private void InitHurtVoice()
    {
        hurtVoiceSource = gameObject.GetComponent<AudioSource>();
        if (hurtVoiceSource == null)
        {
            hurtVoiceSource = gameObject.AddComponent<AudioSource>();
        }
        hurtVoiceSource.playOnAwake = false;
        hurtVoiceSource.spatialBlend = 0f;

        if (hurtVoiceClips == null || hurtVoiceClips.Length == 0)
        {
            hurtVoiceClips = Resources.LoadAll<AudioClip>("Voice/mage/hurt");
        }
    }

    public void PlayRandomHurtVoice()
    {
        var vc = GetComponent<SpawnOfChaos.Entities.PlayerMageVoiceController>() ?? GetCompone
ntInParent<SpawnOfChaos.Entities.PlayerMageVoiceController>();
        if (vc != null) { vc.PlayHurtVoice(); return; }
        if (hurtVoiceClips == null || hurtVoiceClips.Length == 0) return;
        if (hurtVoiceSource == null) InitHurtVoice();
        AudioClip clip = hurtVoiceClips[UnityEngine.Random.Range(0, hurtVoiceClips.Length)];
        if (clip != null)
        {
            float sfxVol = AudioManager.Instance != null ? AudioManager.Instance.GetRealSFXVol
ume() : 1.0f;
            hurtVoiceSource.PlayOneShot(clip, sfxVol);
        }
    }
}

```

FILE 12: Assets/Scenes/scripts/Player/PlayerCurrency.cs
PURPOSE: Currency storage and updates.

```

using System;
using UnityEngine;

public class PlayerCurrency : MonoBehaviour
{
    public static PlayerCurrency Instance { get; private set; }

    [Header("Currency Data")]
    [SerializeField] private int coins = 0;
    [SerializeField] private int healingPotions = 0;

    public int Coins => coins;
}

```

```

public int HealingPotions => healingPotions;

public event Action<int> onCoinsChanged;
public event Action<int> onPotionsChanged;

void Awake()
{
    if (Instance == null)
    {
        Instance = this;
    }
    else
    {
        Destroy(gameObject);
        return;
    }
}

public void AddCoins(int amount)
{
    if (amount < 0) return;
    coins += amount;
    onCoinsChanged?.Invoke(coins);
    Debug.Log("Player gained " + amount + " coins. Total: " + coins);
}

public bool SpendCoins(int amount)
{
    if (amount < 0) return false;
    if (coins >= amount)
    {
        coins -= amount;
        onCoinsChanged?.Invoke(coins);
        Debug.Log("Player spent " + amount + " coins. Total: " + coins);
        return true;
    }
    return false;
}

public void AddPotions(int count)
{
    if (count < 0) return;
    healingPotions += count;
    onPotionsChanged?.Invoke(healingPotions);
    Debug.Log("Player gained " + count + " healing potions. Total: " + healingPotions);
}

public bool UsePotion()
{
    if (healingPotions > 0)
    {
        Health playerHealth = GetComponent<Health>();
        if (playerHealth != null)
        {
            // If already at full health, we might want to prevent use or just consume it.
            Let's heal 40 health points.
            if (playerHealth.CurrentHealth >= playerHealth.MaxHealth)
            {
                Debug.Log("Player is already at full health!");
                return false;
            }

            healingPotions--;
            onPotionsChanged?.Invoke(healingPotions);
            playerHealth.Heal(4);
            Debug.Log("Player consumed a potion. Remaining: " + healingPotions);
            return true;
        }
    }
    return false;
}

void Update()
{
    // Consuming potion on H key press
    if (Input.GetKeyDown(KeyCode.H))

```

```

        {
            // Ignore key while typing in any input fields
            if (UnityEngine.EventSystems.EventSystem.current != null && UnityEngine.EventSystem.current.currentSelectedGameObject != null)
            {
                var go = UnityEngine.EventSystems.EventSystem.current.currentSelectedGameObject;
                if (go.GetComponent<TMPPro.TMP_InputField>() != null || go.GetComponent<UnityEngine.UI.InputField>() != null)
                {
                    return;
                }
            }

            // Ignore key if dialogue, shop, or chat panels are open
            if (NPCDialogueUI.Instance != null && NPCDialogueUI.Instance.IsDialogueActive) return;
            if (ShopUI.Instance != null && ShopUI.Instance.IsShopActive) return;
            if (NyxarisManager.IsChatActive) return;

            UsePotion();
        }
    }
}

```

FILE 13: Assets/Scenes/scripts/Player/PlayerFootstepAudio.cs
PURPOSE: Footstep sound playback.

```

using UnityEngine;

/// <summary>
/// Procedural footstep and landing audio system for the player character.
/// Generates warm, cushioned, organic footstep sounds mathematically via code (no clicky pops).
/// Features realistic heel-toe double-contact, low-pass filtered shoe friction, and dynamic walk/run cadences.
/// </summary>
[AddComponentMenu("Player/Player Footstep Audio")]
[RequireComponent(typeof(AudioSource))]
public class PlayerFootstepAudio : MonoBehaviour
{
    public enum FootwearType
    {
        [Tooltip("Soft, cushioned athletic shoes / ninja cloth wraps (warm, muted, zero click)")]
        SoftShoe,
        [Tooltip("Standard travel boots with solid ground thud and subtle leather scuff.")]
        AdventurerBoots,
        [Tooltip("Deep, grounded bass step for heavy or armored movement.")]
        HeavyBoots,
        [Tooltip("Light, swift footsteps with slight gravel / dust friction.")]
        LightStep
    }

    [Header("Footwear & Tone Settings")]
    [Tooltip("Choose the type of footwear and ground resonance.")]
    public FootwearType footwearStyle = FootwearType.SoftShoe;

    [Range(0f, 1f)]
    [Tooltip("Volume multiplier for footstep playback.")]
    public float footstepVolume = 0.5f;

    [Range(0f, 1f)]
    [Tooltip("Volume multiplier for landing thuds.")]
    public float landingVolume = 0.75f;

    [Range(0.5f, 1.8f)]
    [Tooltip("Base pitch tone of footsteps (lower = deeper bass thump, higher = lighter step).")]
    public float stepTonePitch = 0.95f;

    [Range(0f, 1f)]
    [Tooltip("Softness/cushioning of the shoe sole. Higher values eliminate all harsh edges for a warm pad sound.")]
    public float soleCushion = 0.8f;
}

```

```

[Range(0f, 1f)]
[Tooltip("Amount of muted ground friction / shoe scuff texture in each step.")]
public float surfaceScuff = 0.35f;

[Header("Step Timing & Cadence")]
[Tooltip("Time between footsteps while walking.")]
public float walkStepInterval = 0.36f;

[Tooltip("Time between footsteps while running/dashing.")]
public float runStepInterval = 0.22f;

[Tooltip("Minimum horizontal velocity required to trigger footsteps.")]
public float minVelocityThreshold = 0.2f;

[Header("Organic Variations")]
[Range(0f, 0.25f)]
[Tooltip("Random pitch variation per step to keep it natural and organic.")]
public float pitchRandomness = 0.06f;

[Range(0f, 0.2f)]
[Tooltip("Random volume variation per step.")]
public float volumeRandomness = 0.04f;

[Header("Custom Clip Overrides (Optional)")]
[Tooltip("Optional custom audio clips. If left empty, warm procedural clips are generated automatically.")]
public AudioClip[] customFootstepClips;
public AudioClip customLandingClip;

// Component References
private AudioSource audioSource;
private Rigidbody2D rb;
private move playerMove;
private bool wasGroundedLastFrame = true;

// Internal State
private float stepTimer = 0f;
private AudioClip[] proceduralClips;
private AudioClip proceduralLandingClip;
private const int CLIP_VARIATION_COUNT = 6;

void Awake()
{
    audioSource = GetComponent<AudioSource>();
    if (audioSource == null) audioSource = gameObject.AddComponent<AudioSource>();

    audioSource.playOnAwake = false;
    audioSource.loop = false;
    audioSource.spatialBlend = 0f; // 2D clean audio

    rb = GetComponent<Rigidbody2D>();
    playerMove = GetComponent<move>();

    if (customFootstepClips == null || customFootstepClips.Length == 0)
    {
        GenerateProceduralAudioClips();
    }
}

void Update()
{
    if (playerMove == null) playerMove = GetComponent<move>();
    if (rb == null) rb = GetComponent<Rigidbody2D>();

    bool isGrounded = IsPlayerGrounded();

    // 1. Landing detection
    if (isGrounded && !wasGroundedLastFrame)
    {
        PlayLandingSound();
        stepTimer = walkStepInterval * 0.45f;
    }
    wasGroundedLastFrame = isGrounded;

    // 2. Footstep cadence tracking

```

```

        if (isGrounded)
        {
            float horizontalSpeed = Mathf.Abs(rb != null ? rb.linearVelocity.x : 0f);
            float inputX = Mathf.Abs(Input.GetAxisRaw("Horizontal"));

            if (horizontalSpeed > minVelocityThreshold || inputX > 0.1f)
            {
                bool isRunning = (playerMove != null && playerMove.moveSpeed > 7f) || horizontalSpeed > 7f || (playerMove != null && playerMove.IsDashing);
                float currentInterval = isRunning ? runStepInterval : walkStepInterval;

                stepTimer -= Time.deltaTime;
                if (stepTimer <= 0f)
                {
                    PlayFootstep();
                    stepTimer = currentInterval;
                }
            }
            else
            {
                stepTimer = Mathf.Min(stepTimer, 0.05f);
            }
        }
        else
        {
            stepTimer = 0.08f;
        }
    }

    private bool IsPlayerGrounded()
    {
        if (playerMove != null && rb != null && Mathf.Abs(rb.linearVelocity.y) < 0.05f)
        {
            return true;
        }

        Collider2D col = GetComponent<Collider2D>();
        Vector2 origin = col != null ? (Vector2)col.bounds.center + Vector2.down * (col.bounds.extents.y - 0.05f) : (Vector2)transform.position;
        RaycastHit2D hit = Physics2D.Raycast(origin, Vector2.down, 0.25f);
        return hit.collider != null && hit.collider.gameObject != gameObject;
    }

    /// <summary>
    /// Plays a single footstep with organic randomized pitch and volume.
    /// </summary>
    public void PlayFootstep()
    {
        AudioClip clipToPlay = GetRandomFootstepClip();
        if (clipToPlay == null) return;

        float randomPitch = stepTonePitch + Random.Range(-pitchRandomness, pitchRandomness);
        float randomVol = Mathf.Clamp01(footstepVolume + Random.Range(-volumeRandomness, volumeRandomness));

        audioSource.pitch = randomPitch;
        audioSource.PlayOneShot(clipToPlay, randomVol);
    }

    /// <summary>
    /// Plays a deep landing impact sound.
    /// </summary>
    public void PlayLandingSound()
    {
        AudioClip clip = (customLandingClip != null) ? customLandingClip : proceduralLandingClip;
        if (clip == null) return;

        audioSource.pitch = Random.Range(0.92f, 1.04f);
        audioSource.PlayOneShot(clip, landingVolume);
    }

    private AudioClip GetRandomFootstepClip()
    {
        if (customFootstepClips != null && customFootstepClips.Length > 0)
        {

```

```

        return customFootstepClips[Random.Range(0, customFootstepClips.Length)];
    }

    if (proceduralClips != null && proceduralClips.Length > 0)
    {
        return proceduralClips[Random.Range(0, proceduralClips.Length)];
    }

    return null;
}

#region Organic Procedural Synthesis

/// <summary>
/// Synthesizes warm, cushioned, realistic footsteps with heel-toe contact and zero digita
l clicks.
/// </summary>
public void GenerateProceduralAudioClips()
{
    proceduralClips = new AudioClip[CLIP_VARIATION_COUNT];
    int sampleRate = 44100;

    for (int i = 0; i < CLIP_VARIATION_COUNT; i++)
    {
        // Duration around 110ms - 130ms (enough for full heel-toe acoustic resonance)
        float duration = 0.11f + (i * 0.005f);
        int totalSamples = Mathf.FloorToInt(sampleRate * duration);
        float[] samples = new float[totalSamples];

        // Base resonant frequencies according to style
        float baseStartFreq, baseEndFreq, scuffIntensity, thudDecay;
        switch (footwearStyle)
        {
            case FootwearType.AdventurerBoots:
                baseStartFreq = 145f + (i * 6f);
                baseEndFreq = 58f;
                scuffIntensity = 0.40f * surfaceScuff;
                thudDecay = 24f;
                break;
            case FootwearType.HeavyBoots:
                baseStartFreq = 120f + (i * 4f);
                baseEndFreq = 42f;
                scuffIntensity = 0.30f * surfaceScuff;
                thudDecay = 18f;
                break;
            case FootwearType.LightStep:
                baseStartFreq = 165f + (i * 8f);
                baseEndFreq = 70f;
                scuffIntensity = 0.50f * surfaceScuff;
                thudDecay = 30f;
                break;
            case FootwearType.SoftShoe:
            default:
                baseStartFreq = 135f + (i * 5f);
                baseEndFreq = 50f;
                scuffIntensity = 0.28f * surfaceScuff;
                thudDecay = 26f;
                break;
        }

        // Low-pass filter state for warm friction texture
        float filteredNoise = 0f;
        float filterCoeff = Mathf.Lerp(0.12f, 0.04f, soleCushion); // Heavier lowpass filt
er for softer cushion

        float heelPhase = 0f;
        float toePhase = 0f;
        float toeTimeOffset = 0.024f + (i * 0.002f); // Heel touches, then ball of foot to
uches ~24ms later

        for (int s = 0; s < totalSamples; s++)
        {
            float timeInSec = (float)s / sampleRate;

            // --- 1. HEEL CONTACT (Light initial thump) ---
            float heelEnvelope = 0f;

```

```

float heelWave = 0f;
if (timeInSec < 0.05f)
{
    float tHeel = timeInSec / 0.05f;
    // Smooth attack (4ms) to prevent any click, followed by rapid exponential
    decay
        float attack = Mathf.SmoothStep(0f, 1f, timeInSec / 0.005f);
        heelEnvelope = attack * Mathf.Exp(-tHeel * thudDecay) * 0.45f;

        float heelFreq = Mathf.Lerp(baseStartFreq * 1.15f, baseEndFreq, Mathf.Pow(
tHeel, 0.4f));
        heelPhase += (2f * Mathf.PI * heelFreq) / sampleRate;
        heelWave = Mathf.Sin(heelPhase) * heelEnvelope;
    }

    // --- 2. MAIN FOOT THUD (Toe / Ball of foot pad) ---
    float toeEnvelope = 0f;
    float toeWave = 0f;
    if (timeInSec >= toeTimeOffset)
    {
        float tToe = (timeInSec - toeTimeOffset) / (duration - toeTimeOffset);
        float attackToe = Mathf.SmoothStep(0f, 1f, (timeInSec - toeTimeOffset) / 0
.007f);
        toeEnvelope = attackToe * Mathf.Exp(-tToe * thudDecay * 0.9f);

        float toeFreq = Mathf.Lerp(baseStartFreq, baseEndFreq, Mathf.Pow(tToe, 0.5
f));
        toePhase += (2f * Mathf.PI * toeFreq) / sampleRate;

        // Rich fundamental + warm second sub-harmonic (adds earthy warmth)
        toeWave = (Mathf.Sin(toePhase) * 0.75f + Mathf.Sin(toePhase * 0.5f) * 0.25
f) * toeEnvelope;
    }

    // --- 3. WARM LOW-PASS SCUFF / FRICTION (Zero harsh clicks) ---
    float rawWhiteNoise = Random.value * 2f - 1f;
    // Single-pole IIR low-pass filter (simulates cloth/rubber sole dragging again
st ground)
    filteredNoise = filteredNoise + filterCoeff * (rawWhiteNoise - filteredNoise);

    float totalEnvelope = Mathf.Max(heelEnvelope, toeEnvelope);
    float scuffComponent = filteredNoise * scuffIntensity * totalEnvelope * 3.5f;

    // --- 4. COMBINE & SOFT-CLIP ---
    float combined = heelWave + toeWave + scuffComponent;

    // Soft tanh saturation for organic acoustic warmth
    samples[s] = (float)System.Math.Tanh(combined * 1.2f);
}

AudioClip clip = AudioClip.Create($"Organic_Step_{footwearStyle}_{i}", totalSample
s, 1, sampleRate, false);
clip.SetData(samples, 0);
proceduralClips[i] = clip;
}

// --- LANDING THUD ---
{
    float landDuration = 0.20f;
    int landSamples = Mathf.FloorToInt(sampleRate * landDuration);
    float[] lSamples = new float[landSamples];
    float phase = 0f;
    float landFilteredNoise = 0f;

    for (int s = 0; s < landSamples; s++)
    {
        float timeInSec = (float)s / sampleRate;
        float t = timeInSec / landDuration;

        float attack = Mathf.SmoothStep(0f, 1f, timeInSec / 0.008f);
        float envelope = attack * Mathf.Exp(-t * 13f);

        float freq = Mathf.Lerp(130f, 38f, Mathf.Pow(t, 0.35f));
        phase += (2f * Mathf.PI * freq) / sampleRate;

        float rawNoise = Random.value * 2f - 1f;

```

```

        landFilteredNoise += 0.08f * (rawNoise - landFilteredNoise);

        float body = (Mathf.Sin(phase) * 0.7f + Mathf.Sin(phase * 0.5f) * 0.3f);
        float noise = landFilteredNoise * 0.4f;

        lSamples[s] = (float)System.Math.Tanh((body + noise) * envelope * 1.3f);
    }

    proceduralLandingClip = AudioClip.Create($"Organic_Landing_{footwearStyle}", lands
amples, 1, sampleRate, false);
    proceduralLandingClip.SetData(lSamples, 0);
}

#endregion

#region Editor Tools & Preview

[ContextMenu("Regenerate & Test Play Footstep")]
public void EditorTestPlayFootstep()
{
    GenerateProceduralAudioClips();
    if (audioSource == null) audioSource = GetComponent<AudioSource>();
    if (audioSource != null)
    {
        PlayFootstep();
    }
}

[ContextMenu("Regenerate & Test Play Landing")]
public void EditorTestPlayLanding()
{
    GenerateProceduralAudioClips();
    if (audioSource == null) audioSource = GetComponent<AudioSource>();
    if (audioSource != null)
    {
        PlayLandingSound();
    }
}

#endregion
}

```

FILE 14: Assets/Scenes/scripts/Coin Drop Prefab/CoinItem.cs
PURPOSE: Coin pickup behavior.

```

using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class CoinItem : MonoBehaviour
{
    [Header("Coin Properties")]
    public int coinValue = 10;

    [Header("Magnetism Settings")]
    public float magnetRadius = 4.0f;
    public float magnetSpeed = 8.0f;

    private Transform playerTransform;
    private bool isMagnetizing = false;
    private Rigidbody2D rb;

    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        GetComponent<Collider2D>().isTrigger = true;
    }

    void Update()
    {
        if (playerTransform == null && PlayerCurrency.Instance != null)
        {
            playerTransform = PlayerCurrency.Instance.transform;
        }
    }
}

```

```

        if (playerTransform != null)
        {
            float distance = Vector2.Distance(transform.position, playerTransform.position);

            if (distance <= magnetRadius)
            {
                isMagnetizing = true;
            }

            if (isMagnetizing)
            {
                // Disable gravity if rigid body is present to allow smooth float towards play
er
                if (rb != null)
                {
                    rb.gravityScale = 0f;
                    rb.linearVelocity = Vector2.zero;
                }

                // Move towards player
                transform.position = Vector3.MoveTowards(transform.position, playerTransform.p
osition, magnetSpeed * Time.deltaTime);
            }
        }

        private void OnTriggerEnter2D(Collider2D other)
        {
            if (other.CompareTag("Player"))
            {
                PlayerCurrency pc = other.GetComponent<PlayerCurrency>();
                if (pc == null)
                {
                    pc = other.GetComponentInParent<PlayerCurrency>();
                }

                if (pc != null)
                {
                    pc.AddCoins(coinValue);
                    // Optional: Spawn coin pickup particle or sound here
                    Destroy(gameObject);
                }
            }
        }
    }
}

```

FILE 15: Assets/Scenes/scripts/Cloud Spawner/Cloud.cs
 PURPOSE: Moving cloud behavior.

```

using UnityEngine;

public class Cloud : MonoBehaviour
{
    [Header("Movement Points")]
    public Transform pointA;
    public Transform pointB;

    [Header("Settings")]
    public float speed = 2f;

    private Vector3 target;
    private bool movingToB = true;
    private bool playerOnCloud = false;
    private bool waitingAtPoint = true;

    private Transform playerTransform;

    private void Start()
    {
        if (pointA == null || pointB == null)
        {
            Debug.LogError("Cloud requires pointA and pointB to be assigned.");
            enabled = false;
            return;
        }
    }
}

```

```

        target = pointB.position;
        movingToB = true;
        waitingAtPoint = true;
    }

    private void Update()
    {
        if (pointA == null || pointB == null)
            return;

        if (waitingAtPoint)
        {
            if (!playerOnCloud)
                return;

            waitingAtPoint = false;
        }

        Vector3 currentPosition = transform.position;
        Vector3 newPosition = Vector3.MoveTowards(
            currentPosition,
            target,
            speed * Time.deltaTime
        );

        Vector3 delta = newPosition - currentPosition;
        transform.position = newPosition;

        // Move player along with the cloud if they are standing on it
        if (playerOnCloud && playerTransform != null)
        {
            playerTransform.position += delta;
        }

        if (Vector3.Distance(transform.position, target) < 0.05f)
        {
            transform.position = target;
            movingToB = !movingToB;
            target = movingToB ? pointB.position : pointA.position;
            waitingAtPoint = true;
            return;
        }
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (!collision.gameObject.CompareTag("Player"))
            return;

        playerOnCloud = true;
        playerTransform = collision.transform;
    }

    private void OnCollisionExit2D(Collision2D collision)
    {
        if (!collision.gameObject.CompareTag("Player"))
            return;

        playerOnCloud = false;
        if (playerTransform == collision.transform)
        {
            playerTransform = null;
        }
    }
}

```

FILE 16: Assets/Scenes/scripts/Hazards/DamageZone.cs
 PURPOSE: Damage-causing area trigger.

```

using UnityEngine;

/// <summary>
/// DamageZone - Trigger zone that deals damage-over-time (DPS) to the player while inside.
/// Used as spike pits, lava pools, or poison clouds beneath parkour sections.
/// Optionally teleports the player to a respawn point on entry.

```

```

/// </summary>
public class DamageZone : MonoBehaviour
{
    [Header("Damage Settings")]
    [Tooltip("Damage dealt per second while the player is inside the zone.")]
    public float damagePerSecond = 25f;

    [Tooltip("Damage tick interval in seconds.")]
    public float tickInterval = 0.3f;

    [Tooltip("Knockback force applied upward on each tick (to bounce the player).")]
    public float knockbackUpForce = 8f;

    [Header("Respawn Teleport (Optional)")]
    [Tooltip("If assigned, the player is teleported here when entering the zone.")]
    public Transform respawnPoint;

    [Tooltip("Flat damage dealt instantly on entry before respawn teleport.")]
    public int entryDamage = 30;

    [Header("Visual")]
    public Color zoneColor = new Color(1f, 0.2f, 0.1f, 0.2f);

    private float nextTickTime;
    private bool playerInside;
    private Rigidbody2D playerRb;
    private Health playerHealth;

    void Start()
    {
        Collider2D col = GetComponent<Collider2D>();
        if (col != null) col.isTrigger = true;
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (!IsPlayer(other)) return;

        playerRb = other.GetComponent<Rigidbody2D>();
        playerHealth = other.GetComponent<Health>() ?? other.GetComponentInParent<Health>();

        // If respawn point is set, teleport the player immediately + deal flat entry damage
        if (respawnPoint != null)
        {
            if (playerHealth != null)
            {
                playerHealth.TakeDamage(entryDamage);
            }

            // Teleport player
            other.transform.position = respawnPoint.position;
            if (playerRb != null) playerRb.linearVelocity = Vector2.zero;
            playerInside = false;
            return;
        }

        playerInside = true;
        nextTickTime = Time.time;
    }

    void OnTriggerExit2D(Collider2D other)
    {
        if (!IsPlayer(other)) return;
        playerInside = false;
        playerRb = null;
        playerHealth = null;
    }

    void Update()
    {
        if (!playerInside || playerHealth == null) return;

        if (Time.time >= nextTickTime)
        {
            int tickDamage = Mathf.CeilToInt(damagePerSecond * tickInterval);
            playerHealth.TakeDamage(tickDamage);
        }
    }
}

```

```

        // Bounce player upward to give them a chance to escape
        if (playerRb != null)
        {
            playerRb.linearVelocity = new Vector2(playerRb.linearVelocity.x, knockbackUpForce);
        }

        nextTickTime = Time.time + tickInterval;
    }

    private bool IsPlayer(Collider2D col)
    {
        return col.CompareTag("Player") || col.GetComponent<move>() != null;
    }

    void OnDrawGizmos()
    {
        Gizmos.color = zoneColor;
        Collider2D col = GetComponent<Collider2D>();
        if (col is BoxCollider2D box)
        {
            Vector3 center = transform.position + (Vector3)box.offset;
            Vector3 size = new Vector3(box.size.x * transform.lossyScale.x, box.size.y * transform.lossyScale.y, 0.1f);
            Gizmos.DrawCube(center, size);

            // Draw X pattern for danger
            Gizmos.color = new Color(1f, 0f, 0f, 0.6f);
            Vector3 halfSize = size * 0.5f;
            Gizmos.DrawLine(center - halfSize, center + halfSize);
            Gizmos.DrawLine(center + new Vector3(-halfSize.x, halfSize.y, 0f), center + new Vector3(halfSize.x, -halfSize.y, 0f));
        }

        // Draw respawn point indicator
        if (respawnPoint != null)
        {
            Gizmos.color = Color.green;
            Gizmos.DrawWireSphere(respawnPoint.position, 0.5f);
            Gizmos.DrawLine(transform.position, respawnPoint.position);
        }
    }
}

```

FILE 17: Assets/Scenes/scripts/Popup Text Prefabs/FloatingDamageNumber.cs
PURPOSE: Floating damage text.

```

using System.Collections;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// FloatingDamageNumber - Dynamic floating damage text popup.
/// Spawns damage popups in world-space, animating pop scale, upward float, and smooth fade out.
/// </summary>
public class FloatingDamageNumber : MonoBehaviour
{
    private TextMesh textMesh;
    private CanvasGroup canvasGroup;
    private Text uiText;

    private Vector3 velocity;
    private float lifetime = 0.65f;
    private float elapsed = 0f;
    private Vector3 initialScale;

    /// <summary>
    /// Factory method to spawn a floating damage number in world space.
    /// </summary>
    public static FloatingDamageNumber Spawn(Vector3 worldPos, int damageAmount, bool isHeavyHit = false, Color? customColor = null)
    {
        {

```

```

        GameObject popObj = new GameObject($"DamagePopup_{damageAmount}");
        popObj.transform.position = worldPos + new Vector3(Random.Range(-0.3f, 0.3f), Random.Range(0.2f, 0.5f), -1f);

        FloatingDamageNumber pop = popObj.AddComponent<FloatingDamageNumber>();
        pop.Initialize(damageAmount, isHeavyHit, customColor);
        return pop;
    }

    public static FloatingDamageNumber SpawnText(Vector3 worldPos, string message, Color color)
    {
        GameObject popObj = new GameObject("TextPopup");
        popObj.transform.position = worldPos + new Vector3(0f, 0.5f, -1f);

        FloatingDamageNumber pop = popObj.AddComponent<FloatingDamageNumber>();
        pop.InitializeText(message, color);
        return pop;
    }

    private void InitializeText(string message, Color color)
    {
        textMesh = gameObject.AddComponent<TextMesh>();
        textMesh.text = message;
        textMesh.characterSize = 0.12f;
        textMesh.fontSize = 24;
        textMesh.fontStyle = FontStyle.Bold;
        textMesh.alignment = TextAlignment.Center;
        textMesh.anchor = TextAnchor.MiddleCenter;
        textMesh.color = color;

        Font defaultFont = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        if (defaultFont == null) defaultFont = Font.CreateDynamicFontFromOSFont("Arial", 24);
        if (defaultFont != null)
        {
            textMesh.font = defaultFont;
            MeshRenderer mr = GetComponent<MeshRenderer>();
            if (mr != null)
            {
                mr.material = defaultFont.material;
                mr.sortingOrder = 50;
            }
        }

        initialScale = Vector3.one;
        transform.localScale = initialScale * 0.4f;
        velocity = new Vector3(0f, 1.8f, 0f);
        lifetime = 2.5f;

        StartCoroutine(AnimateRoutine());
    }

    private void Initialize(int damageAmount, bool isHeavyHit, Color? customColor)
    {
        textMesh = gameObject.AddComponent<TextMesh>();
        textMesh.text = damageAmount.ToString();
        textMesh.characterSize = 0.15f;
        textMesh.fontSize = isHeavyHit ? 36 : 28;
        textMesh.fontStyle = FontStyle.Bold;
        textMesh.alignment = TextAlignment.Center;
        textMesh.anchor = TextAnchor.MiddleCenter;

        // Use built-in font supported in Unity 6
        Font defaultFont = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        if (defaultFont == null)
        {
            defaultFont = Font.CreateDynamicFontFromOSFont("Arial", 28);
        }

        if (defaultFont != null)
        {
            textMesh.font = defaultFont;
            MeshRenderer mr = GetComponent<MeshRenderer>();
            if (mr != null)
            {
                mr.material = defaultFont.material;
            }
        }
    }

```

```

        mr.sortingOrder = 50; // Render above sprites
    }
}

// Color coding
if (customColor.HasValue)
{
    textMesh.color = customColor.Value;
}
else if (isHeavyHit)
{
    textMesh.color = new Color(1.0f, 0.25f, 0.1f); // Vibrant fiery orange-red for heavy hit
}
else
{
    textMesh.color = new Color(1.0f, 0.92f, 0.3f); // Bright yellow for normal hit
}

// Pop scale and float velocity setup
float baseScale = isHeavyHit ? 1.3f : 1.0f;
initialScale = new Vector3(baseScale, baseScale, baseScale);
transform.localScale = initialScale * 0.4f; // Start small for pop effect

velocity = new Vector3(Random.Range(-0.8f, 0.8f), Random.Range(2.2f, 3.2f), 0f);

StartCoroutine(AnimateRoutine());
}

private IEnumerator AnimateRoutine()
{
    while (elapsed < lifetime)
    {
        float dt = Time.unscaledDeltaTime; // Unscaled time so hitstop doesn't freeze damage text movement
        elapsed += dt;
        float progress = elapsed / lifetime;

        // 1. Move upwards with deceleration
        transform.position += velocity * dt;
        velocity.y = Mathf.Lerp(velocity.y, 0.5f, dt * 4f);

        // 2. Pop scale effect (scale up quickly in first 20%, then settle)
        if (progress < 0.2f)
        {
            float popT = progress / 0.2f;
            transform.localScale = Vector3.Lerp(initialScale * 0.4f, initialScale * 1.35f, popT);
        }
        else if (progress < 0.4f)
        {
            float settleT = (progress - 0.2f) / 0.2f;
            transform.localScale = Vector3.Lerp(initialScale * 1.35f, initialScale, settleT);
        }

        // 3. Fade out in second half
        if (progress > 0.4f)
        {
            float alpha = Mathf.Clamp01(1f - ((progress - 0.4f) / 0.6f));
            if (textMesh != null)
            {
                Color c = textMesh.color;
                c.a = alpha;
                textMesh.color = c;
            }
        }

        yield return null;
    }

    Destroy(gameObject);
}
}

```

FILE 18: Assets/Scenes/scripts/Popup Text Prefabs/MoveAndFade.cs
PURPOSE: Popup movement and fading.

```
using UnityEngine;
using System.Collections;

public class MoveAndFade : MonoBehaviour
{
    public float moveSpeed = 0.1f;
    public float fadeDuration = 5f;
    public float waitBeforeRespawn = 10f;
    public Vector3 startPosition; // Store the initial position

    private Renderer rend;

    private void Start()
    {
        // Save the starting position
        startPosition = transform.position;
        rend = GetComponent<Renderer>();

        StartCoroutine(LoopRoutine());
    }

    private IEnumerator LoopRoutine()
    {
        while (true) // Infinite loop
        {
            // 1. Reset Position and Opacity
            transform.position = startPosition;
            Color color = rend.material.color;
            color.a = 1f;
            rend.material.color = color;

            // Ensure object is visible
            gameObject.SetActive(true);

            // 2. Move Logic
            while (transform.position.y > 0)
            {
                transform.position += Vector3.down * moveSpeed;
                yield return null;
            }

            // 3. Fade Logic
            float timer = 0;
            while (timer < fadeDuration)
            {
                timer += Time.deltaTime;
                float alpha = Mathf.Clamp01(1 - (timer / fadeDuration));
                color.a = alpha;
                rend.material.color = color;
                yield return null;
            }

            // 4. Wait for 10 seconds before looping back
            yield return new WaitForSeconds(waitBeforeRespawn);
        }
    }
}
```

FILE 19: Assets/Scenes/scripts/Level Goal/LevelGoalTrigger.cs
PURPOSE: Detects level completion.

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
using TMPro;
using System.Collections;

[RequireComponent(typeof(BoxCollider2D))]
public class LevelGoalTrigger : MonoBehaviour
{
    [Header("Scene Navigation")]
    [Tooltip("Scene to load when player clicks 'NEXT LEVEL'.")]
    public string nextLevelSceneName = "SampleScene";
}
```

```

[Tooltip("Scene to load when player clicks 'MAIN MENU'.")]
public string mainMenuSceneName = "MainMenu";

[Tooltip("Button text for proceeding to the next chapter.")]
public string nextLevelButtonText = "JOURNEY TO CHERRY BLOSSOM FOREST";

[Header("Nyxaris Level Complete Celebration")]
[Tooltip("Dialogue spoken by Nyxaris when completing this level.")]
[TextArea(2, 5)]
public string victoryDialogueText = "Impressive, mortal! You've mastered the fundamentals
and survived the fractured rift. Beyond lies the Cherry Blossom Forest???where the true tria
l begins!";

[Tooltip("Expression animation for Nyxaris on level complete (e.g. excited, happy, confide
ntly, proud).")]
public string victoryAnimationKey = "excited";

[Header("Audio Settings")]
[Tooltip("Victory audio jingle to play when level is complete.")]
public AudioClip victorySFX;

private bool isTriggered = false;
private AudioSource audioSource;
private AudioClip speechChirpClip;

void Start()
{
    Collider2D col = GetComponent<Collider2D>();
    if (col != null)
    {
        col.isTrigger = true;
    }

    SynthesizeAudio();
}

private void SynthesizeAudio()
{
    audioSource = gameObject.AddComponent<AudioSource>();
    audioSource.playOnAwake = false;
    audioSource.loop = false;
    audioSource.spatialBlend = 0f;

    int rate = 44100;
    int chirpSamples = Mathf.FloorToInt(rate * 0.045f);
    float[] cSamples = new float[chirpSamples];
    for (int i = 0; i < chirpSamples; i++)
    {
        float t = (float)i / chirpSamples;
        float freq = Mathf.Lerp(540f, 400f, t);
        float env = Mathf.Sin(t * Mathf.PI);
        cSamples[i] = Mathf.Sin(2f * Mathf.PI * freq * (i / (float)rate)) * env * 0.4f;
    }
    speechChirpClip = AudioClip.Create("Nyxaris_VictoryChirp", chirpSamples, 1, rate, false);
    speechChirpClip.SetData(cSamples, 0);
}

void OnTriggerEnter2D(Collider2D other)
{
    if (isTriggered) return;

    if (other.CompareTag("Player") || other.name.Contains("Player") || other.GetComponent<
move>() != null)
    {
        TriggerVictory(other.gameObject);
    }
}

private void TriggerVictory(GameObject player)
{
    isTriggered = true;
    Debug.Log("[LevelGoalTrigger] Goal reached! Level Complete with Nyxaris celebration.");
};

```

```
// Disable player controls  
move playerMove = player.GetComponent<move>() ?? player.GetComponentInParent<move>();  
if (playerMove != null)  
{  
    playerMove.enabled = false;  
}  
  
Rigidbody2D rb = player.GetComponent<Rigidbody2D>() ?? player.GetComponentInParent<Rigibody2D>();  
if (rb != null)  
{  
    rb.linearVelocity = Vector2.zero;  
}  
  
// Play victory sound  
if (victorySFX != null && AudioManager.Instance != null)  
{  
    AudioManager.Instance.PlaySFX(victorySFX);  
}  
  
// Build celebratory Victory UI  
BuildVictoryUI();  
}  
  
// ??????????????????????????????????????????????????????????????????????????????????????????????????????????????  
// ???????????????????????????????????????????????????????????????????????????????????????????????????????????  
// ?????????????????????  
// VICTORY UI WITH ANIMATED NYXARIS PORTRAIT & CELEBRATION SPEECH  
// ??????????????????????????????????????????????????????????????????????????????????????????????????????????  
// ??????????????????????????????????????????????????????????????????????????????????????????????????????????  
// ?????????????????????  
  
private void BuildVictoryUI()  
{  
    Cursor.visible = true;  
    Cursor.lockState = CursorLockMode.None;  
    StartCoroutine(VictoryInputRoutine());  
    // ?????? Canvas (sort order 600 ??? above everything) ??????  
    Canvas canvas = UIFactory.CreateCanvas("VictoryCanvas", 600f);  
  
    // ?????? Full-screen dark celestial overlay ??????  
    RectTransform overlay = UIFactory.CreateFullScreenPanel(  
        canvas.transform, "Overlay", new Color(0.04f, 0.01f, 0.08f, 0.92f)  
    );  
  
    // ?????? '??? LEVEL COMPLETE ???' Title ??????  
    TextMeshProUGUI titleText = UIFactory.CreateText(  
        overlay, "TitleText", "??? LEVEL COMPLETE ???",  
        46f, new Color(1f, 0.85f, 0.35f, 1f), TextAlignOptions.Center  
    );  
    UIFactory.SetRectFixed(titleText.rectTransform,  
        new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),  
        new Vector2(0f, 190f), new Vector2(700f, 65f)  
    );  
    titleText.fontStyle = FontStyles.Bold;  
    titleText.enableWordWrapping = false;  
  
    // ?????? Subtitle ??????  
    TextMeshProUGUI subText = UIFactory.CreateText(  
        overlay, "SubTitle", "Tutorial Chapter Cleared",  
        20f, new Color(0.85f, 0.6f, 1f, 0.9f), TextAlignOptions.Center  
    );  
    UIFactory.SetRectFixed(subText.rectTransform,  
        new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),  
        new Vector2(0f, 150f), new Vector2(500f, 35f)  
    );  
  
    // ?????? Nyxaris Dialogue Card (Glassmorphic Obsidian) ??????  
    GameObject cardGO = new GameObject("NyxarisDialogueCard");  
    cardGO.transform.SetParent(overlay, false);  
    RectTransform cardRT = cardGO.AddComponent<RectTransform>();  
    cardRT.anchorMin = new Vector2(0.5f, 0.5f);  
    cardRT.anchorMax = new Vector2(0.5f, 0.5f);  
    cardRT.sizeDelta = new Vector2(800f, 180f);  
    cardRT.anchoredPosition = new Vector2(0f, 20f);
```

```

Image cardBg = cardGO.AddComponent<Image>();
cardBg.color = new Color(0.07f, 0.02f, 0.14f, 0.95f);

Outline cardOutline = cardGO.AddComponent<Outline>();
cardOutline.effectColor = new Color(0.85f, 0.35f, 1f, 0.95f);
cardOutline.effectDistance = new Vector2(2.5f, -2.5f);

// ?????? Nyxaris Animated Portrait ??????
GameObject portraitGO = new GameObject("NyxarisPortrait");
portraitGO.transform.SetParent(cardGO.transform, false);
RectTransform portraitRT = portraitGO.AddComponent<RectTransform>();
portraitRT.sizeDelta = new Vector2(140f, 140f);
portraitRT.anchoredPosition = new Vector2(-305f, 0f);

Image portraitImg = portraitGO.AddComponent<Image>();
portraitImg.preserveAspect = true;
portraitImg.color = Color.white;

NyxarisFrameAnimator frameAnim = portraitGO.AddComponent<NyxarisFrameAnimator>();
frameAnim.targetImage = portraitImg;
frameAnim.PlayAnimation(victoryAnimationKey, 14f);

Outline portOutline = portraitGO.AddComponent<Outline>();
portOutline.effectColor = new Color(1f, 0.8f, 0.3f, 0.95f);
portOutline.effectDistance = new Vector2(2f, -2f);

// ?????? Nyxaris Name Label ??????
TextMeshProUGUI nameLabel = UIFactory.CreateText(
    cardRT, "NameLabel", "NYXARIS",
    20f, new Color(0.95f, 0.45f, 1f, 1f), TextAlignmentOptions.TopLeft
);
UIFactory.SetRectFixed(nameLabel.rectTransform,
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),
    new Vector2(55f, 50f), new Vector2(520f, 30f)
);
nameLabel.fontStyle = FontStyles.Bold;

// ?????? Dialogue Text with Typewriter Effect ??????
TextMeshProUGUI dialogueComp = UIFactory.CreateText(
    cardRT, "DialogueText", "",
    21f, Color.white, TextAlignmentOptions.TopLeft
);
UIFactory.SetRectFixed(dialogueComp.rectTransform,
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),
    new Vector2(55f, -15f), new Vector2(520f, 95f)
);
dialogueComp.enableWordWrapping = true;

StartCoroutine(TypewriterRoutine(dialogueComp, victoryDialogueText));

// ?????? Button Container ??????
RectTransform btnContainer = UIFactory.CreatePanel(
    overlay, "ButtonContainer", Color.clear,
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),
    Vector2.zero, Vector2.zero
);
UIFactory.SetRectFixed(btnContainer,
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f),
    new Vector2(0f, -125f), new Vector2(760f, 60f)
);
UIFactory.AddHorizontalLayout(btnContainer.gameObject, 30f,
    new RectOffset(0, 0, 0, 0), TextAnchor.MiddleCenter);

// ?????? Next Level Button (loads SampleScene e.g. Cherry Blossom Forest) ??????
Button nextBtn = UIFactory.CreateButton(
    btnContainer, "NextLevelButton", nextLevelButtonText,
    18f, () => LoadNextLevel(nextLevelSceneName)
);
UIFactory.AddLayoutElement(nextBtn.gameObject, preferredHeight: 52f, preferredWidth: 4
00f);

// ?????? Main Menu Button ??????
Button menuBtn = UIFactory.CreateButton(
    btnContainer, "MainMenuButton", "MAIN MENU",
    18f, () => QuitToMainMenu(mainMenuSceneName)
);

```

```

        UIFactory.AddLayoutElement(menuBtn.gameObject, preferredHeight: 52f, preferredWidth: 2
20f);
    }

    private IEnumerator TypewriterRoutine(TextMeshProUGUI label, string text)
    {
        label.text = "";
        for (int i = 0; i < text.Length; i++)
        {
            label.text += text[i];

            if (char.IsLetterOrDigit(text[i]) && audioSource != null && speechChirpClip != nul
l && (i % 2 == 0))
            {
                audioSource.pitch = Random.Range(1.35f, 1.75f);
                audioSource.PlayOneShot(speechChirpClip, 0.45f);
            }

            yield return new WaitForSecondsRealtime(0.024f);
        }
    }

    // ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????
    // SCENE NAVIGATION
    // ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????

    private IEnumerator VictoryInputRoutine()
    {
        yield return new WaitForSecondsRealtime(0.5f);
        while (true)
        {
            if (Input.GetKeyDown(KeyCode.Space) || Input.GetKeyDown(KeyCode.Return) || Input.G
etKeyDown(KeyCode.KeypadEnter) || Input.GetKeyDown(KeyCode.E))
            {
                LoadNextLevel(nextLevelSceneName);
                yield break;
            }
            yield return null;
        }
    }

    public void LoadNextLevel(string sceneName)
    {
        if (!string.IsNullOrEmpty(sceneName))
        {
            Time.timeScale = 1f;
            SpawnOfChaos.Systems.ArcaneLoadingScreen.LoadScene(sceneName);
        }
    }

    public void QuitToMainMenu(string menuScene)
    {
        if (!string.IsNullOrEmpty(menuScene))
        {
            Time.timeScale = 1f;
            SceneManager.LoadScene(menuScene);
        }
    }
}

```

FILE 20: Assets/Scenes/scripts/NPC/NPCFacePlayer.cs
PURPOSE: Makes NPCs face the player.

```

using UnityEngine;

public class NPCFacePlayer : MonoBehaviour
{
    [Header("Detection Settings")]
    [Tooltip("Distance at which the NPC will start facing the player.")]
    public float faceRange = 5f;
}

```

```

[Header("Flipping Method")]
[Tooltip("If checked, it flips the SpriteRenderer.flipX. If unchecked, it flips the Transform's localScale.x (warning: might mirror child UI text).")]
public bool useSpriteRendererFlip = true;

[Tooltip("Does the NPC sprite face right by default in the original asset?")]
public bool spriteFacesRightByDefault = true;

[Tooltip("Check this if the NPC turns away from the player instead of facing them.")]
public bool invertFlipping = false;

private Transform playerTransform;
private SpriteRenderer spriteRenderer;
private bool facingRight;

void Start()
{
    // Find the player automatically in the scene
    GameObject player = GameObject.FindGameObjectWithTag("Player");
    if (player != null)
    {
        playerTransform = player.transform;
    }

    spriteRenderer = GetComponent<SpriteRenderer>();
    if (spriteRenderer == null && useSpriteRendererFlip)
    {
        // If sprite renderer is missing, fallback to localScale flip
        useSpriteRendererFlip = false;
        Debug.LogWarning($"NPCFacePlayer on {gameObject.name}: No SpriteRenderer found, falling back to Transform local scale flip.");
    }

    // Initialize starting direction
    facingRight = spriteFacesRightByDefault;
}

void Update()
{
    if (playerTransform == null) return;

    float distance = Vector2.Distance(transform.position, playerTransform.position);

    if (distance <= faceRange)
    {
        // Check if player is to the right or left of the NPC
        float direction = playerTransform.position.x - transform.position.x;

        if (direction > 0.1f && !facingRight)
        {
            SetFacing(true);
        }
        else if (direction < -0.1f && facingRight)
        {
            SetFacing(false);
        }
    }
}

private void SetFacing(bool faceRight)
{
    if (invertFlipping)
    {
        faceRight = !faceRight;
    }

    facingRight = faceRight;

    if (useSpriteRendererFlip && spriteRenderer != null)
    {
        // If the sprite faces right by default, we flipX when we want to face left
        spriteRenderer.flipX = spriteFacesRightByDefault ? !faceRight : faceRight;
    }
    else
    {
        // Flip using Transform local scale

```

```

        Vector3 scale = transform.localScale;
        // Force the sign of local scale to match the direction
        float absoluteScaleX = Mathf.Abs(scale.x);

        if (spriteFacesRightByDefault)
        {
            scale.x = faceRight ? absoluteScaleX : -absoluteScaleX;
        }
        else
        {
            scale.x = faceRight ? -absoluteScaleX : absoluteScaleX;
        }

        transform.localScale = scale;
    }
}

```

FILE 21: Assets/Scenes/scripts/NPC/NPCInteractable.cs
 PURPOSE: Basic NPC interaction.

```

using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class NPCInteractable : MonoBehaviour
{
    [Header("NPC Configuration")]
    public string npcName = "Villager";
    [TextArea(3, 5)]
    public string[] dialogueLines;

    [Header("Custom Character Dialogue")]
    [Tooltip("If assigned, this NPC will use this custom dialogue asset instead of the raw files above.")]
    public CharacterDialogueAsset dialogueAsset;

    [Header("Special Triggers")]
    public bool isNyxaris = false;
    public bool isShopKeeper = false;

    [Header("UI Prompts")]
    [Tooltip("Visual prompt GameObject like 'Press E to talk' (optional)")]
    public GameObject interactPrompt;

    private bool playerInRange = false;

    void Awake()
    {
        // Force the Collider2D to be a trigger
        GetComponent<Collider2D>().isTrigger = true;

        if (interactPrompt != null)
        {
            interactPrompt.SetActive(false);
        }
    }

    void Update()
    {
        if (NyxarisManager.IsTyping || NyxarisManager.IsChatActive) return;

        if (playerInRange)
        {
            // Check if player presses interaction key E
            if (Input.GetKeyDown(KeyCode.E))
            {
                Interact();
            }
        }
    }

    public void Interact()
    {
        // Hide interaction prompt while talking
        if (interactPrompt != null)
    }
}

```

```

{
    interactPrompt.SetActive(false);
}

if (isNyxaris)
{
    if (NyxarisManager.Instance != null)
    {
        NyxarisManager.Instance.ShowInterface();
    }
    else
    {
        Debug.LogWarning("NyxarisManager not found in scene!");
    }
}
else if (isShopKeeper)
{
    // First display greeting, then open shop
    if (dialogueAsset != null && NPCDialogueUI.Instance != null)
    {
        NPCDialogueUI.Instance.ShowDialogue(dialogueAsset, () =>
        {
            if (ShopUI.Instance != null)
            {
                ShopUI.Instance.OpenShop();
            }
        });
    }
    else if (dialogueLines != null && dialogueLines.Length > 0 && NPCDialogueUI.Instance != null)
    {
        NPCDialogueUI.Instance.ShowDialogue(npcName, dialogueLines, () =>
        {
            if (ShopUI.Instance != null)
            {
                ShopUI.Instance.OpenShop();
            }
        });
    }
    else if (ShopUI.Instance != null)
    {
        ShopUI.Instance.OpenShop();
    }
}
else
{
    // Normal dialogue NPC
    if (NPCDialogueUI.Instance != null)
    {
        if (dialogueAsset != null)
        {
            NPCDialogueUI.Instance.ShowDialogue(dialogueAsset, () =>
            {
                // Dialogue complete callback - show prompt again if player is still in range
                if (playerInRange && interactPrompt != null)
                {
                    interactPrompt.SetActive(true);
                }
            });
        }
        else
        {
            NPCDialogueUI.Instance.ShowDialogue(npcName, dialogueLines, () =>
            {
                // Dialogue complete callback - show prompt again if player is still in range
                if (playerInRange && interactPrompt != null)
                {
                    interactPrompt.SetActive(true);
                }
            });
        }
    }
    else
    {

```

```

        Debug.LogWarning("NPCDialogueUI not found in scene!");
    }
}

private void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        playerInRange = true;
        if (interactPrompt != null && !IsAnyDialogueOrShopOpen())
        {
            interactPrompt.SetActive(true);
        }
    }
}

private void OnTriggerExit2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        playerInRange = false;
        if (interactPrompt != null)
        {
            interactPrompt.SetActive(false);
        }

        // Automatically close dialogue if player walks away
        if (NPCDialogueUI.Instance != null && NPCDialogueUI.Instance.IsDialogueActive)
        {
            NPCDialogueUI.Instance.CloseDialogue();
        }

        // Automatically close shop if player walks away
        if (ShopUI.Instance != null && ShopUI.Instance.IsShopActive)
        {
            ShopUI.Instance.CloseShop();
        }
    }
}

private bool IsAnyDialogueOrShopOpen()
{
    bool dialogueActive = (NPCDialogueUI.Instance != null && NPCDialogueUI.Instance.IsDialogueActive);
    bool shopActive = (ShopUI.Instance != null && ShopUI.Instance.IsShopActive);
    bool nyxarisActive = NyxarisManager.IsChatActive;
    return dialogueActive || shopActive || nyxarisActive;
}
}

```

FILE 22: Assets/Scenes/scripts/Dojo Entrance/DojoTransitionFader.cs
PURPOSE: Scene transition fade effect.

```

using System.Collections;
using UnityEngine;
using UnityEngine.UI;

public class DojoTransitionFader : MonoBehaviour
{
    public static DojoTransitionFader Instance { get; private set; }

    private Image overlay;

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
    static void ResetStatic()
    {
        Instance = null;
    }

    public static DojoTransitionFader Ensure()
    {
        if (Instance != null)
        {
            return Instance;
        }
    }
}

```

```

    }

    GameObject faderObject = new GameObject("DojoTransitionFader");
    DontDestroyOnLoad(faderObject);
    return faderObject.AddComponent<DojoTransitionFader>();
}

private void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }

    Instance = this;
    DontDestroyOnLoad(gameObject);
    BuildOverlay();
}

private void BuildOverlay()
{
    Canvas canvas = UIFactory.CreateCanvas("DojoTransitionCanvas", 60);
    canvas.transform.SetParent(transform, false);

    RectTransform overlayRect = UIFactory.CreateFullscreenPanel(canvas.transform, "DojoFad
eOverlay", Color.clear);
    overlay = overlayRect.GetComponent<Image>();
    overlay.raycastTarget = false;
}

public IEnumerator FadeOutIn(float fadeOutDuration, float holdDuration, float fadeInDurati
on, System.Action onHidden)
{
    yield return FadeTo(1f, fadeOutDuration);
    onHidden?.Invoke();
    if (holdDuration > 0f)
    {
        yield return new WaitForSecondsRealtime(holdDuration);
    }
    yield return FadeTo(0f, fadeInDuration);
}

public void PlayFadeOutIn(float fadeOutDuration, float holdDuration, float fadeInDuration,
System.Action onHidden, System.Action onComplete)
{
    StartCoroutine(FadeOutInRoutine(fadeOutDuration, holdDuration, fadeInDuration, onHidde
n, onComplete));
}

private IEnumerator FadeOutInRoutine(float fadeOutDuration, float holdDuration, float fade
InDuration, System.Action onHidden, System.Action onComplete)
{
    yield return FadeOutIn(fadeOutDuration, holdDuration, fadeInDuration, onHidden);
    onComplete?.Invoke();
}

private IEnumerator FadeTo(float targetAlpha, float duration)
{
    if (overlay == null)
    {
        yield break;
    }

    Color color = overlay.color;
    float startAlpha = color.a;
    float elapsed = 0f;

    if (duration <= 0f)
    {
        color.a = targetAlpha;
        overlay.color = color;
        yield break;
    }

    while (elapsed < duration)

```

```

        {
            elapsed += Time.unscaledDeltaTime;
            color.a = Mathf.Lerp(startAlpha, targetAlpha, elapsed / duration);
            overlay.color = color;
            yield return null;
        }

        color.a = targetAlpha;
        overlay.color = color;
        overlay.raycastTarget = targetAlpha > 0.01f;
    }
}

```

FILE 23: Assets/Scenes/scripts/Dojo Entrance/DojoDoorTransition.cs
PURPOSE: Dojo door access and transition.

```

using System.Collections;
using TMPPro;
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class DojoDoorTransition : MonoBehaviour
{
    [Header("Door")]
    public string doorName = "Clan Dojo";
    public Transform destinationPoint;
    public bool requiresDojoCleared;
    public DojoWaveManager requiredDojoWave;

    [Header("Prompt")]
    public GameObject interactPrompt;
    public string availablePromptText = "Press E to enter";
    public string lockedPromptText = "Clear the dojo first";
    public bool createPromptIfMissing = true;
    public Vector3 promptOffset = new Vector3(0f, 1.4f, 0f);
    public float promptBobAmount = 0.08f;
    public float promptBobSpeed = 3f;

    [Header("Transition")]
    public float preFadeDelay = 0.35f;
    public float fadeOutDuration = 0.35f;
    public float hiddenHoldDuration = 0.2f;
    public float fadeInDuration = 0.35f;
    public string turnInTrigger = "TurnIn";
    public string arriveTrigger = "ArriveFromDoor";
    public AudioClip transitionSFX;

    [Header("Area Switching")]
    public GameObject[] enableOnArrival;
    public GameObject[] disableOnArrival;

    [Header("Scene Loading (Optional Upgrade)")]
    public bool loadScene = false;
    public string targetSceneName = "";
    public string spawnPointName = "";

    private bool playerInRange;
    private bool isTransitioning;
    private GameObject playerObject;
    private Vector3 promptStartLocalPosition;

    private void Awake()
    {
        Collider2D doorCollider = GetComponent<Collider2D>();
        doorCollider.isTrigger = true;

        if (interactPrompt == null && createPromptIfMissing)
        {
            interactPrompt = CreateRuntimePrompt();
        }

        if (interactPrompt != null)
        {
            promptStartLocalPosition = interactPrompt.transform.localPosition;
            interactPrompt.SetActive(false);
        }
    }
}

```

```

    }
}

private void Update()
{
    AnimatePrompt();

    if (!playerInRange || isTransitioning || playerObject == null)
    {
        return;
    }

    RefreshPromptText();

    if (Input.GetKeyDown(KeyCode.E) && !IsPlayerUiBlocked())
    {
        if (IsLocked())
        {
            return;
        }

        StartCoroutine(EnterDoorRoutine(playerObject));
    }
}

private IEnumerator EnterDoorRoutine(GameObject player)
{
    isTransitioning = true;
    SetPromptVisible(false);
    SetPlayerControl(player, false);

    Rigidbody2D rb = player.GetComponent<Rigidbody2D>();
    if (rb != null)
    {
        rb.linearVelocity = Vector2.zero;
    }

    Animator animator = player.GetComponent<Animator>();
    SetAnimatorTriggerIfPresent(animator, turnInTrigger);

    if (transitionSFX != null && AudioManager.Instance != null)
    {
        AudioManager.Instance.PlaySFX(transitionSFX);
    }

    if (preFadeDelay > 0f)
    {
        yield return new WaitForSeconds(preFadeDelay);
    }

    DojoTransitionFader fader = DojoTransitionFader.Ensure();
    fader.PlayFadeOutIn(fadeOutDuration, hiddenHoldDuration, fadeInDuration, () =>
    {
        if (loadScene)
        {
            PlayerSpawnPointManager.targetSpawnPointName = spawnPointName;
            UnityEngine.SceneManagement.SceneManager.LoadScene(targetSceneName);
        }
        else
        {
            SwitchAreaObjects();

            if (destinationPoint != null)
            {
                player.transform.position = destinationPoint.position;
            }
            else
            {
                Debug.LogWarning($"{name}: destinationPoint is not assigned for {doorName}");
            }
        }
    }, () =>
    {
        SetAnimatorTriggerIfPresent(animator, arriveTrigger);
        SetPlayerControl(player, true);
    });
}

```

```

        isTransitioning = false;
    });

    yield break;
}

private GameObject CreateRuntimePrompt()
{
    GameObject prompt = new GameObject("DojoInteractPrompt");
    prompt.transform.SetParent(transform, false);
    prompt.transform.localPosition = promptOffset;

    TextMeshPro text = prompt.AddComponent<TextMeshPro>();
    text.text = availablePromptText;
    text.fontSize = 3.2f;
    text.alignment = TextAlignmentOptions.Center;
    text.color = Color.white;
    text.enableWordWrapping = false;

    MeshRenderer renderer = prompt.GetComponent<MeshRenderer>();
    if (renderer != null)
    {
        renderer.sortingOrder = 50;
    }

    return prompt;
}

private void AnimatePrompt()
{
    if (interactPrompt == null || !interactPrompt.activeSelf)
    {
        return;
    }

    Vector3 bob = Vector3.up * (Mathf.Sin(Time.time * promptBobSpeed) * promptBobAmount);
    interactPrompt.transform.localPosition = promptStartLocalPosition + bob;
}

private void RefreshPromptText()
{
    if (interactPrompt == null)
    {
        return;
    }

    string promptText = IsLocked() ? lockedPromptText : availablePromptText;
    TextMeshPro text = interactPrompt.GetComponent<TextMeshPro>();
    if (text != null)
    {
        text.text = promptText;
    }
}

private void SwitchAreaObjects()
{
    SetObjectsActive(enableOnArrival, true);
    SetObjectsActive(disableOnArrival, false);
}

private void SetObjectsActive(GameObject[] objects, bool active)
{
    if (objects == null)
    {
        return;
    }

    foreach (GameObject obj in objects)
    {
        if (obj != null)
        {
            obj.SetActive(active);
        }
    }
}

```

```

private void SetPlayerControl(GameObject player, bool enabled)
{
    move movement = player.GetComponent<move>();
    if (movement != null)
    {
        movement.enabled = enabled;
    }

    MageCombat combat = player.GetComponent<MageCombat>();
    if (combat != null)
    {
        combat.enabled = enabled;
    }
}

private bool IsLocked()
{
    return requiresDojoCleared && requiredDojoWave != null && !requiredDojoWave.IsChallengeCompleted;
}

private bool IsPlayerUiBlocked()
{
    if (PauseMenu.Instance != null && PauseMenu.Instance.isPaused) return true;
    if (NPCDialogueUI.Instance != null && NPCDialogueUI.Instance.IsDialogueActive) return true;
    if (ShopUI.Instance != null && ShopUI.Instance.IsShopActive) return true;
    if (NyxarisManager.IsChatActive) return true;
    return false;
}

private void SetAnimatorTriggerIfPresent(Animator animator, string triggerName)
{
    if (animator == null || string.IsNullOrEmpty(triggerName))
    {
        return;
    }

    foreach (AnimatorControllerParameter parameter in animator.parameters)
    {
        if (parameter.type == AnimatorControllerParameterType.Trigger && parameter.name == triggerName)
        {
            animator.SetTrigger(triggerName);
            return;
        }
    }
}

private void SetPromptVisible(bool visible)
{
    if (interactPrompt != null)
    {
        interactPrompt.SetActive(visible);
    }
}

private void OnTriggerEnter2D(Collider2D other)
{
    if (!other.CompareTag("Player"))
    {
        return;
    }

    playerInRange = true;
    playerObject = other.gameObject;
    RefreshPromptText();
    SetPromptVisible(true);
}

private void OnTriggerExit2D(Collider2D other)
{
    if (!other.CompareTag("Player"))
    {
        return;
    }
}

```

```

        playerInRange = false;
        playerObject = null;
        SetPromptVisible(false);
    }
}

```

FILE 24: Assets/Scenes/scripts/NPC/SpeechBubbleDialogue.cs
PURPOSE: NPC speech bubble dialogue.

```

using System.Collections;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

/// <summary>
/// ?????????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????
/// ??? SPEECH BUBBLE DIALOGUE ??? Drag-and-Drop System ???
/// ?????????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????
/// ??? HOW TO USE ???
/// ??? ?????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????????????????????????????????????????????????????????????
/// ???
/// ??? 1. Attach this script to any NPC GameObject. ???
/// ??? 2. Expand "Dialogue Lines" in the Inspector. ???
/// ??? Write each line of dialogue ??? one entry per bubble page. ???
/// ??? 3. (Optional) Set a Character Name, colours, size, etc. ???
/// ??? 4. Press Play. Walk near the NPC. Press E. Done. ???
/// ???
/// ??? REQUIRES NOTHING ELSE ???
/// ??? ?????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????????????????????????????????????????????????????????????
/// ???
/// ??? ??? Builds its own Canvas, bubble, text and button via code. ???
/// ??? ??? Bubble is World-Space and parented to the NPC, so it ???
/// ??? follows automatically with zero extra scripting. ???
/// ??? ??? Player is located by the "Player" tag automatically. ???
/// ??? ??? Purple interaction gizmo shown in Scene view. ???
/// ?????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????????????????????????????????????????????????????????????
/// ?????????????????????????
/// </summary>
[AddComponentMenu("Dialogue/Speech Bubble Dialogue")]
public class SpeechBubbleDialogue : MonoBehaviour
{
    // ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????
    // INSPECTOR ??? fill these in; everything else is automatic
    // ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????

    [Header("?????? Dialogue Lines ?????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????")]
    [Tooltip("Each entry = one page of the speech bubble.")]
    [TextArea(2, 8)]
    public string[] dialogueLines = { "Hello, stranger. What brings you here?" };

    [Header("?????? Character (optional) ?????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????")]
    [Tooltip("Name shown at the top of the bubble. Leave blank to hide.")]
    public string characterName = "";

    [Tooltip("Colour for the name label.")]
    public Color nameColor = new Color(0.74f, 0.52f, 1f, 1f);

    [Header("?????? Shopkeeper Integration ?????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????")]
    [Tooltip("If true, this NPC functions as a shopkeeper. Approaching shows '[E] Shop' and fi
nishing/pressing trade opens the Shop UI.")]

```

[illegible]

```

// ?????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????
????????????????????????????
// UNITY LIFECYCLE
// ?????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????
????????????????????????????

private void Awake()
{
    if (GetComponent<ShopUI>() != null)
    {
        isShopKeeper = true;
    }
    BuildUI();
}

private void Start()
{
    // Auto-locate the player by tag ??? no manual assignment needed
    var playerGo = GameObject.FindGameObjectWithTag("Player");
    if (playerGo != null) _playerTf = playerGo.transform;

    // Assign world-space camera now that the scene is loaded
    if (_canvas != null && Camera.main != null)
        _canvas.worldCamera = Camera.main;

    SetBubbleActive(false);
    SetPromptActive(false);
}

private void Update()
{
    UpdateProximity();
    HandleInput();
    PulseAdvanceButton();
}

[Header("?????? Combat / Cutscene Lock ??????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????")]
[Tooltip("If true, interaction prompt and key inputs are completely disabled (e.g. during
active combat).")]
public bool interactionDisabled = false;

// ?????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????
????????????????????????????
// PROXIMITY CHECK
// ?????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????
????????????????????????????

private void UpdateProximity()
{
    if (interactionDisabled || (DojoWaveManager.Instance != null && DojoWaveManager.Instance.IsChallengeStarted))
    {
        SetPromptActive(false);
        return;
    }

    if (ShopUI.Instance != null && ShopUI.Instance.IsShopActive)
    {
        SetPromptActive(false);
        if (_isOpen) SetBubbleActive(false);
        return;
    }

    if (_playerTf == null)
    {
        var pGo = GameObject.FindGameObjectWithTag("Player");
        if (pGo != null) _playerTf = pGo.transform;
        if (_playerTf == null) return;
    }
}

```

[illegible]

```

        OnPageChanged?.Invoke(_lineIndex);
    }

    private void AdvanceLine()
    {
        _lineIndex++;
        if (_lineIndex < dialogueLines.Length)
        {
            TypeLine(dialogueLines[_lineIndex]);
            OnPageChanged?.Invoke(_lineIndex);
        }
        else
            CloseDialogue(true);
    }

    public void CloseDialogue(bool openShopIfShopKeeper = true)
    {
        if (_typeRoutine != null) StopCoroutine(_typeRoutine);
        _isOpen = false;
        _typing = false;
        SetBubbleActive(false);
        if (_playerNear && !(ShopUI.Instance != null && ShopUI.Instance.IsShopActive))
            SetPromptActive(true);

        OnDialogueComplete?.Invoke();

        if (openShopIfShopKeeper && isShopKeeper && ShopUI.Instance != null)
        {
            ShopUI.Instance.OpenShop();
        }
    }

    public void OpenShopDirectly()
    {
        if (_typeRoutine != null) StopCoroutine(_typeRoutine);
        _isOpen = false;
        _typing = false;
        SetBubbleActive(false);
        SetPromptActive(false);
        if (ShopUI.Instance != null)
        {
            ShopUI.Instance.OpenShop();
        }
    }

    private void TypeLine(string line)
    {
        if (_typeRoutine != null) StopCoroutine(_typeRoutine);
        string processed = SpawnOfChaos.Systems.TelepathyOrbSystem.ProcessDialogueLine(line, false);
        _typeRoutine = StartCoroutine(TypeRoutine(processed));
    }

    private IEnumerator TypeRoutine(string line)
    {
        _typing = true;
        _bodyText.text = "";
        SetAdvanceActive(false);

        foreach (char c in line)
        {
            _bodyText.text += c;
            yield return new WaitForSeconds(typeSpeed);
        }

        _typing = false;
        SetAdvanceActive(true);
    }

    private void SkipTyping()
    {
        if (_typeRoutine != null) StopCoroutine(_typeRoutine);
        _bodyText.text = SpawnOfChaos.Systems.TelepathyOrbSystem.ProcessDialogueLine(dialogueLines[_lineIndex], false);
        _typing = false;
        SetAdvanceActive(true);
    }

```

```

}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????
// BUTTON ANIMATION
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????

private void PulseAdvanceButton()
{
    if (_advanceBtn == null || !_advanceBtn.gameObject.SetActive()) return;
    float s = 1f + Mathf.Sin(Time.time * 5.5f) * 0.07f;
    _advanceBtn.transform.localScale = new Vector3(s, s, 1f);
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????
// VISIBILITY HELPERS
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????

private void SetBubbleActive(bool on)
{
    if (_bubbleRt != null) _bubbleRt.gameObject.SetActive(on);
    if (!on) SetAdvanceActive(false);
}

private void SetAdvanceActive(bool on)
{
    if (_advanceBtn != null) _advanceBtn.SetActive(on);
}

private void SetPromptActive(bool on)
{
    if (_promptGo != null) _promptGo.SetActive(on);
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????
// UI CONSTRUCTION ??? everything built here from zero
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????

private void BuildUI()
{
    // ?????? World-Space Canvas ?????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    // Parented to this NPC ??? follows it for free.
    var canvasGo = new GameObject("[SpeechBubble]");
    canvasGo.transform.SetParent(transform, false);

    _canvas = canvasGo.AddComponent<Canvas>();
    _canvas.renderMode = RenderMode.WorldSpace;
    _canvas.sortingOrder = 50; // always on top of 2-D sprites

    canvasGo.AddComponent<GraphicRaycaster>();

    // 100 px = 1 Unity world unit; canvas size in pixels
    const float PPU = 100f;
    var canvasRt = canvasGo.GetComponent<RectTransform>();
    canvasRt.sizeDelta = new Vector2(900f, 600f);
    canvasRt.localScale = Vector3.one / PPU;
    canvasRt.localPosition = new Vector3(0f, heightAboveNPC, -0.1f);

    // ?????? Bubble panel ?????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????
    ??????
    var panelGo = new GameObject("BubblePanel", typeof(RectTransform));
    panelGo.transform.SetParent(canvasGo.transform, false);

```

```

_bubbleRt = panelGo.GetComponent<RectTransform>();
_bubbleRt.anchorMin = new Vector2(0.5f, 0.5f);
_bubbleRt.anchorMax = new Vector2(0.5f, 0.5f);
_bubbleRt.pivot = new Vector2(0.5f, 0.5f);
_bubbleRt.sizeDelta = new Vector2(bubbleWidth, bubbleHeight);
_bubbleRt.anchoredPosition = Vector2.zero;

var panelImg = panelGo.AddComponent<Image>();
panelImg.color = bubbleColor;
panelImg.sprite = GetRoundedSprite();
panelImg.type = Image.Type.Sliced;

// ?????? Bubble tail (rotated square ??? bottom-left area) ??????????????????????
????????????????????????????????????
// Overlapping the panel edge creates the classic speech-bubble pointer.
var tailGo = MakeChild("Tail", panelGo.transform);
var tailRt = tailGo.GetComponent<RectTransform>();
tailRt.anchorMin = new Vector2(0.18f, 0f);
tailRt.anchorMax = new Vector2(0.18f, 0f);
tailRt.pivot = new Vector2(0.5f, 0.5f);
tailRt.sizeDelta = new Vector2(36f, 36f);
tailRt.anchoredPosition = new Vector2(0f, -15f); // half-overlap below panel
tailRt.localRotation = Quaternion.Euler(0f, 0f, 45f);
tailGo.AddComponent<Image>().color = bubbleColor;

// ?????? Character name label (top-left inside bubble) ??????????????????????
????????????????????????????????????
float bodyTopOffset = -22f; // how much body text is pushed from top
if (!string.IsNullOrEmpty(characterName))
{
    var nameLGo = MakeChild("NameLabel", panelGo.transform);
    var nrt = nameLGo.GetComponent<RectTransform>();
    nrt.anchorMin = new Vector2(0f, 1f);
    nrt.anchorMax = new Vector2(1f, 1f);
    nrt.pivot = new Vector2(0f, 1f);
    nrt.sizeDelta = new Vector2(0f, 46f);
    nrt.anchoredPosition = new Vector2(28f, -10f);

    var nlTxt = nameLGo.AddComponent<TextMeshProUGUI>();
    ApplyTMPDefaults(nlTxt);
    nlTxt.text = characterName;
    nlTxt.color = nameColor;
    nlTxt.fontSize = 28f;
    nlTxt.fontStyle = FontStyles.Bold;
    nlTxt.enableWordWrapping = false;

    bodyTopOffset = -54f; // body text clears the name row
}

// ?????? Body text ?????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????
????????????????
var bodyGo = MakeChild("BodyText", panelGo.transform);
var brt = bodyGo.GetComponent<RectTransform>();
brt.anchorMin = Vector2.zero;
brt.anchorMax = Vector2.one;
brt.offsetMin = new Vector2(28f, 32f); // left, bottom padding
brt.offsetMax = new Vector2(-76f, bodyTopOffset); // right (clears button), top

_bodyText = bodyGo.AddComponent<TextMeshProUGUI>();
ApplyTMPDefaults(_bodyText);
_bodyText.text = "";
_bodyText.color = textColor;
_bodyText.fontSize = fontSize;
_bodyText.enableWordWrapping = true;
_bodyText.overflowMode = TextOverflowModes.Truncate;

// ?????? Advance button (circle, bottom-right of panel) ??????????????????????
????????????????????????????????
_advanceBtn = MakeChild("AdvanceBtn", panelGo.transform);
var art = _advanceBtn.GetComponent<RectTransform>();
art.anchorMin = new Vector2(1f, 0f);
art.anchorMax = new Vector2(1f, 0f);
art.pivot = new Vector2(1f, 0f);
art.sizeDelta = new Vector2(56f, 56f);
art.anchoredPosition = new Vector2(-14f, 14f);

```

```
var btnImg = _advanceBtn.AddComponent<Image>();  
btnImg.color = buttonColor;  
btnImg.sprite = MakeCircleSprite(64);  
  
// Arrow label inside button  
var arrowGo = MakeChild("Arrow", _advanceBtn.transform);  
var arrRt = arrowGo.GetComponent<RectTransform>();  
arrRt.anchorMin = Vector2.zero;  
arrRt.anchorMax = Vector2.one;  
arrRt.sizeDelta = Vector2.zero;  
arrRt.anchoredPosition = new Vector2(3f, 0f); // optical centering  
  
var arrTxt = arrowGo.AddComponent<TextMeshProUGUI>();  
ApplyTMPDefaults(arrTxt);  
arrTxt.text = "???" ;  
arrTxt.color = new Color(0.92f, 1f, 0.88f, 1f);  
arrTxt.fontSize = 26f;  
arrTxt.alignment = TextAlignmentOptions.Center;  
  
_advanceBtn.SetActive(false);  
  
// ?????? "Press E to talk" prompt ??????????????????????????????????????????????????????????  
????????????????????????????????????????????????????????????????????????????????????  
// Appears below centre of the canvas when player is in range.  
_promptGo = MakeChild("Prompt", canvasGo.transform);  
var prt = _promptGo.GetComponent<RectTransform>();  
prt.anchorMin = new Vector2(0.5f, 0f);  
prt.anchorMax = new Vector2(0.5f, 0f);  
prt.pivot = new Vector2(0.5f, 0.5f);  
prt.sizeDelta = new Vector2(220f, 46f);  
prt.anchoredPosition = new Vector2(0f, 35f); // sits just below the bubble canvas  
  
var promptImg = _promptGo.AddComponent<Image>();  
promptImg.color = new Color(0.06f, 0.04f, 0.14f, 0.88f);  
promptImg.sprite = GetRoundedSprite();  
promptImg.type = Image.Type.Sliced;  
  
var ptxtGo = MakeChild("PromptText", _promptGo.transform);  
var ptrt = ptxtGo.GetComponent<RectTransform>();  
ptrt.anchorMin = Vector2.zero;  
ptrt.anchorMax = Vector2.one;  
ptrt.sizeDelta = Vector2.zero;  
ptrt.anchoredPosition = Vector2.zero;  
  
var ptxt = ptxtGo.AddComponent<TextMeshProUGUI>();  
ApplyTMPDefaults(ptxt);  
string actionText = !string.IsNullOrEmpty(promptActionText)  
    ? promptActionText  
    : (isShopKeeper ? "Shop" : "Talk");  
ptxt.text = $"[{interactKey}] {actionText}";  
ptxt.color = isShopKeeper ? new Color(1.0f, 0.85f, 0.35f, 1f) : new Color(0.74f, 0  
.52f, 1f, 1f);  
ptxt.fontSize = 20f;  
ptxt.alignment = TextAlignmentOptions.Center;  
ptxt.fontStyle = FontStyles.Bold;  
  
// If shopkeeper, add an instant [TRADE / SHOP] button inside the bubble  
if (isShopKeeper)  
{  
    var shopBtnGo = MakeChild("ShopBtn", panelGo.transform);  
    var srt = shopBtnGo.GetComponent<RectTransform>();  
    srt.anchorMin = new Vector2(1f, 0f);  
    srt.anchorMax = new Vector2(1f, 0f);  
    srt.pivot = new Vector2(1f, 0f);  
    srt.sizeDelta = new Vector2(140f, 44f);  
    srt.anchoredPosition = new Vector2(-76f, 14f);  
  
    var sbtnImg = shopBtnGo.AddComponent<Image>();  
    sbtnImg.color = new Color(0.20f, 0.14f, 0.38f, 0.95f);  
    sbtnImg.sprite = GetRoundedSprite();  
    sbtnImg.type = Image.Type.Sliced;  
  
    var sbtn = shopBtnGo.AddComponent<Button>();  
    sbtn.onClick.AddListener(OpenShopDirectly);
```

```

        var stxtGo = MakeChild("ShopBtnText", shopBtnGo.transform);
        var strt = stxtGo.GetComponent<RectTransform>();
        strt.anchorMin = Vector2.zero;
        strt.anchorMax = Vector2.one;
        strt.sizeDelta = Vector2.zero;
        strt.anchoredPosition = Vector2.zero;

        var stxt = stxtGo.AddComponent<TextMeshProUGUI>();
        ApplyTMPDefaults(stxt);
        stxt.text = "SHOP ???";
        stxt.color = new Color(1.0f, 0.88f, 0.35f, 1f);
        stxt.fontSize = 18f;
        stxt.alignment = TextAlignmentOptions.Center;
        stxt.fontStyle = FontStyles.Bold;
    }

    _promptGo.SetActive(false);
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????
// SMALL UTILITIES
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????

/// <summary>Resets TMP properties that Unity sometimes initialises badly.</summary>
private static void ApplyTMPDefaults(TMP_Text t)
{
    t.characterSpacing      = 0f;
    t.characterHorizontalScale = 1f;
    t.lineSpacing           = 0f;
    t.paragraphSpacing      = 0f;
    t.margin                = Vector4.zero;
}

/// <summary>Creates a named, empty child GameObject.</summary>
private static GameObject MakeChild(string name, Transform parent)
{
    var go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    return go;
}

/// <summary>Returns dynamic 9-slice rounded-rect sprite from UIFactory.</summary>
private static Sprite GetRoundedSprite()
    => UIFactory.GetRoundedSprite();

/// <summary>Generates a crisp circle sprite from pixels at runtime.</summary>
private static Sprite MakeCircleSprite(int size)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false)
        { filterMode = FilterMode.Bilinear };
    float c = size * 0.5f, r = c - 0.5f;
    for (int y = 0; y < size; y++)
        for (int x = 0; x < size; x++)
        {
            float dx = x - c, dy = y - c;
            tex.SetPixel(x, y, dx * dx + dy * dy <= r * r ? Color.white : Color.clear);
        }
    tex.Apply();
    return Sprite.Create(tex, new Rect(0, 0, size, size), Vector2.one * 0.5f);
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????
// EDITOR GIZMO
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????
#endif UNITY_EDITOR
private void OnDrawGizmosSelected()
{

```

```

        // Interaction range ring
        Gizmos.color = new Color(0.60f, 0.30f, 1f, 0.35f);
        Gizmos.DrawWireSphere(transform.position, interactRange);

        // Solid fill so it's obvious in Scene view
        Gizmos.color = new Color(0.60f, 0.30f, 1f, 0.08f);
        Gizmos.DrawSphere(transform.position, interactRange);

        // Label
        UnityEditor.Handles.color = new Color(0.74f, 0.52f, 1f, 1f);
        UnityEditor.Handles.Label(
            transform.position + Vector3.up * (interactRange + 0.25f),
            $" ??? interact range {interactRange:F1} u");
    }
#endif
}

```

FILE 25: Assets/Scenes/scripts/Systems/AudioManager.cs
 PURPOSE: Central audio playback manager.

```

using System.Collections;
using UnityEngine;

[AddComponentMenu("Systems/Audio Manager")]
public class AudioManager : MonoBehaviour
{
    private static AudioManager instance;

    public static AudioManager Instance
    {
        get
        {
            if (instance == null)
            {
                instance = FindFirstObjectByType<AudioManager>();
                if (instance == null)
                {
                    GameObject go = new GameObject("Audio Manager (Auto)");
                    instance = go.AddComponent<AudioManager>();
                }
            }
            return instance;
        }
    }

    [Header("Audio Sources")]
    [Tooltip("AudioSource used for playing background music.")]
    public AudioSource musicSource;
    [Tooltip("AudioSource used for playing general sound effects.")]
    public AudioSource sfxSource;

    [Header("Volume Settings")]
    [Range(0f, 1f)] public float masterVolume = 1.0f;
    [Range(0f, 1f)] public float musicVolume = 0.8f;
    [Range(0f, 1f)] public float sfxVolume = 0.9f;

    [Header("Transition Settings")]
    [Tooltip("Duration in seconds for fading music tracks in and out.")]
    public float fadeDuration = 1.0f;

    private Coroutine musicFadeCoroutine;

    void Awake()
    {
        // Enforce Singleton instance pattern
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
            InitializeAudioSources();
        }
        else if (instance != this)
        {
            Destroy(gameObject);
        }
    }
}

```

```

    }

    void Start()
    {
        // Silence any other AudioSources in the scene if we are on the main menu
        if (!MainMenuUIToolkitController.isPlaying)
        {
            AudioSource[] allSources = FindObjectsByType<AudioSource>(FindObjectsSortMode.None
);
            foreach (var source in allSources)
            {
                if (source != musicSource && source != sfxSource)
                {
                    source.playOnAwake = false;
                    if (source.isPlaying)
                    {
                        source.Stop();
                    }
                }
            }
        }
    }

    private void InitializeAudioSources()
    {
        // Create audio source components if they are not already assigned
        if (musicSource == null)
        {
            musicSource = gameObject.AddComponent<AudioSource>();
            musicSource.loop = true;
            musicSource.playOnAwake = false;
        }
        else
        {
            // Safeguard: Disable playOnAwake and stop any auto-playing clip on assigned music
source
            musicSource.playOnAwake = false;
            if (musicSource.isPlaying)
            {
                musicSource.Stop();
            }
        }

        if (sfxSource == null)
        {
            sfxSource = gameObject.AddComponent<AudioSource>();
            sfxSource.loop = false;
            sfxSource.playOnAwake = false;
        }
        else
        {
            sfxSource.playOnAwake = false;
        }
    }

    /// <summary>
    /// Play background music track, with optional smooth cross-fade.
    /// </summary>
    public void PlayBGM(AudioClip clip, bool fade = true)
    {
        if (musicSource == null) InitializeAudioSources();

        if (musicSource.clip == clip && musicSource.isPlaying) return; // Already playing this
clip

        if (fade && musicSource.isPlaying)
        {
            if (musicFadeCoroutine != null) StopCoroutine(musicFadeCoroutine);
            musicFadeCoroutine = StartCoroutine(FadeMusic(clip));
        }
        else
        {
            musicSource.clip = clip;
            musicSource.volume = GetRealMusicVolume();
            if (clip != null)
            {

```

```

        musicSource.Play();
    }
    else
    {
        musicSource.Stop();
    }
}

/// <summary>
/// Plays a 2D sound effect one-shot.
/// </summary>
public void PlaySFX(AudioClip clip, float volumeScale = 1.0f)
{
    if (clip == null) return;
    sfxSource.PlayOneShot(clip, volumeScale * GetRealSFXVolume());
}

/// <summary>
/// Plays a 3D spatial sound effect at the specified position coordinates.
/// </summary>
public void PlaySFXAtPosition(AudioClip clip, Vector3 position, float volumeScale = 1.0f)
{
    if (clip == null) return;
    AudioSource.PlayClipAtPoint(clip, position, volumeScale * GetRealSFXVolume());
}

/// <summary>
/// Smoothly fades out the current BGM track and fades in the new BGM clip.
/// </summary>
private IEnumerator FadeMusic(AudioClip newClip)
{
    float startVolume = musicSource.volume;
    float elapsed = 0f;

    // Fade out current track
    while (elapsed < fadeDuration)
    {
        elapsed += Time.deltaTime;
        musicSource.volume = Mathf.Lerp(startVolume, 0f, elapsed / fadeDuration);
        yield return null;
    }

    musicSource.Stop();
    musicSource.clip = newClip;

    if (newClip != null)
    {
        musicSource.Play();
        elapsed = 0f;
        float targetVolume = GetRealMusicVolume();

        // Fade in new track
        while (elapsed < fadeDuration)
        {
            elapsed += Time.deltaTime;
            musicSource.volume = Mathf.Lerp(0f, targetVolume, elapsed / fadeDuration);
            yield return null;
        }

        musicSource.volume = targetVolume;
    }
}

// Vol helpers
public float GetRealMusicVolume() => musicVolume * masterVolume;
public float GetRealSFXVolume() => sfxVolume * masterVolume;

public void SetMusicVolume(float volume)
{
    musicVolume = Mathf.Clamp01(volume);
    if (musicSource != null && musicSource.isPlaying)
    {
        musicSource.volume = GetRealMusicVolume();
    }
}

```

```

    public void SetSFXVolume(float volume)
    {
        sfxVolume = Mathf.Clamp01(volume);
    }

    public void SetMasterVolume(float volume)
    {
        masterVolume = Mathf.Clamp01(volume);
        if (musicSource != null && musicSource.isPlaying)
        {
            musicSource.volume = GetRealMusicVolume();
        }
    }
}

```

FILE 26: Assets/Scenes/scripts/Systems/LevelMusicPlayer.cs
PURPOSE: Plays level music.

```

using UnityEngine;

/// <summary>
/// Lightweight player that triggers level background music only when active gameplay starts.
/// Works seamlessly within single-scene architectures.
/// </summary>
[AddComponentMenu("Systems/Level Music Player")]
public class LevelMusicPlayer : MonoBehaviour
{
    [Header("Level Music Settings")]
    [Tooltip("The music track to play during gameplay.")]
    public AudioClip levelMusic;

    [Tooltip("Whether to fade the music track in smoothly.")]
    public bool fadeOnStart = true;

    void Start()
    {
        EnsureTrackAssigned();

        // Only play the gameplay music if we are actively playing the game (not on the title
screen)
        if (MainMenuUIToolkitController.isPlaying)
        {
            StartLevelMusic();
        }
    }

    public void StartLevelMusic()
    {
        EnsureTrackAssigned();

        if (levelMusic != null && AudioManager.Instance != null)
        {
            AudioManager.Instance.PlayBGM(levelMusic, fadeOnStart);
            Debug.Log("[LevelMusicPlayer] Started level music: " + levelMusic.name);
        }
    }

    private void EnsureTrackAssigned()
    {
        if (levelMusic == null)
        {
            levelMusic = Resources.Load<AudioClip>("Audio/bamboo-incense");
            if (levelMusic == null)
            {
                levelMusic = Resources.Load<AudioClip>("Audio/temple-thunder");
            }
        }
    }

    #if UNITY_EDITOR
        if (levelMusic == null)
        {
            levelMusic = UnityEditor.AssetDatabase.LoadAssetAtPath<AudioClip>("Assets/Audio/bamboo-incense.mp3");
        }
    #endif
}

```

```

    }
}

```

FILE 27: Assets/Scenes/scripts/Pause Manager/PauseMenu.cs
PURPOSE: Pause menu controls.

```

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
using TMPPro;

/// <summary>
/// Pause menu with the Main Menu's Cyber-Gothic Glassmorphic aesthetic ???
/// removes the card container box and presents sleek neon-accented cyber buttons
/// floating seamlessly over the misty Torii background.
/// Triggered by P or Escape.
/// </summary>
public class PauseMenu : MonoBehaviour
{
    public static PauseMenu Instance { get; private set; }

    public bool isPaused { get; private set; }

    /// <summary>
    /// Scene name to load when the player selects "Quit to Menu".
    /// </summary>
    public string menuSceneName = "MainMenu";

    // ?????? Palette: midnight void + aether cyan + restrained violet ??????
    private static readonly Color VoidOverlay = new Color(0.012f, 0.005f, 0.025f, 0.90f);
;
    private static readonly Color NeonPurple = new Color(0.75f, 0.52f, 0.99f, 1f); // #
c084fc Radiant Violet
    private static readonly Color NeonPurpleDim = new Color(0.75f, 0.52f, 0.99f, 0.42f);
    private static readonly Color ArcaneViolet = new Color(0.66f, 0.33f, 0.97f, 1f); // #
a855f7 Mystic Amethyst
    private static readonly Color ArcaneVioletDim = new Color(0.66f, 0.33f, 0.97f, 0.35f);

    [Header("Title Screen Artwork & Sprites")]
    public Sprite logoSprite;
    public Sprite backgroundSprite;

    private GameObject pauseOverlay;

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
    static void ResetStatic()
    {
        Instance = null;
    }

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    static void AutoCreate()
    {
        if (Instance == null)
        {
            GameObject go = new GameObject("PauseManager");
            go.AddComponent<PauseMenu>();
            DontDestroyOnLoad(go);
        }
    }

    void Awake()
    {
        Instance = this;
        AutoLoadSprites();
        BuildUI();
        Time.timeScale = 1f;
    }

    public void AutoLoadSprites()
    {
        if (logoSprite == null)
        {
            logoSprite = LoadSpriteDirectly("mainmenulogo") ?? LoadSpriteDirectly("main_menu_l
ogo");

```

[illegible]

```

CanvasScaler scaler = canvas.GetComponent<CanvasScaler>();
if (scaler != null)
    scaler.matchWidthOrHeight = 1.0f;

// ?????? Layer 0: background image + dark overlay ??????
RectTransform overlayRT = UIFactory.CreateFullScreenPanel(
    canvas.transform, "PauseOverlay", VoidOverlay);
pauseOverlay = overlayRT.gameObject;

if (backgroundSprite != null)
{
    RectTransform bgRT = UIFactory.CreatePanel(
        overlayRT, "TitleBackground", Color.white,
        Vector2.zero, Vector2.one, Vector2.zero, Vector2.zero);
    Image bgImgComp = bgRT.GetComponent<Image>();
    bgImgComp.sprite = backgroundSprite;
    bgImgComp.color = new Color(0.40f, 0.36f, 0.55f, 0.50f); // Soft dark violet overl
ay tint
    bgImgComp.raycastTarget = false;
}

// ?????? Layer 1: radial vignette ??????
Sprite vignetteSprite = CreateVignetteSprite(512, 512);
RectTransform vignetteRT = UIFactory.CreatePanel(
    overlayRT, "Vignette", Color.white,
    Vector2.zero, Vector2.one, Vector2.zero, Vector2.zero);
Image vignetteImg = vignetteRT.GetComponent<Image>();
vignetteImg.sprite = vignetteSprite;
vignetteImg.raycastTarget = false;
vignetteImg.color = new Color(0.04f, 0.01f, 0.08f, 0.85f);

// ?????? Layer 2: Main Content Column (NO Card Frame - Open & Seamless) ??????
RectTransform contentColumnRT = UIFactory.CreatePanel(
    overlayRT, "PauseContentColumn", Color.clear,
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
contentColumnRT.sizeDelta = new Vector2(440f, 760f);

// Content vertical layout
VerticalLayoutGroup vlg = UIFactory.AddVerticalLayout(contentColumnRT.gameObject, 14f,
    new RectOffset(10, 10, 10, 10), TextAnchor.MiddleCenter);
vlg.childControlWidth = false;
vlg.childControlHeight = false;
vlg.childForceExpandWidth = false;
vlg.childForceExpandHeight = false;

// ?????? Logo / Title Header ??????
if (logoSprite != null)
{
    GameObject logoGO = new GameObject("TitleLogo", typeof(RectTransform), typeof(Image));
    logoGO.transform.SetParent(contentColumnRT, false);
    Image logoImgComp = logoGO.GetComponent<Image>();
    logoImgComp.sprite = logoSprite;
    logoImgComp.preserveAspect = true;
    logoImgComp.raycastTarget = false;

    LayoutElement le = logoGO.AddComponent<LayoutElement>();
    le.preferredWidth = 380f;
    le.preferredHeight = 110f;
    le.minWidth = 380f;
    le.minHeight = 110f;
}
else
{
    TextMeshProUGUI titleText = UIFactory.CreateText(
        contentColumnRT, "PausedTitle", "PAUSED",
        48f, UIFactory.TextWhite, TextAlignmentOptions.Center);
    titleText.fontStyle = FontStyles.Bold;
    titleText.characterSpacing = 12f;

    LayoutElement le = titleText.gameObject.AddComponent<LayoutElement>();
    le.preferredWidth = 420f;
    le.preferredHeight = 56f;
}

// Subtitle & divider

```

```

TextMeshProUGUI subtitleText = UIFactory.CreateText(
    contentColumnRT, "Subtitle", "GAME PAUSED",
    14f, NeonPurpleDim, TextAlignmentOptions.Center);
subtitleText.characterSpacing = 6f;
subtitleText.fontStyle = FontStyles.Bold;

LayoutElement subLe = subtitleText.gameObject.AddComponent<LayoutElement>();
subLe.preferredWidth = 420f;
subLe.preferredHeight = 20f;

CreateArcaneDivider(contentColumnRT, "TitleDivider");

// ?????? Main Menu Styled Cyber-Gothic Glassmorphic Buttons (Shades of Purple) ??????
// 1. Resume (Radiant Violet / Electric Lavender - Primary Action)
CreateCyberGothicButton(
    contentColumnRT, "ResumeBtn", "RESUME", "CONTINUE EXPEDITION", "???",
    new Color(0.75f, 0.52f, 0.99f, 1f), // #c084fc Radiant Violet
    () => ResumeGame()
);

// 2. Restart (Mystic Amethyst Purple)
CreateCyberGothicButton(
    contentColumnRT, "RestartBtn", "RESTART", "RETRY FROM CHECKPOINT", "???",
    new Color(0.66f, 0.33f, 0.97f, 1f), // #a855f7 Mystic Amethyst
    () => RestartLevel()
);

// 3. Minigames (Celestial Orchid / Neon Magenta-Purple)
CreateCyberGothicButton(
    contentColumnRT, "MinigamesBtn", "ARCADE VAULT", "TRAINING & MINIGAMES", "???",
    new Color(0.85f, 0.27f, 0.94f, 1f), // #d946ef Celestial Orchid
    () => OpenMinigamesMenu()
);

// 4. Quit to Menu (Dusky Void Plum)
CreateCyberGothicButton(
    contentColumnRT, "QuitBtn", "QUIT TO MENU", "RETURN TO TITLE SCREEN", "???",
    new Color(0.55f, 0.20f, 0.85f, 1f), // #8c33d9 Dusky Void Plum
    () => QuitToMenu(menuSceneName)
);

// Footer hint
CreateArcaneDivider(contentColumnRT, "BottomDivider");

TextMeshProUGUI hintText = UIFactory.CreateText(
    contentColumnRT, "Hint", "PRESS ESC OR P TO RESUME",
    12f, new Color(0.80f, 0.65f, 0.95f, 0.75f), TextAlignmentOptions.Center);
hintText.characterSpacing = 3f;

LayoutElement hintLe = hintText.gameObject.AddComponent<LayoutElement>();
hintLe.preferredWidth = 420f;
hintLe.preferredHeight = 20f;

// Ambient FX
PauseMenuAmbientFX ambientFX = pauseOverlay.AddComponent<PauseMenuAmbientFX>();
ambientFX.Initialize(null, vignetteRT, new Color(0.4f, 0.2f, 0.8f, 0.25f));

pauseOverlay.SetActive(false);
}

// ?????? Procedural Cyber-Gothic Button Construction (Matching Main Menu) ??????
private Button CreateCyberGothicButton(
    Transform parent, string name, string labelText, string hintText, string runeIcon,
    Color themeColor, UnityEngine.Events.UnityAction onClick)
{
    Vector2 size = new Vector2(380f, 68f);

    // Outer Button Container
    GameObject btnGO = new GameObject(name, typeof(RectTransform));
    btnGO.transform.SetParent(parent, false);
    RectTransform btnRT = btnGO.GetComponent<RectTransform>();
    btnRT.sizeDelta = size;

    LayoutElement le = btnGO.AddComponent<LayoutElement>();
    le.minWidth = size.x;

```

```

le.minHeight = size.y;
le.preferredWidth = size.x;
le.preferredHeight = size.y;

// Button Background (Dark Glassmorphic Cyber Frame)
Image bgImg = btnG0.AddComponent<Image>();
bgImg.sprite = CreateCyberFrameSprite(128, 64, 14);
bgImg.type = Image.Type.Sliced;
bgImg.color = new Color(0.07f, 0.08f, 0.12f, 0.96f);

// Glowing Border Outline
RectTransform borderRT = UIFactory.CreatePanel(
    btnRT, "Border", new Color(themeColor.r, themeColor.g, themeColor.b, 0.50f),
    Vector2.zero, Vector2.one, Vector2.zero, Vector2.zero);
Image borderImg = borderRT.GetComponent<Image>();
borderImg.sprite = CreateCyberFrameSprite(128, 64, 14);
borderImg.type = Image.Type.Sliced;
borderImg.raycastTarget = false;

// Top Accent Glow Line
RectTransform topAccentRT = UIFactory.CreatePanel(
    btnRT, "TopAccent", new Color(themeColor.r, themeColor.g, themeColor.b, 0.70f),
    new Vector2(0f, 1f), new Vector2(1f, 1f),
    new Vector2(12f, -3f), new Vector2(-12f, 0f));
Image topAccentImg = topAccentRT.GetComponent<Image>();
topAccentImg.raycastTarget = false;

// Bottom Accent Line
RectTransform botAccentRT = UIFactory.CreatePanel(
    btnRT, "BotAccent", new Color(themeColor.r, themeColor.g, themeColor.b, 0.30f),
    new Vector2(0f, 0f), new Vector2(1f, 0f),
    new Vector2(20f, 0f), new Vector2(-20f, 2f));
Image botAccentImg = botAccentRT.GetComponent<Image>();
botAccentImg.raycastTarget = false;

// Left Rune Badge Circle
RectTransform runeLeftRT = UIFactory.CreatePanel(
    btnRT, "RuneLeft", new Color(themeColor.r, themeColor.g, themeColor.b, 0.18f),
    new Vector2(0f, 0.5f), new Vector2(1f, 0.5f),
    new Vector2(16f, -15f), new Vector2(46f, 15f));
Image runeLeftBg = runeLeftRT.GetComponent<Image>();
runeLeftBg.sprite = CreateOrbSprite(32);
runeLeftBg.raycastTarget = false;

TextMeshProUGUI runeLeftTxt = UIFactory.CreateText(
    runeLeftRT, "RuneTxt", runeIcon, 14f, themeColor, TextAlignmentOptions.Center);
runeLeftTxt.fontStyle = FontStyles.Bold;

// Right Rune Badge Circle
RectTransform runeRightRT = UIFactory.CreatePanel(
    btnRT, "RuneRight", new Color(themeColor.r, themeColor.g, themeColor.b, 0.18f),
    new Vector2(1f, 0.5f), new Vector2(1f, 0.5f),
    new Vector2(-46f, -15f), new Vector2(-16f, 15f));
Image runeRightBg = runeRightRT.GetComponent<Image>();
runeRightBg.sprite = CreateOrbSprite(32);
runeRightBg.raycastTarget = false;

TextMeshProUGUI runeRightTxt = UIFactory.CreateText(
    runeRightRT, "RuneTxt", runeIcon, 14f, themeColor, TextAlignmentOptions.Center);
runeRightTxt.fontStyle = FontStyles.Bold;

// Center Content Text (Title + Sub-hint)
RectTransform textContainerRT = UIFactory.CreatePanel(
    btnRT, "TextContainer", Color.clear,
    new Vector2(0f, 0f), new Vector2(1f, 1f),
    new Vector2(50f, 0f), new Vector2(-50f, 0f));

VerticalLayoutGroup textVlg = UIFactory.AddVerticalLayout(textContainerRT.gameObject,
1f,
    new RectOffset(0, 0, 8, 8), TextAnchor.MiddleCenter);
textVlg.childControlWidth = false;
textVlg.childControlHeight = false;

TextMeshProUGUI label = UIFactory.CreateText(
    textContainerRT, "Label", labelText, 17f,
    new Color(0.96f, 0.97f, 1.0f, 1f), TextAlignmentOptions.Center);

```

```

label.fontStyle = FontStyles.Bold;
label.characterSpacing = 5f;

LayoutElement labelLe = label.gameObject.AddComponent<LayoutElement>();
labelLe.preferredWidth = 260f;
labelLe.preferredHeight = 24f;

TextMeshProUGUI hint = UIFactory.CreateText(
    textContainerRT, "Hint", hintText, 9.5f,
    new Color(0.70f, 0.75f, 0.88f, 0.70f), TextAlignmentOptions.Center);
hint.characterSpacing = 2f;

LayoutElement hintLe = hint.gameObject.AddComponent<LayoutElement>();
hintLe.preferredWidth = 260f;
hintLe.preferredHeight = 16f;

// Button Component
Button btn = btnGO.AddComponent<Button>();
btn.targetGraphic = bgImg;

ColorBlock cb = btn.colors;
cb.normalColor = Color.white;
cb.highlightedColor = Color.white;
cb.pressedColor = new Color(0.85f, 0.85f, 0.95f, 1f);
cb.fadeDuration = 0.1f;
btn.colors = cb;

if (onClick != null)
    btn.onClick.AddListener(onClick);

// Attach Interactive Cyber-Gothic Hover & Press Animation
CyberGothicButtonFX fx = btnGO.AddComponent<CyberGothicButtonFX>();
fx.Initialize(bgImg, borderImg, topAccentImg, botAccentImg, label, hint, themeColor);

return btn;
}

// ?????? Decorative Elements ?????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????

private void CreateArcaneDivider(Transform parent, string name)
{
    RectTransform rowRT = UIFactory.CreatePanel(
        parent, name, Color.clear,
        new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
    rowRT.sizeDelta = new Vector2(380f, 16f);
    UIFactory.AddLayoutElement(rowRT.gameObject, preferredHeight: 16f, preferredWidth: 380
f);

    HorizontalLayoutGroup hlg = UIFactory.AddHorizontalLayout(rowRT.gameObject, 0f,
        new RectOffset(0, 0, 0, 0), TextAnchor.MiddleCenter);
    hlg.childControlWidth = false;
    hlg.childForceExpandWidth = false;

    // Left fade line
    CreateDividerSegment(rowRT, 130f, new Color(0.65f, 0.20f, 0.95f, 0.45f));
    // Left small orb
    CreateDividerOrb(rowRT, 6f, NeonPurpleDim);
    // Center line
    CreateDividerSegment(rowRT, 50f, new Color(0.50f, 0.15f, 0.85f, 0.65f));
    // Center orb (larger, brighter)
    CreateDividerOrb(rowRT, 10f, NeonPurple);
    // Right line
    CreateDividerSegment(rowRT, 50f, new Color(0.50f, 0.15f, 0.85f, 0.65f));
    // Right small orb
    CreateDividerOrb(rowRT, 6f, NeonPurpleDim);
    // Right fade line
    CreateDividerSegment(rowRT, 130f, new Color(0.65f, 0.20f, 0.95f, 0.45f));
}

private void CreateDividerSegment(Transform parent, float width, Color color)
{
    RectTransform segRT = UIFactory.CreatePanel(
        parent, "Seg", color,
        new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
    segRT.sizeDelta = new Vector2(width, 1.5f);

```

[illegible]

```

Color fill = Color.white;
Color clear = new Color(1f, 1f, 1f, 0f);

for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        bool inside = IsInsideRoundedRect(x, y, width, height, radius);
        if (inside)
        {
            float edgeDist = GetRoundedRectEdgeDist(x, y, width, height, radius);
            float alpha = Mathf.Clamp01(edgeDist + 0.5f);
            tex.SetPixel(x, y, new Color(1f, 1f, 1f, alpha));
        }
        else
        {
            tex.SetPixel(x, y, clear);
        }
    }
}

tex.Apply();
int border = radius + 2;
return Sprite.Create(tex, new Rect(0, 0, width, height), new Vector2(0.5f, 0.5f), 100f
,
    0, SpriteMeshType.FullRect, new Vector4(border, border, border, border));
}

private static bool IsInsideRoundedRect(int x, int y, int w, int h, int r)
{
    if (x < r && y < r) return (r - x) * (r - x) + (r - y) * (r - y) <= r * r;
    if (x >= w - r && y < r) return (x - (w - r - 1)) * (x - (w - r - 1)) + (r - y) * (r -
y) <= r * r;
    if (x < r && y >= h - r) return (r - x) * (r - x) + (y - (h - r - 1)) * (y - (h - r -
1)) <= r * r;
    if (x >= w - r && y >= h - r) return (x - (w - r - 1)) * (x - (w - r - 1)) + (y - (h -
r - 1)) * (y - (h - r - 1)) <= r * r;
    return true;
}

private static float GetRoundedRectEdgeDist(int x, int y, int w, int h, int r)
{
    float cx = 0, cy = 0;
    bool inCorner = false;

    if (x < r && y < r) { cx = r; cy = r; inCorner = true; }
    else if (x >= w - r && y < r) { cx = w - r - 1; cy = r; inCorner = true; }
    else if (x < r && y >= h - r) { cx = r; cy = h - r - 1; inCorner = true; }
    else if (x >= w - r && y >= h - r) { cx = w - r - 1; cy = h - r - 1; inCorner = true; }
}

    if (inCorner)
    {
        float dist = Mathf.Sqrt((x - cx) * (x - cx) + (y - cy) * (y - cy));
        return r - dist;
    }

    return Mathf.Min(Mathf.Min(x, w - 1 - x), Mathf.Min(y, h - 1 - y));
}

private static Sprite CreateOrbSprite(int size)
{
    Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.filterMode = FilterMode.Bilinear;
    tex.wrapMode = TextureWrapMode.Clamp;

    float center = size * 0.5f;
    float radius = center * 0.85f;

    for (int y = 0; y < size; y++)
    {
        for (int x = 0; x < size; x++)
        {
            float dx = x - center;
            float dy = y - center;
            float dist = Mathf.Sqrt(dx * dx + dy * dy);

```

```

        float norm = dist / radius;

        float alpha;
        if (norm <= 0.5f)
            alpha = 1f;
        else if (norm <= 1f)
            alpha = 1f - (norm - 0.5f) * 2f;
        else
            alpha = Mathf.Max(0f, 1f - (norm - 1f) * 3f) * 0.3f;

        alpha = alpha * alpha;
        tex.SetPixel(x, y, new Color(1f, 1f, 1f, alpha));
    }
}

tex.Apply();
return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), 100f);
}

private static Sprite CreateVignetteSprite(int size, int _)
{
    Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.filterMode = FilterMode.Bilinear;
    tex.wrapMode = TextureWrapMode.Clamp;

    float center = size * 0.5f;
    float maxDist = center * 1.15f;

    for (int y = 0; y < size; y++)
    {
        for (int x = 0; x < size; x++)
        {
            float dx = (x - center) / maxDist;
            float dy = (y - center) / maxDist;
            float dist = Mathf.Sqrt(dx * dx + dy * dy);
            float alpha = Mathf.Clamp01((dist - 0.15f) / 0.85f);
            alpha = alpha * alpha;
            tex.SetPixel(x, y, new Color(1f, 1f, 1f, alpha));
        }
    }

    tex.Apply();
    return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f), 100f);
}
}

// ?????? Interactive Cyber-Gothic Button Animation (Unscaled Time) ??????

public class CyberGothicButtonFX : MonoBehaviour,
    UnityEngine.EventSystems.IPointerEnterHandler,
    UnityEngine.EventSystems.IPointerExitHandler,
    UnityEngine.EventSystems.IPointerDownHandler,
    UnityEngine.EventSystems.IPointerUpHandler
{
    private RectTransform rectTransform;
    private Image bgImage;
    private Image borderImage;
    private Image topAccentImage;
    private Image botAccentImage;
    private TextMeshProUGUI labelText;
    private TextMeshProUGUI hintText;
    private Color themeColor;

    private Vector3 originalScale;
    private Vector3 targetScale = Vector3.one;
    private bool isHovered;

    private Color origBgColor;
    private Color targetBgColor;
    private Color origBorderColor;
    private Color targetBorderColor;
    private Color origTopAccentColor;
    private Color targetTopAccentColor;
    private Color origLabelColor;
    private Color targetLabelColor;

```

```

public void Initialize(
    Image bg, Image border, Image topAccent, Image botAccent,
    TextMeshProUGUI label, TextMeshProUGUI hint, Color theme)
{
    rectTransform = GetComponent<RectTransform>();
    bgImage = bg;
    borderImage = border;
    topAccentImage = topAccent;
    botAccentImage = botAccent;
    labelText = label;
    hintText = hint;
    themeColor = theme;

    originalScale = rectTransform.localScale;
    targetScale = originalScale;

    origBgColor = bgImage != null ? bgImage.color : new Color(0.07f, 0.08f, 0.12f, 0.96f);
    targetBgColor = origBgColor;

    origBorderColor = borderImage != null ? borderImage.color : new Color(theme.r, theme.g
, theme.b, 0.50f);
    targetBorderColor = origBorderColor;

    origTopAccentColor = topAccentImage != null ? topAccentImage.color : new Color(theme.r
, theme.g, theme.b, 0.70f);
    targetTopAccentColor = origTopAccentColor;

    origLabelColor = labelText != null ? labelText.color : Color.white;
    targetLabelColor = origLabelColor;
}

void Update()
{
    float dt = Time.unscaledDeltaTime * 14f;

    rectTransform.localScale = Vector3.Lerp(rectTransform.localScale, targetScale, dt);

    if (bgImage != null)
        bgImage.color = Color.Lerp(bgImage.color, targetBgColor, dt);

    if (borderImage != null)
        borderImage.color = Color.Lerp(borderImage.color, targetBorderColor, dt);

    if (topAccentImage != null)
        topAccentImage.color = Color.Lerp(topAccentImage.color, targetTopAccentColor, dt);

    if (labelText != null)
        labelText.color = Color.Lerp(labelText.color, targetLabelColor, dt);
}

public void OnPointerEnter(UnityEngine.EventSystems.PointerEventData eventData)
{
    isHovered = true;
    targetScale = originalScale * 1.04f;
    targetBgColor = new Color(0.18f, 0.08f, 0.32f, 0.98f);
    targetBorderColor = new Color(themeColor.r, themeColor.g, themeColor.b, 1.0f);
    targetTopAccentColor = new Color(0.95f, 0.85f, 1.0f, 1.0f); // Bright white-neon flash
    targetLabelColor = Color.white;
}

public void OnPointerExit(UnityEngine.EventSystems.PointerEventData eventData)
{
    isHovered = false;
    targetScale = originalScale;
    targetBgColor = origBgColor;
    targetBorderColor = origBorderColor;
    targetTopAccentColor = origTopAccentColor;
    targetLabelColor = origLabelColor;
}

public void OnPointerDown(UnityEngine.EventSystems.PointerEventData eventData)
{
    targetScale = originalScale * 0.96f;
    targetBgColor = new Color(themeColor.r * 0.3f, themeColor.g * 0.3f, themeColor.b * 0.3
f, 0.98f);
}

```

```

        public void OnPointerUp(UnityEngine.EventSystems.PointerEventData eventData)
        {
            targetScale = isHovered ? originalScale * 1.04f : originalScale;
            targetBgColor = isHovered ? new Color(0.18f, 0.08f, 0.32f, 0.98f) : origBgColor;
        }
    }
}

```

FILE 28: Assets/Scenes/scripts/UI/TouchControlsManager.cs
 PURPOSE: Mobile control visibility and input.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using TMPro;

/// <summary>
/// TouchControlsManager - Complete Cyber-Gothic On-Screen Mobile Touch Controller:
/// 1. D-Pad (Left Thumb): Dedicated Left (???) and Right (???) buttons with continuous frame-
    by-frame
    input holding, IDragHandler pointer locking, and double-tap dash trigger.
/// 2. Action Cluster (Right Thumb): Glassmorphic buttons for Attack, Jump, Dash, Blob, Magic,
    Spear.
/// 3. Utility Buttons: Pause (top-right) & Contextual Interact (floating prompt).
/// 4. Tactile Micro-Animations: Scale-punch (0.90x) on touch down.
/// </summary>
public class TouchControlsManager : MonoBehaviour
{
    private static TouchControlsManager instance;
    public static TouchControlsManager Instance
    {
        get
        {
            if (instance == null) instance = FindFirstObjectByType<TouchControlsManager>();
            return instance;
        }
    }

    [Header("Visibility Settings")]
    [Tooltip("Enables touch controls inside the Unity Editor for instant mouse playtesting.")]
    public bool forceEnableInEditor = true;

    // Runtime UI elements
    private Canvas touchCanvas;

    private static Sprite circleSprite;
    private static Sprite ringSprite;

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    static void AutoInitializeTouchControls()
    {
        string sceneName = SceneManager.GetActiveScene().name;
        if (sceneName == "MainMenu") return;

        if (Instance == null && FindFirstObjectByType<TouchControlsManager>() == null)
        {
            GameObject tcGO = new GameObject("TouchControlsManager");
            tcGO.AddComponent<TouchControlsManager>();
            DontDestroyOnLoad(tcGO);
        }
    }

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else if (instance != this)
        {
            Destroy(gameObject);
        }
    }
}

```

```

        return;
    }

    SceneManager.sceneLoaded += OnSceneLoaded;
    BuildTouchUI();
}

void OnDestroy()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
}

void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (touchCanvas != null)
    {
        touchCanvas.gameObject.SetActive(false);
    }
}

public static bool IsTitleScreenActive()
{
    if (SceneManager.GetActiveScene().name == "MainMenu") return true;

    var menuController = FindFirstObjectByType<MainMenuUIToolkitController>();
    if (menuController != null && menuController.gameObject.activeInHierarchy)
    {
        var doc = menuController.GetComponent<UnityEngine.UIElements.UIDocument>();
        if (doc != null && doc.rootVisualElement != null && doc.rootVisualElement.style.display != UnityEngine.UIElements.DisplayStyle.None)
        {
            return true;
        }
        if (!MainMenuUIToolkitController.isPlaying)
        {
            return true;
        }
    }

    return false;
}

void Update()
{
    bool isTitle = IsTitleScreenActive();
    bool isPaused = PauseMenu.Instance != null && PauseMenu.Instance.isPaused;

    bool isShop = ShopUI.Instance != null && ShopUI.Instance.IsShopActive;

    // Strictly hide on Title Screen, during Pause, or when shop is active
    bool shouldBeVisible = (Application.isMobilePlatform || forceEnableInEditor) &&
        !isTitle &&
        !isPaused &&
        !isShop;

    if (touchCanvas != null && touchCanvas.gameObject.activeSelf != shouldBeVisible)
    {
        touchCanvas.gameObject.SetActive(shouldBeVisible);
    }
}

void BuildTouchUI()
{
    EnsureEventSystem();
    GenerateProceduralSprites();

    // 1. Create Screen-Space Overlay Canvas
    GameObject canvasGO = new GameObject("TouchControlsCanvas");
    canvasGO.transform.SetParent(transform);
    touchCanvas = canvasGO.AddComponent<Canvas>();
    touchCanvas.renderMode = RenderMode.ScreenSpaceOverlay;
    touchCanvas.sortingOrder = 95;

    CanvasScaler scaler = canvasGO.AddComponent<CanvasScaler>();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(1920, 1080);
}

```

[illegible]

[illegible]

```

        onUp: () => {
            if (move.Instance != null)
            {
                move.Instance.virtualJumpHeld = false;
            }
        }
    };

    // 3. DASH (Shift) - Electric Sky Blue
    CreateTouchButton(parent, "TouchBtn_Dash",
        new Vector2(-350, 130), new Vector2(95, 95),
        new Color(0.1f, 0.82f, 1.0f, 0.80f), "DASH", 18,
        onDown: () => {
            if (move.Instance != null)
            {
                move.Instance.virtualDashPressed = true;
            }
        }
    );

    // 4. BLOB (B) - Dedicated Morph Button
    CreateTouchButton(parent, "TouchBtn_Blob",
        new Vector2(-220, 395), new Vector2(90, 90),
        new Color(0.72f, 0.25f, 1.0f, 0.80f), "BLOB", 18,
        onDown: () => {
            if (move.Instance != null)
            {
                move.Instance.virtualBlobPressed = !move.Instance.virtualBlobPressed;
            }
        }
    );

    // 5. MAGIC (K) - Solar Gold
    CreateTouchButton(parent, "TouchBtn_Magic",
        new Vector2(-355, 265), new Vector2(90, 90),
        new Color(1.0f, 0.68f, 0.1f, 0.80f), "MAGIC", 18,
        onDown: () => {
            if (MageCombat.Instance != null)
            {
                MageCombat.Instance.TriggerRangedAttackFromTouch();
            }
        }
    );

    // 6. SPEAR (Q / X) - Cyan Lightning
    CreateTouchButton(parent, "TouchBtn_Spear",
        new Vector2(-95, 285), new Vector2(90, 90),
        new Color(0.2f, 0.98f, 0.95f, 0.80f), "SPEAR", 18,
        onDown: () => {
            if (LumiSpearWeapon.Instance != null)
            {
                LumiSpearWeapon.Instance.TriggerSpearActionFromTouch();
            }
        }
    );
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// 3. UTILITY BUTTONS (PAUSE & INTERACT)
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
void CreateUtilityButtons(Transform parent)
{
    CreateTouchButton(parent, "TouchBtn_Pause",
        new Vector2(-70, -70), new Vector2(70, 70),
        new Color(0.25f, 0.85f, 1.0f, 0.75f), "II", 28,
        anchorMin: new Vector2(1, 1), anchorMax: new Vector2(1, 1),
        onDown: () => {
            if (PauseMenu.Instance != null)
            {
                PauseMenu.Instance.TogglePause();
            }
        }
    );
}

```

```

    );

    CreatePillButton(parent, "TouchBtn_Interact",
        new Vector2(-220, 520), new Vector2(170, 55),
        new Color(1.0f, 0.82f, 0.2f, 0.85f), "INTERACT (E)", 16,
        onDown: () => {
            TriggerNearbyInteraction();
        }
    );
}

void TriggerNearbyInteraction()
{
    GameObject player = GameObject.FindWithTag("Player") ?? (move.Instance != null ? move.
Instance.gameObject : null);
    if (player == null) return;

    Collider2D[] colliders = Physics2D.OverlapCircleAll(player.transform.position, 4.5f);
    foreach (var col in colliders)
    {
        var interactable = col.GetComponent<NPCInteractable>() ?? col.GetComponentInParent
<NPCInteractable>();
        if (interactable != null)
        {
            interactable.Interact();
            return;
        }

        var speechBubble = col.GetComponent<SpeechBubbleDialogue>() ?? col.GetComponentInP
arent<SpeechBubbleDialogue>();
        if (speechBubble != null)
        {
            speechBubble.OpenDialogueExternally();
            return;
        }

        var shopUi = col.GetComponent<ShopUI>() ?? col.GetComponentInParent<ShopUI>();
        if (shopUi != null)
        {
            shopUi.OpenShop();
            return;
        }
    }
}

RectTransform CreateTouchButton(Transform parent, string name, Vector2 anchoredPos, Vector
2 size,
    Color accentColor, string labelText, int fontSize, System.Action onDown, System.Action
onUp = null,
    Vector2? anchorMin = null, Vector2? anchorMax = null)
{
    GameObject btnObj = new GameObject(name);
    btnObj.transform.SetParent(parent, false);

    RectTransform rt = btnObj.AddComponent<RectTransform>();
    rt.anchorMin = anchorMin ?? new Vector2(1, 0);
    rt.anchorMax = anchorMax ?? new Vector2(1, 0);
    rt.pivot = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = anchoredPos;
    rt.sizeDelta = size;

    Image bgImg = btnObj.AddComponent<Image>();
    bgImg.sprite = circleSprite;
    bgImg.color = new Color(0.04f, 0.05f, 0.12f, 0.65f);
    bgImg.raycastTarget = true;

    GameObject ringObj = new GameObject("AccentRing");
    ringObj.transform.SetParent(btnObj.transform, false);
    RectTransform ringRT = ringObj.AddComponent<RectTransform>();
    ringRT.anchorMin = Vector2.zero;
    ringRT.anchorMax = Vector2.one;
    ringRT.sizeDelta = Vector2.zero;
    Image ringImg = ringObj.AddComponent<Image>();
    ringImg.sprite = ringSprite;
    ringImg.color = accentColor;
    ringImg.raycastTarget = false;

```

```

GameObject txtObj = new GameObject("Label");
txtObj.transform.SetParent(btnObj.transform, false);
RectTransform txtRT = txtObj.AddComponent<RectTransform>();
txtRT.anchorMin = Vector2.zero;
txtRT.anchorMax = Vector2.one;
txtRT.sizeDelta = Vector2.zero;

TextMeshProUGUI tmp = txtObj.AddComponent<TextMeshProUGUI>();
tmp.text = labelText;
tmp.fontSize = fontSize;
tmp.fontStyle = FontStyles.Bold;
tmp.alignment = TextAlignmentOptions.Center;
tmp.color = Color.white;
tmp.enableWordWrapping = false;
tmp.raycastTarget = false;

TouchButtonTrigger trigger = btnObj.AddComponent<TouchButtonTrigger>();
trigger.Init(accentColor, onDown, onUp);

return rt;
}

RectTransform CreatePillButton(Transform parent, string name, Vector2 anchoredPos, Vector2
size,
Color accentColor, string labelText, int fontSize, System.Action onDown)
{
    GameObject btnObj = new GameObject(name);
    btnObj.transform.SetParent(parent, false);

    RectTransform rt = btnObj.AddComponent<RectTransform>();
    rt.anchorMin = new Vector2(1, 0);
    rt.anchorMax = new Vector2(1, 0);
    rt.pivot = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = anchoredPos;
    rt.sizeDelta = size;

    Image bgImg = btnObj.AddComponent<Image>();
    bgImg.color = new Color(0.06f, 0.08f, 0.15f, 0.85f);
    bgImg.raycastTarget = true;

    Outline outline = btnObj.AddComponent<Outline>();
    outline.effectColor = accentColor;
    outline.effectDistance = new Vector2(2f, 2f);

    GameObject txtObj = new GameObject("Label");
    txtObj.transform.SetParent(btnObj.transform, false);
    RectTransform txtRT = txtObj.AddComponent<RectTransform>();
    txtRT.anchorMin = Vector2.zero;
    txtRT.anchorMax = Vector2.one;
    txtRT.sizeDelta = Vector2.zero;

    TextMeshProUGUI tmp = txtObj.AddComponent<TextMeshProUGUI>();
    tmp.text = labelText;
    tmp.fontSize = fontSize;
    tmp.fontStyle = FontStyles.Bold;
    tmp.alignment = TextAlignmentOptions.Center;
    tmp.color = Color.white;
    tmp.raycastTarget = false;

    TouchButtonTrigger trigger = btnObj.AddComponent<TouchButtonTrigger>();
    trigger.Init(accentColor, onDown, null);

    return rt;
}

void GenerateProceduralSprites()
{
    if (circleSprite != null && ringSprite != null) return;

    int res = 128;
    Texture2D circleTex = new Texture2D(res, res, TextureFormat.RGBA32, false);
    Texture2D ringTex = new Texture2D(res, res, TextureFormat.RGBA32, false);

    float center = res * 0.5f;
    float radius = center - 2f;

```

```

float innerRadius = radius - 7f;

for (int y = 0; y < res; y++)
{
    for (int x = 0; x < res; x++)
    {
        float dist = Vector2.Distance(new Vector2(x, y), new Vector2(center, center));

        float circleAlpha = Mathf.Clamp01((radius - dist) + 0.5f);
        circleTex.SetPixel(x, y, new Color(1, 1, 1, circleAlpha));

        float outerAlpha = Mathf.Clamp01((radius - dist) + 0.5f);
        float innerAlpha = Mathf.Clamp01((dist - innerRadius) + 0.5f);
        float ringAlpha = Mathf.Min(outerAlpha, innerAlpha);
        ringTex.SetPixel(x, y, new Color(1, 1, 1, ringAlpha));
    }
}

circleTex.Apply();
ringTex.Apply();

circleSprite = Sprite.Create(circleTex, new Rect(0, 0, res, res), new Vector2(0.5f, 0.5f));
ringSprite = Sprite.Create(ringTex, new Rect(0, 0, res, res), new Vector2(0.5f, 0.5f));
;
}
}

/// <summary>
/// TouchDPadButton - Dedicated Left/Right directional button:
/// 1. Continuously asserts virtualHorizontalInput every single frame while held.
/// 2. Implements IDragHandler to lock pointer capture so finger micro-drifts never drop input
/// 3. Detects double-tap natively to trigger the Dash Run.
/// 4. Provides tactile scale punch on press/release.
/// </summary>
public class TouchDPadButton : MonoBehaviour, IPointerDownHandler, IPointerUpHandler, IDragHandler
{
    private float direction; // -1 for Left, +1 for Right
    private bool isHeld = false;
    private float lastTapTime = -10f;
    private const float doubleTapThreshold = 0.35f;
    private Coroutine scaleCoroutine;

    public void Init(float dir)
    {
        direction = dir;
    }

    public void OnPointerDown(PointerEventData eventData)
    {
        isHeld = true;

        float timeSinceLast = Time.unscaledTime - lastTapTime;
        bool isDoubleTap = timeSinceLast <= doubleTapThreshold;
        lastTapTime = Time.unscaledTime;

        if (move.Instance != null)
        {
            if (direction < 0f)
            {
                move.Instance.virtualLeftDown = true;
            }
            else
            {
                move.Instance.virtualRightDown = true;
            }

            move.Instance.virtualHorizontalInput = direction;
        }

        if (scaleCoroutine != null) StopCoroutine(scaleCoroutine);
        scaleCoroutine = StartCoroutine(AnimateScale(0.88f, 0.05f));
    }
}

```

```

public void OnDrag(PointerEventData eventData)
{
    // Maintains unbroken pointer capture on touchscreens even with finger drift
}

public void OnPointerUp(PointerEventData eventData)
{
    isHeld = false;

    if (move.Instance != null)
    {
        // Reset horizontal input if this button was active
        if ((direction < 0f && move.Instance.virtualHorizontalInput < 0f) ||
            (direction > 0f && move.Instance.virtualHorizontalInput > 0f))
        {
            move.Instance.virtualHorizontalInput = 0f;
        }
    }

    if (scaleCoroutine != null) StopCoroutine(scaleCoroutine);
    scaleCoroutine = StartCoroutine(AnimateScale(1.0f, 0.10f));
}

private RectTransform rectTransform;

void Awake()
{
    rectTransform = GetComponent<RectTransform>();
}

void Update()
{
    // Direct multi-touch hit-test fallback for Windows touchscreens & mobile:
    // Even if Windows OS tries to drop or gesture-cancel the pointer event,
    // we check if any active touch or mouse is physically inside this button's rect!
    bool isTouchInside = false;

    for (int i = 0; i < Input.touchCount; i++)
    {
        Touch t = Input.GetTouch(i);
        if (t.phase != TouchPhase.Ended && t.phase != TouchPhase.Canceled)
        {
            if (rectTransform != null && RectTransformUtility.RectangleContainsScreenPoint
(rectTransform, t.position, null))
            {
                isTouchInside = true;
                break;
            }
        }
    }

    if (Input.GetMouseButton(0))
    {
        if (rectTransform != null && RectTransformUtility.RectangleContainsScreenPoint(rec
tTransform, Input.mousePosition, null))
        {
            isTouchInside = true;
        }
    }

    bool activeHold = isHeld || isTouchInside;

    if (activeHold && move.Instance != null)
    {
        move.Instance.virtualHorizontalInput = direction;
    }
}

private IEnumerator AnimateScale(float targetScale, float duration)
{
    Vector3 start = transform.localScale;
    Vector3 end = Vector3.one * targetScale;
    float elapsed = 0f;

    while (elapsed < duration)
    {

```

```

        elapsed += Time.unscaledDeltaTime;
        transform.localScale = Vector3.Lerp(start, end, elapsed / duration);
        yield return null;
    }

    transform.localScale = end;
    scaleCoroutine = null;
}

/// <summary>
/// TouchButtonTrigger - Handles tactile scale punch (0.90x) and triggers game action.
/// Implements IDragHandler to prevent finger micro-drift from breaking hold.
/// </summary>
public class TouchButtonTrigger : MonoBehaviour, IPointerDownHandler, IPointerUpHandler, IDrag
    Handler
{
    private Color baseAccentColor;
    private System.Action onDownAction;
    private System.Action onUpAction;
    private Coroutine scaleCoroutine;

    public void Init(Color accent, System.Action onDown, System.Action onUp)
    {
        baseAccentColor = accent;
        onDownAction = onDown;
        onUpAction = onUp;
    }

    public void OnPointerDown(PointerEventData eventData)
    {
        if (scaleCoroutine != null) StopCoroutine(scaleCoroutine);
        scaleCoroutine = StartCoroutine(AnimateScale(0.90f, 0.05f));

        onDownAction?.Invoke();
    }

    public void OnDrag(PointerEventData eventData)
    {
        // Maintains unbroken pointer capture
    }

    public void OnPointerUp(PointerEventData eventData)
    {
        if (scaleCoroutine != null) StopCoroutine(scaleCoroutine);
        scaleCoroutine = StartCoroutine(AnimateScale(1.0f, 0.10f));

        onUpAction?.Invoke();
    }

    private IEnumerator AnimateScale(float targetScale, float duration)
    {
        Vector3 start = transform.localScale;
        Vector3 end = Vector3.one * targetScale;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.unscaledDeltaTime;
            transform.localScale = Vector3.Lerp(start, end, elapsed / duration);
            yield return null;
        }

        transform.localScale = end;
        scaleCoroutine = null;
    }
}

```

FILE 29: Assets/Scenes/scripts/Minigames/OrbSplash/CollectibleOrb.cs
 PURPOSE: Collectible orb movement and pickup.

```

using System;
using System.Collections;
using SpawnOfChaos.Minigames;
using SpawnOfChaos.Systems;

```

```

using UnityEngine;
using TMPro;

namespace SpawnOfChaos.Entities
{
    /// <summary>
    /// Collectible 2D World Orb item that drops in the environment or from enemies.
    /// Features:
    /// - Skybound-style floating bob and ambient sparkling trail.
    /// - Skybound-style explosive pickup burst: shockwave ring, radiant velocity shards, and
    floating text.
    /// - Distinct procedural 2D shapes (Heart, Arcane Tear, Diamond Coin, Cosmic Crystal) and
    scale variations.
    /// - Real-time stat restoration & HUD liquid splash synchronization.
    /// </summary>
    [RequireComponent(typeof(SpriteRenderer), typeof(CircleCollider2D))]
    public class CollectibleOrb : MonoBehaviour
    {
        public OrbType orbType = OrbType.Currency;
        public int restoreAmount = 10;
        public AudioClip pickupSFX;

        [Header("Magnet Settings")]
        public float magnetRadius = 4.5f;
        public float moveSpeed = 8f;

        private Transform playerTransform;
        private SpriteRenderer spriteRenderer;
        private Vector3 basePosition;
        private float floatTimer;
        private float trailTimer;
        private bool isBeingCollected = false;

        void Awake()
        {
            spriteRenderer = GetComponent<SpriteRenderer>();
            CircleCollider2D col = GetComponent<CircleCollider2D>();
            col.isTrigger = true;
            col.radius = 0.45f;

            ApplyScaleByType();
            EnsureProceduralSprite();
        }

        void Start()
        {
            basePosition = transform.position;
            floatTimer = UnityEngine.Random.Range(0f, 10f);
            FindPlayer();
        }

        private void ApplyScaleByType()
        {
            switch (orbType)
            {
                case OrbType.Health:
                    transform.localScale = Vector3.one * 1.15f;
                    break;
                case OrbType.Currency:
                    transform.localScale = Vector3.one * 1.0f;
                    break;
                case OrbType.Mana:
                    transform.localScale = Vector3.one * 0.95f;
                    break;
                case OrbType.EP:
                    transform.localScale = Vector3.one * 0.85f;
                    break;
            }
        }

        void FindPlayer()
        {
            GameObject p = GameObject.FindGameObjectWithTag("Player");
            if (p == null) p = GameObject.Find("Player");
            if (p == null) p = GameObject.Find("BasePlayer");
            if (p != null) playerTransform = p.transform;
        }
    }
}

```

```

        else if (PlayerCurrency.Instance != null) playerTransform = PlayerCurrency.Instance.transform;
    }

    void Update()
    {
        if (!Application.isPlaying) return;
        if (HUDManager.IsInMainMenu()) return;

        if (playerTransform == null)
        {
            FindPlayer();
        }

        // Skybound-style floating oscillation bob
        floatTimer += Time.deltaTime * 5f;
        float bobY = Mathf.Sin(floatTimer) * 0.16f;

        // Ambient sparkle trail tick
        trailTimer += Time.deltaTime;
        if (trailTimer >= 0.25f && !isBeingCollected)
        {
            trailTimer = 0f;
            SpawnAmbientTrailSpark();
        }

        if (playerTransform != null)
        {
            float dist = Vector3.Distance(transform.position, playerTransform.position);

            // Magnetic homing towards player when close
            if (dist <= magnetRadius)
            {
                transform.position = Vector3.MoveTowards(transform.position, playerTransform.position, moveSpeed * Time.deltaTime);
                moveSpeed += Time.deltaTime * 14f; // rapid acceleration

                if (dist < 0.6f && !isBeingCollected)
                {
                    Collect(playerTransform.gameObject);
                }
                return;
            }
        }

        transform.position = basePosition + new Vector3(0f, bobY, 0f);
    }

    private void SpawnAmbientTrailSpark()
    {
        Color32 orbCol = GetOrbColor(orbType);
        GameObject spark = new GameObject("TrailSpark");
        spark.transform.position = transform.position + new Vector3(UnityEngine.Random.Range(-0.15f, 0.15f), 0f);
        spark.transform.localScale = Vector3.one * 0.12f;

        SpriteRenderer sr = spark.AddComponent<SpriteRenderer>();
        sr.sprite = spriteRenderer.sprite;
        sr.color = new Color(orbCol.r / 255f, orbCol.g / 255f, orbCol.b / 255f, 0.65f);
        sr.sortingOrder = 5;

        StartCoroutine(FadeAndDestroy(spark, 0.35f));
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (HUDManager.IsInMainMenu()) return;
        if (isBeingCollected) return;

        if (other.CompareTag("Player") || other.name.Contains("Player") || other.GetComponent<Health>() != null || other.GetComponent<PlayerCurrency>() != null)
        {
            Collect(other.gameObject);
        }
    }
}

```

```

public void Collect(GameObject player)
{
    if (isBeingCollected) return;
    isBeingCollected = true;

    string rewardText = "";
    Color textColor = Color.white;

    // Apply stat restoration based on OrbType
    switch (orbType)
    {
        case OrbType.Health:
            Health h = player.GetComponent<Health>();
            if (h != null) h.Heal(restoreAmount);
            rewardText = $"#{restoreAmount} HP!";
            textColor = new Color(1f, 0.2f, 0.3f, 1f);
            break;

        case OrbType.Mana:
            MageCombat mc = player.GetComponent<MageCombat>();
            if (mc != null) mc.currentMana = Mathf.Min(mc.maxMana, mc.currentMana + restoreAmount);
            rewardText = $"#{restoreAmount} MANA!";
            textColor = new Color(0f, 0.85f, 1f, 1f);
            break;

        case OrbType.Currency:
            if (PlayerCurrency.Instance != null)
            {
                PlayerCurrency.Instance.AddCoins(restoreAmount);
            }
            rewardText = $"#{restoreAmount} COINS!";
            textColor = new Color(1f, 0.15f, 0.25f, 1f);
            break;

        case OrbType.EP:
            if (PlayerLevelSystem.Instance != null)
            {
                PlayerLevelSystem.Instance.AddExperience(restoreAmount);
            }
            rewardText = $"#{restoreAmount} EXP!";
            textColor = new Color(0.85f, 0.35f, 1f, 1f);
            break;
    }

    // Skybound-style explosive shockwave & velocity shard burst
    SpawnSkyboundExplosion();

    // Floating reward popup text
    SpawnFloatingRewardText(rewardText, textColor);

    // Trigger HUD Orb Splash in real-time
    if (HUDOrbPanel.Instance != null)
    {
        HUDOrbPanel.Instance.TriggerSplash(orbType, 1.8f);
    }

    // Play pickup sound
    if (pickupSFX != null && AudioManager.Instance != null)
    {
        AudioManager.Instance.PlaySFX(pickupSFX);
    }

    Debug.Log($"[CollectibleOrb] Collected {orbType} Orb ({rewardText})!");

    // Hide visual immediately while particles finish, then destroy
    if (spriteRenderer != null) spriteRenderer.enabled = false;
    Collider2D c = GetComponent<Collider2D>();
    if (c != null) c.enabled = false;

    Destroy(gameObject, 0.8f);
}

private void SpawnSkyboundExplosion()
{
    Vector3 pos = transform.position;

```

```

Color32 pCol = GetOrbColor(orbType);

// 1. Concentric shockwave ring
GameObject ring = new GameObject("ShockwaveRing");
ring.transform.position = pos;
ring.transform.localScale = Vector3.one * 0.2f;
SpriteRenderer ringSR = ring.AddComponent<SpriteRenderer>();
ringSR.sprite = spriteRenderer.sprite;
ringSR.color = new Color(pCol.r / 255f, pCol.g / 255f, pCol.b / 255f, 0.9f);
ringSR.sortingOrder = 10;
StartCoroutine(AnimateShockwaveRing(ring, 0.45f));

// 2. Radiant velocity shards (14 particles)
for (int i = 0; i < 14; i++)
{
    float angle = (i / 14f) * Mathf.PI * 2f + UnityEngine.Random.Range(-0.2f, 0.2f);

    float speed = UnityEngine.Random.Range(3.5f, 8.5f);
    Vector2 vel = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle)) * speed;

    GameObject shard = new GameObject("Shard");
    shard.transform.position = pos;
    shard.transform.localScale = Vector3.one * UnityEngine.Random.Range(0.10f, 0.18f);

    SpriteRenderer shardSR = shard.AddComponent<SpriteRenderer>();
    shardSR.sprite = spriteRenderer.sprite;
    shardSR.color = new Color(pCol.r / 255f, pCol.g / 255f, pCol.b / 255f, 1f);
    shardSR.sortingOrder = 11;

    StartCoroutine(AnimateShard(shard, vel, UnityEngine.Random.Range(0.35f, 0.6f)))
}

private IEnumerator AnimateShockwaveRing(GameObject ring, float duration)
{
    SpriteRenderer sr = ring.GetComponent<SpriteRenderer>();
    Color startColor = sr.color;
    Vector3 startScale = Vector3.one * 0.2f;
    Vector3 targetScale = Vector3.one * 1.6f;
    float elapsed = 0f;

    while (elapsed < duration)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / duration;
        ring.transform.localScale = Vector3.Lerp(startScale, targetScale, t);
        sr.color = new Color(startColor.r, startColor.g, startColor.b, (1f - t) * startColor.a);
        yield return null;
    }

    Destroy(ring);
}

private IEnumerator AnimateShard(GameObject shard, Vector2 velocity, float lifetime)
{
    SpriteRenderer sr = shard.GetComponent<SpriteRenderer>();
    Color startColor = sr.color;
    float elapsed = 0f;

    while (elapsed < lifetime)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / lifetime;

        shard.transform.position += (Vector3)velocity * Time.deltaTime;
        velocity *= 0.92f; // drag friction
        velocity.y -= 4f * Time.deltaTime; // slight gravity

        sr.color = new Color(startColor.r, startColor.g, startColor.b, (1f - t));
        yield return null;
    }

    Destroy(shard);
}

```

```

    }

    private void SpawnFloatingRewardText(string text, Color textColor)
    {
        GameObject txtGO = new GameObject("RewardText");
        txtGO.transform.position = transform.position + Vector3.up * 0.4f;

        TextMeshPro tmp = txtGO.AddComponent<TextMeshPro>();
        tmp.text = text;
        tmp.fontSize = 4.5f;
        tmp.fontStyle = FontStyles.Bold;
        tmp.alignment = TextAlignmentOptions.Center;
        tmp.color = textColor;
        tmp.sortingOrder = 20;

        StartCoroutine(AnimateFloatingText(txtGO, 0.7f));
    }

    private IEnumerator AnimateFloatingText(GameObject txtGO, float duration)
    {
        TextMeshPro tmp = txtGO.GetComponent<TextMeshPro>();
        Color startCol = tmp.color;
        Vector3 startPos = txtGO.transform.position;
        Vector3 endPos = startPos + Vector3.up * 1.2f;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.deltaTime;
            float t = elapsed / duration;

            txtGO.transform.position = Vector3.Lerp(startPos, endPos, Mathf.SmoothStep(0f,
1f, t));
            tmp.color = new Color(startCol.r, startCol.g, startCol.b, 1f - (t * t));
            yield return null;
        }

        Destroy(txtGO);
    }

    private IEnumerator FadeAndDestroy(GameObject target, float duration)
    {
        SpriteRenderer sr = target.GetComponent<SpriteRenderer>();
        Color c = sr.color;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.deltaTime;
            float t = elapsed / duration;
            sr.color = new Color(c.r, c.g, c.b, (1f - t) * c.a);
            yield return null;
        }

        Destroy(target);
    }

    private static readonly System.Collections.Generic.Dictionary<OrbType, Sprite> cachedS
prites = new System.Collections.Generic.Dictionary<OrbType, Sprite>();

    /// <summary>
    /// Generates distinct procedural glowing shapes (Heart, Arcane Tear, Diamond Coin, Co
smic Crystal).
    /// </summary>
    private void EnsureProceduralSprite()
    {
        if (spriteRenderer.sprite != null) return;

        if (cachedSprites.TryGetValue(orbType, out Sprite cached) && cached != null)
        {
            spriteRenderer.sprite = cached;
            return;
        }

        int size = 64;
        Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32, false);

```

```

Color32 pCol = GetOrbColor(orbType);
Color32 hCol = new Color32(255, 255, 255, 250);

float radius = size * 0.40f;
float cx = size / 2f;
float cy = size / 2f;

for (int y = 0; y < size; y++)
{
    for (int x = 0; x < size; x++)
    {
        float dx = x - cx;
        float dy = y - cy;
        bool inside = false;
        float shapeRatio = 1f;

        switch (orbType)
        {
            case OrbType.Currency:
                // Red Diamond Coin Shape (Rhombus facets)
                float dDiam = (Mathf.Abs(dx) / (radius * 0.85f)) + (Mathf.Abs(dy)
/ (radius * 1.15f));
                if (dDiam <= 1f)
                {
                    inside = true;
                    shapeRatio = dDiam;
                }
                break;

            case OrbType.Health:
                // Obsidian Crimson Heart Shape
                float nx = dx / (radius * 0.95f);
                float ny = (dy / (radius * 0.95f)) * 1.15f - 0.1f;
                float term = nx * nx + ny * ny - 1f;
                if (term * term * term - (nx * nx * ny * ny * ny) <= 0f)
                {
                    inside = true;
                    shapeRatio = Mathf.Clamp01(Mathf.Sqrt(dx * dx + dy * dy) / rad
ius);
                }
                break;

            case OrbType.Mana:
                // Arcane Teardrop with Radiant Rays
                float distM = Mathf.Sqrt(dx * dx + dy * dy);
                if (distM <= radius * 0.85f || (Mathf.Abs(dx) < 2f && Mathf.Abs(dy)
) < radius) || (Mathf.Abs(dy) < 2f && Mathf.Abs(dx) < radius))
                {
                    inside = true;
                    shapeRatio = distM / radius;
                }
                break;

            case OrbType.EP:
                // Cosmic 8-Pointed Crystal Star
                float distEP = Mathf.Sqrt(dx * dx + dy * dy);
                float ang = Mathf.Atan2(dy, dx);
                float ray = Mathf.Abs(Mathf.Cos(ang * 4f));
                if (distEP <= radius * (0.45f + ray * 0.55f))
                {
                    inside = true;
                    shapeRatio = distEP / radius;
                }
                break;
        }

        if (inside)
        {
            Color32 c = Color32.Lerp(hCol, pCol, shapeRatio);
            c.a = (byte)((1f - shapeRatio * 0.6f) * 255);
            tex.SetPixel(x, y, c);
        }
        else
        {
            tex.SetPixel(x, y, Color.clear);
        }
    }
}

```

```

        }
    }

    tex.Apply();
    Sprite newSprite = Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f
, 0.5f), 32);
    cachedSprites[orbType] = newSprite;
    spriteRenderer.sprite = newSprite;
}

public static Color32 GetOrbColor(OrbType type)
{
    switch (type)
    {
        case OrbType.Health: return new Color32(235, 25, 45, 255); // Crimson Heart
        case OrbType.Mana: return new Color32(0, 190, 255, 255); // Electric Neon
        case OrbType.Currency: return new Color32(220, 20, 35, 255); // Rich Dark Bloo
        case OrbType.EP: return new Color32(175, 45, 245, 255); // Cosmic Purple
        default: return new Color32(255, 255, 255, 255);
    }
}
}
}
}

```

FILE 30: Assets/Scenes/scripts/Minigames/OrbSplash/OrbSpawner.cs

PURPOSE: Spawns collectible orbs.

```

using SpawnOfChaos.Entities;
using SpawnOfChaos.Minigames;
using UnityEngine;

namespace SpawnOfChaos.Systems
{
    /// <summary>
    /// Utility class for spawning collectible 2D world Orbs (Health, Mana, Currency, EP).
    /// </summary>
    public static class OrbSpawner
    {
        /// <summary>
        /// Spawns a specific CollectibleOrb at the given world position.
        /// </summary>
        public static GameObject SpawnOrb(Vector3 position, OrbType type, int amount = 15)
        {
            GameObject orbGO = new GameObject($"CollectibleOrb_{type}");
            orbGO.transform.position = position;

            CollectibleOrb orb = orbGO.AddComponent<CollectibleOrb>();
            orb.orbType = type;
            orb.restoreAmount = amount;

            return orbGO;
        }

        /// <summary>
        /// Drops a random cluster of loot orbs (Health, Mana, Currency, EXP) at a location (e
.g. on enemy death or chest open).
        /// </summary>
        public static void SpawnLootCluster(Vector3 position, int orbCount = 3)
        {
            for (int i = 0; i < orbCount; i++)
            {
                Vector3 offset = new Vector3(Random.Range(-0.8f, 0.8f), Random.Range(-0.4f, 0.
6f), 0f);
                float roll = Random.value;

                OrbType type;
                int amount;

                if (roll < 0.35f)
                {
                    type = OrbType.EP; // Experience
                    amount = Random.Range(20, 35);
                }
            }
        }
    }
}

```

```

    }
    else if (roll < 0.65f)
    {
        type = OrbType.Currency; // Coins
        amount = Random.Range(5, 15);
    }
    else if (roll < 0.85f)
    {
        type = OrbType.Mana;
        amount = Random.Range(15, 30);
    }
    else
    {
        type = OrbType.Health;
        amount = Random.Range(2, 5);
    }

    SpawnOrb(position + offset, type, amount);
}
}
}
}
}

```

FILE 31: Assets/Scenes/scripts/Minigames/MinigameHubUI.cs
PURPOSE: Minigame selection interface.

```

using UnityEngine;
using UnityEngine.UI;
using TMPro;

namespace SpawnOfChaos.Minigames
{
    /// <summary>
    /// Central Minigame Hub Selector UI opened directly from the Pause Menu.
    /// Premium dark arcade aesthetic with neon-accented game cards, rounded corners,
    /// colored accent stripes, and rich visual hierarchy.
    /// Lists all available arcade minigames with expandable slots for future additions!
    /// </summary>
    public class MinigameHubUI : MonoBehaviour
    {
        public static MinigameHubUI Instance { get; private set; }

        private GameObject hubModal;

        void Awake()
        {
            if (Instance == null)
            {
                Instance = this;
                DontDestroyOnLoad(gameObject);
            }
            else if (Instance != this)
            {
                Destroy(gameObject);
            }
        }

        public static void OpenHub()
        {
            if (Instance == null)
            {
                GameObject go = new GameObject("MinigameHubManager");
                Instance = go.AddComponent<MinigameHubUI>();
            }
            Instance.ShowHubWindow();
        }

        public void ShowHubWindow()
        {
            if (hubModal == null) BuildUI();
            hubModal.SetActive(true);
        }

        public void CloseHubWindow()
        {

```

```

        if (hubModal != null) hubModal.SetActive(false);
    }

    private void BuildUI()
    {
        Canvas canvas = UIFactory.CreateCanvas("MinigameHubCanvas", 22);
        canvas.transform.SetParent(transform, false);

        CanvasScaler scaler = canvas.GetComponent<CanvasScaler>();
        if (scaler != null) scaler.matchWidthOrHeight = 1.0f;

        // ?????? Fullscreen dark overlay with subtle blue tint ??????
        RectTransform overlayRT = UIFactory.CreateFullScreenPanel(
            canvas.transform, "HubOverlay", new Color(0.01f, 0.015f, 0.04f, 0.95f));
        hubModal = overlayRT.gameObject;

        // ?????? Main Panel (centered modal) ??????
        RectTransform panelRT = UIFactory.CreatePanel(
            overlayRT, "HubPanel", new Color(0.025f, 0.02f, 0.06f, 0.97f),
            new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
        panelRT.sizeDelta = new Vector2(620f, 760f);

        // Rounded corners for main panel
        Image panelImg = panelRT.GetComponent<Image>();
        Sprite roundedSprite = UIFactory.GetRoundedSprite();
        if (roundedSprite != null && panelImg != null)
        {
            panelImg.sprite = roundedSprite;
            panelImg.type = Image.Type.Sliced;
        }

        // Subtle border glow
        RectTransform borderGlow = UIFactory.CreatePanel(panelRT, "BorderGlow",
            new Color(0.57f, 0.41f, 1f, 0.06f),
            Vector2.zero, Vector2.one, new Vector2(-3f, -3f), new Vector2(3f, 3f));
        Image borderImg = borderGlow.GetComponent<Image>();
        if (roundedSprite != null && borderImg != null)
        {
            borderImg.sprite = roundedSprite;
            borderImg.type = Image.Type.Sliced;
        }

        // ?????????????????????????????????????????????????????????????????????????????????????
        ?????????????????????????????????????????????????????????????????????????????????????
        //  HEADER SECTION
        // ?????????????????????????????????????????????????????????????????????????????????????
        ?????????????????????????????????????????????????????????????????????????????????????

        // Small decorative top accent bar
        RectTransform topAccent = UIFactory.CreatePanel(panelRT, "TopAccent",
            new Color(0.57f, 0.41f, 1f, 0.5f),
            new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
        topAccent.sizeDelta = new Vector2(540f, 2f);
        topAccent.anchoredPosition = new Vector2(0f, -20f);

        // Title: "ARCADE"
        TextMeshProUGUI titleText = UIFactory.CreateText(
            panelRT, "Title", "ARCADE", 40f,
            new Color(0.57f, 0.41f, 1f, 1f), TextAlignmentOptions.Center);
        titleText.fontStyle = FontStyles.Bold;
        titleText.characterSpacing = 8f;
        UIFactory.SetRectTransform(titleText.rectTransform,
            new Vector2(0.5f, 1f), new Vector2(0.5f, 1f),
            new Vector2(0f, -52f), new Vector2(500f, 50f));

        // Subtitle
        TextMeshProUGUI subtitleText = UIFactory.CreateText(
            panelRT, "Subtitle", "Select a minigame to play while paused", 13f,
            new Color(0.55f, 0.6f, 0.75f, 0.7f), TextAlignmentOptions.Center);
        UIFactory.SetRectTransform(subtitleText.rectTransform,
            new Vector2(0.5f, 1f), new Vector2(0.5f, 1f),
            new Vector2(0f, -88f), new Vector2(500f, 24f));

        // Divider below header
        RectTransform headerDiv = UIFactory.CreatePanel(panelRT, "HeaderDivider",
            new Color(0.57f, 0.41f, 1f, 0.12f),

```

[illegible]

```

        OrbSplashArcadeUI.ShowMinigame();
    }
};

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// FOOTER
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????

// Divider above footer
RectTransform footerDiv = UIFactory.CreatePanel(panelRT, "FooterDivider",
    new Color(0.57f, 0.41f, 1f, 0.08f),
    new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));
footerDiv.sizeDelta = new Vector2(540f, 1f);
footerDiv.anchoredPosition = new Vector2(0f, 75f);

// Close button with rounded corners
GameObject closeBtnGO = new GameObject("CloseBtn", typeof(RectTransform), typeof(I
mage), typeof(Button));
closeBtnGO.transform.SetParent(panelRT, false);
RectTransform closeRT = closeBtnGO.GetComponent<RectTransform>();
closeRT.sizeDelta = new Vector2(260f, 44f);
closeRT.anchoredPosition = new Vector2(0f, -338f);

Image closeImg = closeBtnGO.GetComponent<Image>();
closeImg.color = new Color(0.08f, 0.06f, 0.16f, 0.9f);
if (roundedSprite != null)
{
    closeImg.sprite = roundedSprite;
    closeImg.type = Image.Type.Sliced;
}

TextMeshProUGUI closeLabel = UIFactory.CreateText(
    closeBtnGO.transform, "Label", "??? BACK TO PAUSE MENU", 12f,
    new Color(0.6f, 0.65f, 0.8f, 0.9f), TextAlignmentOptions.Center);
closeLabel.characterSpacing = 2f;
UIFactory.SetRect(closeLabel.rectTransform, Vector2.zero, Vector2.one, Vector2.zer
o, Vector2.zero);

Button closeBtn = closeBtnGO.GetComponent<Button>();
closeBtn.targetGraphic = closeImg;
ColorBlock closeCB = closeBtn.colors;
closeCB.normalColor = new Color(0.08f, 0.06f, 0.16f, 0.9f);
closeCB.highlightedColor = new Color(0.14f, 0.12f, 0.28f, 1f);
closeCB.pressedColor = new Color(0.05f, 0.04f, 0.10f, 1f);
closeCB.fadeDuration = 0.08f;
closeBtn.colors = closeCB;
closeBtn.onClick.AddListener(() => CloseHubWindow());

hubModal.SetActive(false);
}

/// <summary>
/// Creates a premium minigame card with colored accent stripe, icon, title, descripti
on, and play button.
/// </summary>
private void CreateMinigameCard(Transform parent, string title, string desc,
    string icon, string btnText, Color accentColor, UnityEngine.Events.UnityAction onC
lick, bool isEnabled = true)
{
    Sprite roundedSprite = UIFactory.GetRoundedSprite();

    // ?????? Card Container ??????
    GameObject cardGO = new GameObject("Card_" + title, typeof(RectTransform), typeof(
Image));
    cardGO.transform.SetParent(parent, false);
    RectTransform cardRT = cardGO.GetComponent<RectTransform>();
    cardRT.sizeDelta = new Vector2(540f, 108f);

    Image cardImg = cardGO.GetComponent<Image>();
    cardImg.color = isEnabled
        ? new Color(0.04f, 0.035f, 0.10f, 0.95f)
        : new Color(0.025f, 0.02f, 0.06f, 0.5f);

    if (roundedSprite != null)

```

```

{
    cardImg.sprite = roundedSprite;
    cardImg.type = Image.Type.Sliced;
}

// ?????? Colored Left Accent Stripe ??????
RectTransform accentStripe = UIFactory.CreatePanel(cardRT, "AccentStripe",
    accentColor,
    new Vector2(0f, 0f), new Vector2(0f, 1f));
accentStripe.sizeDelta = new Vector2(4f, 0f);
accentStripe.anchoredPosition = new Vector2(2f, 0f);
// Slight margin from edges
accentStripe.offsetMin = new Vector2(0f, 8f);
accentStripe.offsetMax = new Vector2(4f, -8f);

// ?????? Icon ??????
TextMeshProUGUI iconText = UIFactory.CreateText(cardRT, "Icon", icon, 26f,
    accentColor, TextAlignmentOptions.Center);
UIFactory.SetRectFixed(iconText.rectTransform,
    new Vector2(0f, 0.5f), new Vector2(0f, 0.5f),
    new Vector2(32f, 0f), new Vector2(40f, 40f));

// ?????? Title ??????
TextMeshProUGUI tText = UIFactory.CreateText(cardRT, "Title", title, 18f,
    accentColor, TextAlignmentOptions.Left);
tText.fontStyle = FontStyles.Bold;
tText.characterSpacing = 3f;
UIFactory.SetRectFixed(tText.rectTransform,
    new Vector2(0f, 1f), new Vector2(0f, 1f),
    new Vector2(62f, -30f), new Vector2(300f, 28f));

// ?????? Description ??????
TextMeshProUGUI dText = UIFactory.CreateText(cardRT, "Desc", desc, 12f,
    new Color(0.55f, 0.6f, 0.75f, 0.8f), TextAlignmentOptions.Left);
UIFactory.SetRectFixed(dText.rectTransform,
    new Vector2(0f, 0f), new Vector2(0f, 0f),
    new Vector2(62f, 22f), new Vector2(320f, 40f));

// ?????? Play Button ??????
if (isEnabled && onClick != null)
{
    GameObject btnGO = new GameObject("ActionBtn", typeof(RectTransform), typeof(I
mage), typeof(Button));
    btnGO.transform.SetParent(cardRT, false);
    RectTransform btnRT = btnGO.GetComponent<RectTransform>();
    btnRT.sizeDelta = new Vector2(110f, 38f);
    btnRT.anchorMin = new Vector2(1f, 0.5f);
    btnRT.anchorMax = new Vector2(1f, 0.5f);
    btnRT.anchoredPosition = new Vector2(-72f, 0f);

    Image btnImg = btnGO.GetComponent<Image>();
    // Button background uses a tinted version of the accent color
    Color btnBg = new Color(accentColor.r * 0.15f, accentColor.g * 0.15f, accentCo
lor.b * 0.15f, 0.9f);
    btnImg.color = btnBg;

    if (roundedSprite != null)
    {
        btnImg.sprite = roundedSprite;
        btnImg.type = Image.Type.Sliced;
    }

    TextMeshProUGUI btnLabel = UIFactory.CreateText(
        btnGO.transform, "Text", btnText + " ???", 13f,
        accentColor, TextAlignmentOptions.Center);
    btnLabel.fontStyle = FontStyles.Bold;
    btnLabel.characterSpacing = 3f;
    UIFactory.SetRect(btnLabel.rectTransform, Vector2.zero, Vector2.one, Vector2.z
ero, Vector2.zero);

    Button btn = btnGO.GetComponent<Button>();
    btn.targetGraphic = btnImg;
    ColorBlock cb = btn.colors;
    cb.normalColor = btnBg;
    cb.highlightedColor = new Color(accentColor.r * 0.3f, accentColor.g * 0.3f, ac
centColor.b * 0.3f, 1f);

```

```

        cb.pressedColor = new Color(accentColor.r * 0.1f, accentColor.g * 0.1f, accent
Color.b * 0.1f, 1f);
        cb.fadeDuration = 0.08f;
        btn.colors = cb;
        btn.onClick.AddListener(onClick);
    }
}
}
}
}

```

FILE 32: Assets/Scenes/scripts/Minigames/OrbSplash/OrbSplashArcadeUI.cs
PURPOSE: Orb Splash minigame UI.

```

using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;

namespace SpawnOfChaos.Minigames
{
    /// <summary>
    /// Interactive Arcade Minigame / Showcase UI for Procedural Liquid Orbs (Health, Mana, Cu
urrency, EP).
    /// Displays real-time procedural orb rendering with splash particle physics, fill control
s, and splash triggers.
    /// </summary>
    public class OrbSplashArcadeUI : MonoBehaviour
    {
        public static OrbSplashArcadeUI Instance { get; private set; }

        private GameObject modal;
        private List<ProceduralOrbUI> activeOrbUIs = new List<ProceduralOrbUI>();

        public bool IsMinigameActive => modal != null && modal.activeSelf;

        void Awake()
        {
            if (Instance != null && Instance != this)
            {
                Destroy(gameObject);
                return;
            }
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }

        public static void ShowMinigame()
        {
            if (Instance == null)
            {
                GameObject go = new GameObject("OrbSplashArcadeManager");
                Instance = go.AddComponent<OrbSplashArcadeUI>();
            }
            Instance.Open();
        }

        public void Open()
        {
            if (modal == null) BuildUI();
            modal.SetActive(true);
        }

        public void Close()
        {
            if (modal != null) modal.SetActive(false);
        }

        private void BuildUI()
        {
            Canvas canvas = UIFactory.CreateCanvas("OrbSplashArcadeCanvas", 26);
            canvas.transform.SetParent(transform, false);

            CanvasScaler scaler = canvas.GetComponent<CanvasScaler>();
            if (scaler != null) scaler.matchWidthOrHeight = 1.0f;
        }
    }
}

```

```

// Fullscreen dark backdrop
RectTransform overlay = UIFactory.CreateFullScreenPanel(
    canvas.transform, "MinigameOverlay", new Color(0.015f, 0.02f, 0.05f, 0.94f));
modal = overlay.gameObject;

// Main Window Frame
RectTransform frame = UIFactory.CreatePanel(overlay, "ArcadeFrame",
    new Color(0.04f, 0.05f, 0.09f, 0.96f),
    new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
frame.sizeDelta = new Vector2(900f, 550f);

// Header Title
TextMeshProUGUI title = UIFactory.CreateText(
    frame, "Title", "ARCADE PROCEDURAL ORBS", 28f, new Color(0.35f, 0.88f, 1f, 1f)
, TextAlignmentOptions.Center);
UIFactory.SetRectFixed(title.rectTransform, new Vector2(0.5f, 1f), new Vector2(0.5
f, 1f), new Vector2(0f, -28f), new Vector2(600f, 40f));

TextMeshProUGUI subtitle = UIFactory.CreateText(
    frame, "Subtitle", "Procedural Dual-Wave Liquid Physics, Emblem Motifs & Dynam
ic Splashes", 13f, new Color(0.6f, 0.72f, 0.85f, 0.85f), TextAlignmentOptions.Center);
UIFactory.SetRectFixed(subtitle.rectTransform, new Vector2(0.5f, 1f), new Vector2(
0.5f, 1f), new Vector2(0f, -54f), new Vector2(700f, 24f));

// Grid Container for 4 Orbs
GameObject gridGO = new GameObject("OrbGrid", typeof(RectTransform));
gridGO.transform.SetParent(frame, false);
RectTransform gridRT = gridGO.GetComponent<RectTransform>();
UIFactory.SetRectFixed(gridRT, new Vector2(0.5f, 0.52f), new Vector2(0.5f, 0.52f),
Vector2.zero, new Vector2(840f, 360f));

GridLayoutGroup glg = gridGO.AddComponent<GridLayoutGroup>();
glg.cellSize = new Vector2(195f, 340f);
glg.spacing = new Vector2(16f, 0f);
glg.childAlignment = TextAnchor.MiddleCenter;

activeOrbUIs.Clear();

// Create 4 Orbs: Health, Mana, Currency, EP
CreateOrbCard(gridRT, OrbType.Health, "HEALTH ORB", "Crimson Vitality", new Color(
1f, 0.2f, 0.35f, 1f));
CreateOrbCard(gridRT, OrbType.Mana, "MANA ORB", "Arcane Sapphire", new Color(0.2f,
0.75f, 1f, 1f));
CreateOrbCard(gridRT, OrbType.Currency, "CURRENCY ORB", "Liquid Gold", new Color(1
f, 0.78f, 0.15f, 1f));
CreateOrbCard(gridRT, OrbType.EP, "EP ORB", "Cosmic EXP", new Color(0.75f, 0.25f,
1f, 1f));

// Bottom Global Action Bar
GameObject bottomBar = new GameObject("BottomBar", typeof(RectTransform));
bottomBar.transform.SetParent(frame, false);
RectTransform bottomRT = bottomBar.GetComponent<RectTransform>();
UIFactory.SetRectFixed(bottomRT, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f), new
Vector2(0f, 35f), new Vector2(840f, 48f));

// "SPLASH ALL" Button
CreateArcaneBtn(bottomRT, "SPLASH ALL ORBS! ????", new Vector2(-150f, 0f), new Vec
tor2(240f, 42f),
    new Color(0.85f, 0.25f, 0.95f, 1f), () =>
    {
        foreach (var orb in activeOrbUIs)
        {
            orb.TriggerSplash(1.5f);
        }
    });

// "CLOSE" Button
CreateArcaneBtn(bottomRT, "BACK TO MENU", new Vector2(150f, 0f), new Vector2(220f,
42f),
    new Color(0.16f, 0.22f, 0.35f, 1f), () => Close());
}

private void CreateOrbCard(Transform parent, OrbType type, string cardTitle, string su
btitleText, Color accentColor)
{
    GameObject cardGO = new GameObject("Card_" + type, typeof(RectTransform), typeof(I

```

```

mage));
    cardGO.transform.SetParent(parent, false);
    Image cardBg = cardGO.GetComponent<Image>();
    cardBg.color = new Color(0.06f, 0.08f, 0.14f, 0.92f);

    RectTransform cardRT = cardGO.GetComponent<RectTransform>();

    // Title Label
    TextMeshProUGUI title = UIFactory.CreateText(cardRT, "Title", cardTitle, 15f, accentColor, TextAlignmentOptions.Center);
    title.fontStyle = FontStyles.Bold;
    UIFactory.SetRectFixed(title.rectTransform, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f), new Vector2(0f, -18f), new Vector2(180f, 22f));

    TextMeshProUGUI sub = UIFactory.CreateText(cardRT, "Sub", subtitleText, 11f, new Color(0.6f, 0.7f, 0.8f, 0.8f), TextAlignmentOptions.Center);
    UIFactory.SetRectFixed(sub.rectTransform, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f), new Vector2(0f, -36f), new Vector2(180f, 18f));

    // RawImage for Procedural Orb Display (140x140)
    GameObject orbGO = new GameObject("ProceduralOrbDisplay", typeof(RectTransform), typeof(RawImage), typeof(ProceduralOrbUI));
    orbGO.transform.SetParent(cardRT, false);

    RectTransform orbRT = orbGO.GetComponent<RectTransform>();
    UIFactory.SetRectFixed(orbRT, new Vector2(0.5f, 0.56f), new Vector2(0.5f, 0.56f), Vector2.zero, new Vector2(140f, 140f));

    ProceduralOrbUI orbUI = orbGO.GetComponent<ProceduralOrbUI>();
    orbUI.orbType = type;
    orbUI.fillAmount = 0.75f;
    orbUI.textureResolution = 140;
    orbUI.autoUpdate = true;
    orbUI.autoOscillateFill = false;

    activeOrbUIs.Add(orbUI);

    // SPLASH Button
    CreateArcaneBtn(cardRT, "SPLASH! ????", new Vector2(0f, -85f), new Vector2(160f, 34f), accentColor, () =>
    {
        orbUI.TriggerSplash(1.2f);
    });

    // FILL + / DRAIN - Buttons
    CreateArcaneBtn(cardRT, "FILL +", new Vector2(45f, -125f), new Vector2(70f, 28f), new Color(0.18f, 0.35f, 0.22f, 1f), () =>
    {
        orbUI.fillAmount = Mathf.Clamp01(orbUI.fillAmount + 0.15f);
        orbUI.TriggerSplash(0.6f);
    });

    CreateArcaneBtn(cardRT, "DRAIN -", new Vector2(-45f, -125f), new Vector2(70f, 28f), new Color(0.38f, 0.18f, 0.22f, 1f), () =>
    {
        orbUI.fillAmount = Mathf.Clamp01(orbUI.fillAmount - 0.15f);
    });
}

private GameObject CreateArcaneBtn(Transform parent, string label, Vector2 pos, Vector2 size, Color color, UnityEngine.Events.UnityAction onClick)
{
    GameObject btnGO = new GameObject("Btn_" + label, typeof(RectTransform), typeof(Image), typeof(Button));
    btnGO.transform.SetParent(parent, false);

    RectTransform rt = btnGO.GetComponent<RectTransform>();
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;

    Image img = btnGO.GetComponent<Image>();
    img.color = color;

    Button btn = btnGO.GetComponent<Button>();
    ColorBlock cb = btn.colors;
    cb.normalColor = color;

```

```

        cb.highlightedColor = new Color(color.r * 1.25f, color.g * 1.25f, color.b * 1.25f,
1f);
        cb.pressedColor = new Color(color.r * 0.7f, color.g * 0.7f, color.b * 0.7f, 1f);
        btn.colors = cb;
        btn.onClick.AddListener(onClick);

        TextMeshProUGUI txt = UIFactory.CreateText(btnGO.transform, "Text", label, 12f, Co
lor.white, TextAlignmentOptions.Center);
        txt.fontStyle = FontStyles.Bold;
        UIFactory.SetRect(txt.rectTransform, Vector2.zero, Vector2.one, Vector2.zero, Vect
or2.zero);

        return btnGO;
    }
}
}

```

FILE 33: Assets/Scenes/scripts/Platforms/DynamicPinkBridgeCluster.cs
PURPOSE: Activates and animates bridge pieces.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// DynamicPinkBridgeCluster - Master manager script for platform clusters (e.g. TutorialScene
).
/// Automatically discovers all child platform objects and constructs a dynamic, glowing pink
bridge
/// that scales up in front of the player and retracts behind them as they walk across.
/// Features one-way effector colliders, spring-bounce scale assembly, neon pink color tinting
, and star particle bursts.
/// </summary>
public class DynamicPinkBridgeCluster : MonoBehaviour
{
    [Header("Proximity & Range Settings")]
    [Tooltip("Distance from player at which child platforms begin assembling.")]
    public float buildDistance = 6.5f;

    [Tooltip("Distance from player at which child platforms begin retracting.")]
    public float retractDistance = 8.5f;

    [Header("Animation Easing")]
    public float assembleSpeed = 12f;
    public float retractSpeed = 8f;

    [Tooltip("Spring overshoot factor when platform scales up (1.15 = 15% bounce).")]
    public float scaleBounceFactor = 1.15f;

    [Header("Neon Pink Visuals")]
    public Color activePinkColor = new Color(1f, 0.25f, 0.75f, 1f); // Neon Pink #FF40BF
    public bool enableStarParticles = true;
    public int starParticleCount = 14;

    [Header("Player Target Override")]
    public Transform playerTransform;

    private class BridgePieceData
    {
        public GameObject gameObject;
        public Transform transform;
        public SpriteRenderer spriteRenderer;
        public Collider2D collider;
        public PlatformEffector2D effector;
        public Vector3 originalLocalScale;
        public Vector3 originalLocalPos;
        public Quaternion originalLocalRot;
        public Color originalColor;

        public float currentProgress; // 0 = retracted, 1 = fully assembled
        public bool particleBurstFired;
    }

    private List<BridgePieceData> pieces = new List<BridgePieceData>();
}

```

```

void Awake()
{
    InitializeChildPlatforms();
}

void Start()
{
    FindPlayer();
}

public void InitializeChildPlatforms()
{
    pieces.Clear();

    foreach (Transform child in transform)
    {
        // Skip containers or non-renderable helper objects if any
        SpriteRenderer sr = child.GetComponent<SpriteRenderer>();
        Collider2D col = child.GetComponent<Collider2D>();

        if (sr == null && col == null) continue;

        BridgePieceData piece = new BridgePieceData
        {
            gameObject = child.gameObject,
            transform = child,
            spriteRenderer = sr,
            collider = col,
            originalLocalScale = child.localScale,
            originalLocalPos = child.localPosition,
            originalLocalRot = child.localRotation,
            originalColor = sr != null ? sr.color : Color.white,
            currentProgress = 0f,
            particleBurstFired = false
        };

        // Ensure Rigidbody2D is kinematic for safe platform movement/physics
        Rigidbody2D rb = child.GetComponent<Rigidbody2D>();
        if (rb == null)
        {
            rb = child.gameObject.AddComponent<Rigidbody2D>();
        }
        rb.isKinematic = true;
        rb.simulated = true;

        // Configure One-Way Platform Effector if collider exists
        if (col != null)
        {
            col.usedByEffector = true;
            PlatformEffector2D effector = child.GetComponent<PlatformEffector2D>();
            if (effector == null)
            {
                effector = child.gameObject.AddComponent<PlatformEffector2D>();
            }
            effector.useOneWay = true;
            effector.useSideFriction = false;
            effector.surfaceArc = 160f;
            piece.effector = effector;

            // Initially disable collider until platform scales up
            col.enabled = false;
        }

        // Set initial scale to zero (retracted state)
        child.localScale = Vector3.zero;
        if (sr != null)
        {
            sr.color = new Color(activePinkColor.r * 0.3f, activePinkColor.g * 0.3f, activePinkColor.b * 0.3f, 0f);
        }

        pieces.Add(piece);
    }

    Debug.Log($"[DynamicPinkBridgeCluster] Initialized {pieces.Count} child platforms on '{gameObject.name}'");
}

```

```

    }

    private void FindPlayer()
    {
        if (playerTransform != null) return;

        GameObject player = GameObject.FindGameObjectWithTag("Player");
        if (player == null)
        {
            // Fallback: search by name
            player = GameObject.Find("Player") ?? GameObject.Find("PlayerMage") ?? GameObject.
Find("Mage");
        }

        if (player != null)
        {
            playerTransform = player.transform;
        }
    }

    void Update()
    {
        if (playerTransform == null)
        {
            FindPlayer();
            if (playerTransform == null) return;
        }

        Vector3 playerPos = playerTransform.position;

        foreach (var piece in pieces)
        {
            if (piece.transform == null) continue;

            // Calculate distance to player (prioritize horizontal & 2D distance)
            float distance = Vector2.Distance(piece.transform.position, playerPos);

            float targetProgress = 0f;
            if (distance <= buildDistance)
            {
                targetProgress = 1f;
            }
            else if (distance > retractDistance)
            {
                targetProgress = 0f;
            }
            else
            {
                // Smooth transition band between buildDistance and retractDistance
                targetProgress = Mathf.InverseLerp(retractDistance, buildDistance, distance);
            }

            // Interpolate progress smoothly
            float speed = (targetProgress > piece.currentProgress) ? assembleSpeed : retractSp
eed;
            piece.currentProgress = Mathf.MoveTowards(piece.currentProgress, targetProgress, T
ime.deltaTime * speed);

            // 1. Calculate Spring Bounce Scale Curve
            float scaleMultiplier = EvaluateSpringScale(piece.currentProgress);
            piece.transform.localScale = piece.originalLocalScale * scaleMultiplier;

            // 2. Calculate Neon Pink Color & Alpha Fade
            if (piece.spriteRenderer != null)
            {
                Color darkDimPink = new Color(activePinkColor.r * 0.25f, activePinkColor.g * 0
.1f, activePinkColor.b * 0.25f, 0f);
                Color currentPink = Color.Lerp(darkDimPink, activePinkColor, piece.currentProg
ress);
                piece.spriteRenderer.color = currentPink;
            }

            // 3. Collider Safety Threshold (Enable collider only when progress >= 0.75)
            if (piece.collider != null)
            {
                bool shouldBeSolid = (piece.currentProgress >= 0.75f);
            }
        }
    }

```

```

        if (piece.collider.enabled != shouldBeSolid)
        {
            piece.collider.enabled = shouldBeSolid;
        }
    }

    // 4. Trigger Neon Pink Star Particle Burst on Assembly
    if (piece.currentProgress >= 0.8f && !piece.particleBurstFired)
    {
        piece.particleBurstFired = true;
        if (enableStarParticles)
        {
            SpawnPinkStarBurst(piece.transform.position + Vector3.up * 0.3f);
        }
    }
    else if (piece.currentProgress <= 0.15f)
    {
        piece.particleBurstFired = false;
    }
}

}

/// <summary>
/// Evaluates a spring overshoot curve for smooth scaling bounce.
/// t=0 -> 0; t=0.8 -> scaleBounceFactor (e.g. 1.15); t=1 -> 1.0
/// </summary>
private float EvaluateSpringScale(float t)
{
    if (t <= 0f) return 0f;
    if (t >= 1f) return 1f;

    // Custom spring bounce formula
    float bounce = Mathf.Sin(t * Mathf.PI * 1.15f) * scaleBounceFactor;
    return Mathf.Clamp(bounce, 0f, scaleBounceFactor);
}

private void SpawnPinkStarBurst(Vector3 position)
{
    GameObject burstObj = new GameObject("PinkStarBurst");
    burstObj.transform.position = position;

    ParticleSystem ps = burstObj.AddComponent<ParticleSystem>();
    ParticleSystemRenderer psr = burstObj.GetComponent<ParticleSystemRenderer>();

    var main = ps.main;
    // Removed main.duration assignment to prevent Unity runtime error
    main.loop = false;
    main.startLifetime = new ParticleSystem.MinMaxCurve(0.3f, 0.6f);
    main.startSpeed = new ParticleSystem.MinMaxCurve(2.5f, 5.5f);
    main.startSize = new ParticleSystem.MinMaxCurve(0.15f, 0.32f);
    main.startColor = new ParticleSystem.MinMaxGradient(activePinkColor, Color.white);
    main.gravityModifier = 0.5f;

    var emission = ps.emission;
    emission.SetBursts(new ParticleSystem.Burst[] { new ParticleSystem.Burst(0f, starParticleCount) });

    var shape = ps.shape;
    shape.shapeType = ParticleSystemShapeType.Hemisphere;
    shape.radius = 0.4f;

    var sizeOverLifetime = ps.sizeOverLifetime;
    sizeOverLifetime.enabled = true;
    AnimationCurve curve = new AnimationCurve();
    curve.AddKey(0f, 1f);
    curve.AddKey(1f, 0f);
    sizeOverLifetime.size = new ParticleSystem.MinMaxCurve(1f, curve);

    psr.material = new Material(Shader.Find("Sprites/Default"));

    ps.Play();
    Destroy(burstObj, 1.2f);
}
}

```

FILE 34: Assets/UI/MainMenu/MainMenuUIToolkitController.cs
PURPOSE: Unity UI Toolkit menu, logo, buttons, and orbs.

```
using System.Collections;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.UIElements;
using UnityEngine.SceneManagement;
using SpawnOfChaos.Systems;
#if UNITY_EDITOR
using UnityEditor;
#endif
using Random = UnityEngine.Random;

/// <summary>
/// High-aesthetic Controller for the UI Toolkit Main Menu.
/// Plays full-speed frame animation sequence from the 'keep-54947fd1' folder for the centerpi
/// ece logo.
/// Features gentle fireflies floating upward through the menu atmosphere.
/// </summary>
[RequireComponent(typeof(UIDocument))]
public class MainMenuUIToolkitController : MonoBehaviour
{
    [Header("Scene Configuration")]
    [Tooltip("Target scene to load when Enter Realm is clicked.")]
    public string gameSceneName = "TutorialScene";

    /// <summary>
    /// Global state to track if gameplay is active or title menu is showing.
    /// </summary>
    public static bool isPlaying = false;
    public static bool IsAnyMenuOpen() => !isPlaying;

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
    private static void InitStaticState()
    {
        isPlaying = false;
    }

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    private static void AutoEnsureMainMenuActive()
    {
        if (SceneManager.GetActiveScene().name != "SampleScene") return;
        if (PlayerSpawnPointManager.isRespawning) return;

        var allMenus = Resources.FindObjectsOfTypeAll<MainMenuUIToolkitController>();
        foreach (var m in allMenus)
        {
            if (m.gameObject.scene == SceneManager.GetActiveScene())
            {
                if (!m.gameObject.activeSelf && !isPlaying)
                {
                    Debug.Log("[MainMenuUIToolkitController] Auto-activating MainMenu_UIToolki
t on scene start.");
                    m.gameObject.SetActive(true);
                }
                break;
            }
        }
    }

    [Header("Logo Frame Animation & Sizing")]
    [Tooltip("Dimensions of the logo in pixels (Width, Height).")]
    public Vector2 logoDimensions = new Vector2(1120f, 630f);
    [Tooltip("Overall scale multiplier for the logo animation.")]
    [Range(0.2f, 2.5f)]
    public float logoScale = 1.0f;
    [Tooltip("Folder path containing the animation frames.")]
    public string framesFolderPath = "Assets/Scenes/animations/frames/keep-54947fd1";
    [Tooltip("Frames per second for the logo animation.")]
    [Range(4f, 60f)]
    public float animationFPS = 16f;
    [Tooltip("Whether to continuously loop the logo animation.")]
    public bool loopAnimation = true;
    [Tooltip("Pre-assigned animation frames (populated automatically if empty).")]
```

```

public Texture2D[] logoAnimationFrames;

[Header("Logo Wing Flap & Pause Behavior")]
[Tooltip("Whether to pause the logo animation when the wings reach full extension.")]
public bool pauseWhenWingsFullyExtended = true;

[Tooltip("Frame index where wings are fully extended. Frame 8 (frame_009) is the first full horizontal wing expansion. Frame 46 (frame_047) is the grand wingspan with smoke.")]
public int fullyExtendedFrameIndex = 8;

[Tooltip("Duration in seconds to stay paused on fully extended wings. If <= 0, pauses indefinitely on extended wings.")]
public float fullyExtendedHoldDuration = 0f;

[Tooltip("Duration in seconds to linger/hold when the wings reach their highest apex (frames 16-18, especially frame 17).")]
public float highestWingsHoldDuration = 0.65f;

[Header("Gentle Fireflies (Bottom to Top)")]
[Tooltip("Total number of gentle fireflies.")]
[Range(10, 80)]
public int orbCount = 38;
[Tooltip("Upward drift speed multiplier.")]
[Range(0.2f, 3f)]
public float fallSpeedMultiplier = 1.0f;
[Tooltip("Horizontal firefly sway intensity.")]
[Range(0.2f, 3f)]
public float swayIntensity = 1.0f;

[Header("Audio SFX & Music")]
[Tooltip("Background music track played on the title menu.")]
public AudioClip titleMusic;
public AudioClip hoverClip;
public AudioClip clickClip;
public AudioClip modalOpenClip;

private UIDocument uiDocument;
private VisualElement root;

// Visual Elements for Animation
private VisualElement titleLogo;
private VisualElement titleContainer;
private VisualElement ambientGlow;
private VisualElement orbsContainer;
private Button btnPlay;
private Button btnContinue;
private Button btnNewGame;
private Button btnLoadGame;
private Button btnSettings;
private Button btnQuit;

// Modals & Settings
private VisualElement modalSettings;
private VisualElement modalLoadGame;
private Button btnCloseSettings;
private Button btnCloseLoadGame;
private VisualElement slotsGrid;
private Slider sliderMaster;
private Slider sliderMusic;
private Slider sliderSFX;
private Toggle toggleFullscreen;
private Toggle toggleScreenShake;

private Coroutine frameAnimationCoroutine;
private Coroutine glowPulseCoroutine;

// Spirit Orb Particle Simulation
private class SpiritOrb
{
    public VisualElement element;
    public float baseX; // Fixed horizontal anchor (0..1)
    public float posY; // Vertical position (0..1, where 1 is bottom, 0 is top)
    public float speedY; // Upward drift speed
    public float swayAmp; // Horizontal sway amplitude
    public float swayFreq; // Horizontal sway frequency
    public float swayPhase; // Phase offset

```

```

        public float size;           // Diameter in px
        public float baseAlpha;      // Peak opacity
        public float pulseSpeed;     // Gentle breathing frequency
        public float pulsePhase;     // Breathing offset
        public Color color;          // Glowing purple/lavender color
        public int depthLayer;       // 0 = Foreground 3D sphere, 1 = Midground, 2 = Deep Blur
    }

    private static Texture2D s_blurryBokehTexture;

    private static void EnsureOrbTextures()
    {
        if (s_blurryBokehTexture == null)
        {
            s_blurryBokehTexture = GenerateBlurryBokehTexture();
        }
    }

    private static Texture2D GenerateBlurryBokehTexture()
    {
        int size = 256;
        Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
        tex.wrapMode = TextureWrapMode.Clamp;
        tex.filterMode = FilterMode.Bilinear;

        Vector2 center = new Vector2((size - 1) * 0.5f, (size - 1) * 0.5f);
        float radius = size * 0.49f;

        for (int y = 0; y < size; y++)
        {
            for (int x = 0; x < size; x++)
            {
                float dist = Vector2.Distance(new Vector2(x, y), center) / radius;
                if (dist >= 1.0f)
                {
                    tex.SetPixel(x, y, Color.clear);
                    continue;
                }

                // Pure out-of-focus optical bokeh Gaussian blur with smooth zero-falloff edge
                float edgeFactor = Mathf.Clamp01(1.0f - dist);
                float smoothEdge = edgeFactor * edgeFactor * (3.0f - 2.0f * edgeFactor);
                // Ultra-diffuse wide Gaussian curve - soft, misty, dreamy ethereal glow
                float alpha = Mathf.Exp(-1.9f * dist * dist) * smoothEdge;
                tex.SetPixel(x, y, new Color(1f, 1f, 1f, alpha));
            }
        }

        tex.Apply();
        return tex;
    }

    private readonly List<SpiritOrb> activeOrbs = new List<SpiritOrb>();

    // Warm, restrained lights that read as fireflies against the dark menu.
    // Deep cosmic purple and royal violet atmospheric lights matching Tutorial prologue
    private static readonly Color[] OrbColors = new Color[]
    {
        new Color(0.65f, 0.20f, 0.95f, 0.85f), // Radiant violet (Opening Scene)
        new Color(0.85f, 0.35f, 1.00f, 0.88f), // Neon purple (Opening Scene)
        new Color(0.92f, 0.50f, 0.98f, 0.82f), // Ethereal magenta (Opening Scene)
        new Color(0.48f, 0.15f, 0.85f, 0.86f), // Deep amethyst (Opening Scene)
        new Color(0.78f, 0.45f, 0.90f, 0.84f)  // Soft lilac (Opening Scene)
    };

    private void Awake()
    {
        // One-time auto-migration: adjust to refined goldilocks scale (720x405)
        if (logoScale > 1.4f)
        {
            logoScale = 1.0f;
        }
        if (logoDimensions.x < 620f || logoDimensions.x > 800f)
        {
            logoDimensions = new Vector2(720f, 405f);
        }
    }

```

```

    }

    uiDocument = GetComponent<UIDocument>();
    if (uiDocument != null)
    {
        uiDocument.sortingOrder = 30000f; // Layer Main Menu above all canvases and game elements
    }
    LoadFramesIfNeeded();
    EnsureTitleMusicAssigned();
}

private void Start()
{
    if (uiDocument == null) uiDocument = GetComponent<UIDocument>();
    if (uiDocument != null)
    {
        uiDocument.sortingOrder = 30000f;
    }
    if (!isPlaying)
    {
        Time.timeScale = 0f;
        EnablePlayerGameplay(false);
        if (HUDManager.Instance != null) HUDManager.Instance.UpdateVisibility();
        if (SpawnOfChaos.Minigames.HUDOrbPanel.Instance != null) SpawnOfChaos.Minigames.HUDOrbPanel.Instance.UpdateVisibility();
    }
    PlayTitleMusic();
}

private void EnablePlayerGameplay(bool enable)
{
    if (enable)
    {
        move.ExternalMovementLock = false;
        if (Time.timeScale == 0f) Time.timeScale = 1f;
    }

    GameObject p = GameObject.FindGameObjectWithTag("Player");
    if (p == null) p = GameObject.Find("Player");
    if (p == null) p = GameObject.Find("BasePlayer");

    if (p != null)
    {
        move m = p.GetComponent<move>();
        if (m != null) m.enabled = enable;

        MageCombat mc = p.GetComponent<MageCombat>();
        if (mc != null) mc.enabled = enable;

        Rigidbody2D rb = p.GetComponent<Rigidbody2D>();
        if (rb != null)
        {
            if (!enable)
            {
                rb.linearVelocity = Vector2.zero;
                rb.simulated = false;
            }
            else
            {
                rb.simulated = true;
            }
        }
    }
}

private void OnEnable()
{
    if (isPlaying || PlayerSpawnPointManager.isRespawning)
    {
        isPlaying = true;
        Time.timeScale = 1f;
        EnablePlayerGameplay(true);
        if (uiDocument == null) uiDocument = GetComponent<UIDocument>();
        if (uiDocument != null && uiDocument.rootVisualElement != null)
        {

```

```

        uiDocument.rootVisualElement.style.display = DisplayStyle.None;
    }
    gameObject.SetActive(false);
    return;
}

isPlaying = false;
Time.timeScale = 0f; // Freeze game background when Main Menu is up
EnablePlayerGameplay(false);
if (HUDManager.Instance != null) HUDManager.Instance.UpdateVisibility();
if (SpawnOfChaos.Minigames.HUDOrbPanel.Instance != null) SpawnOfChaos.Minigames.HUDOrb
Panel.Instance.UpdateVisibility();

if (uiDocument == null)
    uiDocument = GetComponent<UIDocument>();

if (uiDocument != null)
{
    uiDocument.sortingOrder = 30000f;
}

// Guarantee no Nyxaris dialogue or canvas covers the Main Menu
var allNyxaris = Object.FindObjectsByType<NyxarisManager>(FindObjectsInactive.Include,
FindObjectsSortMode.None);
foreach (var nm in allNyxaris)
{
    if (nm != null) nm.HideInterface();
}
GameObject mi = GameObject.Find("MainInterface");
if (mi != null) mi.SetActive(false);

if (uiDocument == null) return;

root = uiDocument.rootVisualElement;
if (root == null) return;
root.style.display = DisplayStyle.Flex;

BindVisualElements();
RegisterCallbacks();
LoadSavedSettings();
InitializeSpiritOrbs();
PlayTitleMusic();

// Start logo frame-by-frame animation
if (frameAnimationCoroutine != null) StopCoroutine(frameAnimationCoroutine);
frameAnimationCoroutine = StartCoroutine(PlayLogoFrameAnimation());

// Start ambient background pulse
if (glowPulseCoroutine != null) StopCoroutine(glowPulseCoroutine);
glowPulseCoroutine = StartCoroutine(AnimateGlowPulse());
}

private void OnDisable()
{
    if (frameAnimationCoroutine != null)
    {
        StopCoroutine(frameAnimationCoroutine);
        frameAnimationCoroutine = null;
    }
    if (glowPulseCoroutine != null)
    {
        StopCoroutine(glowPulseCoroutine);
        glowPulseCoroutine = null;
    }
    ClearSpiritOrbs();
    UnregisterCallbacks();
}

private void Update()
{
    // Continuously ensure game background is frozen while in title screen
    if (!isPlaying)
    {
        if (Time.timeScale != 0f) Time.timeScale = 0f;
        EnablePlayerGameplay(false);
    }
}

```

```

        // ESC key closes modal
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            if (modalLoadGame != null && modalLoadGame.ClassListContains("panel--visible"))
            {
                CloseModal(modalLoadGame);
            }
            else if (modalSettings != null && modalSettings.ClassListContains("panel--visible"))
        {
            CloseModal(modalSettings);
        }
    }

    // Update atmospheric living void purple orbs drifting bottom to top
    UpdateSpiritOrbs(Time.unscaledDeltaTime);
}

private void BindVisualElements()
{
    titleLogo = root.Q<VisualElement>("title-logo");
    titleContainer = root.Q<VisualElement>("title-container");
    ambientGlow = root.Q<VisualElement>("ambient-glow");
    orbsContainer = root.Q<VisualElement>("orbs-container");
    if (orbsContainer != null)
    {
        orbsContainer.pickingMode = PickingMode.Ignore;
        orbsContainer.style.display = DisplayStyle.Flex;
    }

    ApplyLogoSize();

    btnPlay = root.Q<Button>("btn-play");
    btnContinue = root.Q<Button>("btn-continue");
    btnNewGame = root.Q<Button>("btn-new-game");
    btnLoadGame = root.Q<Button>("btn-load-game");
    btnSettings = root.Q<Button>("btn-settings");
    btnQuit = root.Q<Button>("btn-quit");

    modalSettings = root.Q<VisualElement>("modal-settings");
    modalLoadGame = root.Q<VisualElement>("modal-load-game");
    btnCloseSettings = root.Q<Button>("btn-close-settings");
    btnCloseLoadGame = root.Q<Button>("btn-close-load-game");
    slotsGrid = root.Q<VisualElement>("slots-grid");

    RefreshContinueButtonState();

    sliderMaster = root.Q<Slider>("slider-master");
    sliderMusic = root.Q<Slider>("slider-music");
    sliderSFX = root.Q<Slider>("slider-sfx");
    toggleFullscreen = root.Q<Toggle>("toggle-fullscreen");
    toggleScreenShake = root.Q<Toggle>("toggle-screenshake");
}

public void ApplyLogoSize()
{
    if (titleLogo != null)
    {
        if (logoScale > 2.0f) logoScale = 1.0f;
        logoDimensions = new Vector2(1120f, 630f);

        // 2X Centerpiece logo scaling with responsive bounds so buttons are never obstructed

        titleLogo.style.width = logoDimensions.x * logoScale;
        titleLogo.style.height = logoDimensions.y * logoScale;
        titleLogo.style.maxWidth = Length.Percent(88f);
        titleLogo.style.maxHeight = Length.Percent(58f);
        titleLogo.style.flexShrink = 1f;
    }
    if (titleContainer != null)
    {
        titleContainer.pickingMode = PickingMode.Ignore;
    }
}

```

```

private void OnValidate()
{
    if (root != null && titleLogo != null)
    {
        ApplyLogoSize();
    }
}

#region Gentle Fireflies Simulation (Bottom to Top)

private void InitializeSpiritOrbs()
{
    ClearSpiritOrbs();
    if (orbsContainer == null) return;

    for (int i = 0; i < orbCount; i++)
    {
        SpiritOrb orb = CreateSpiritOrb(randomY: true);
        activeOrbs.Add(orb);
        orbsContainer.Add(orb.element);
    }
}

private void ClearSpiritOrbs()
{
    if (orbsContainer != null)
    {
        orbsContainer.Clear();
    }
    activeOrbs.Clear();
}

private SpiritOrb CreateSpiritOrb(bool randomY)
{
    EnsureOrbTextures();

    SpiritOrb orb = new SpiritOrb();
    orb.element = new VisualElement();
    orb.element.AddToClassList("menu-firefly");
    orb.element.pickingMode = PickingMode.Ignore;

    // Pure out-of-focus optical bokeh blur for all orbs
    orb.element.style.backgroundImage = new StyleBackground(s_blurryBokehTexture);

    // Multi-Plane Depth Layers with rich blurry bokeh sizing:
    // Layer 2: Deep background blur (soft ambient, smaller)
    // Layer 1: Midground dreamy bokeh
    // Layer 0: Foreground large out-of-focus bokeh orbs
    float layerRoll = Random.value;
    if (layerRoll < 0.38f)
    {
        orb.depthLayer = 2; // Deep Background Ambient Haze
        orb.size = Random.Range(45f, 80f);
        orb.baseAlpha = Random.Range(0.20f, 0.40f);
        orb.speedY = Random.Range(0.040f, 0.068f) * fallSpeedMultiplier;
    }
    else if (layerRoll < 0.78f)
    {
        orb.depthLayer = 1; // Midground Dreamy Bokeh
        orb.size = Random.Range(85f, 140f);
        orb.baseAlpha = Random.Range(0.26f, 0.48f);
        orb.speedY = Random.Range(0.060f, 0.090f) * fallSpeedMultiplier;
    }
    else
    {
        orb.depthLayer = 0; // Large Foreground Out-of-Focus Bokeh Blooms
        orb.size = Random.Range(145f, 220f);
        orb.baseAlpha = Random.Range(0.30f, 0.52f);
        orb.speedY = Random.Range(0.075f, 0.110f) * fallSpeedMultiplier;
    }

    orb.baseX = Random.Range(0.03f, 0.97f);
    orb.posY = randomY ? Random.Range(0.02f, 0.98f) : (1.08f + Random.Range(0f, 0.08f));

    // Subtle peaceful horizontal wafting
    orb.swayAmp = Random.Range(0.012f, 0.026f) * swayIntensity;
}

```

```

        orb.swayFreq = Random.Range(0.4f, 1.1f);
        orb.swayPhase = Random.Range(0f, Mathf.PI * 2f);

        orb.pulseSpeed = Random.Range(1.1f, 2.0f);
        orb.pulsePhase = Random.Range(0f, Mathf.PI * 2f);
        orb.color = OrbColors[Random.Range(0, OrbColors.Length)];

        // Visual styling: Transparent background with 3D tinted texture
        orb.element.style.width = orb.size;
        orb.element.style.height = orb.size;
        orb.element.style.backgroundColor = Color.clear;
        orb.element.style.unityBackgroundImageTintColor = orb.color;

        float currentX = Mathf.Clamp(orb.baseX, 0.01f, 0.99f);
        orb.element.style.left = Length.Percent(currentX * 100f);
        orb.element.style.top = Length.Percent(orb.posY * 100f);
        return orb;
    }

private void UpdateSpiritOrbs(float dt)
{
    if (activeOrbs.Count == 0 || orbsContainer == null) return;

    float time = Time.unscaledTime;

    for (int i = 0; i < activeOrbs.Count; i++)
    {
        SpiritOrb orb = activeOrbs[i];

        // Gentle steady upward movement (bottom to top)
        orb.posY -= orb.speedY * dt;

        // Gentle organic horizontal sway relative to fixed baseX (never accumulates!)
        float swayOffset = Mathf.Sin(time * orb.swayFreq + orb.swayPhase) * orb.swayAmp;
        float currentX = Mathf.Clamp(orb.baseX + swayOffset, 0.01f, 0.99f);

        // Vertical soft fade in at bottom and fade out at top
        float verticalFade = 1.0f;
        if (orb.posY > 0.88f)
        {
            verticalFade = Mathf.InverseLerp(1.08f, 0.88f, orb.posY);
        }
        else if (orb.posY < 0.12f)
        {
            verticalFade = Mathf.InverseLerp(-0.08f, 0.12f, orb.posY);
        }

        // Gentle breathing alpha pulse
        float pulse = 0.72f + 0.28f * Mathf.Sin(time * orb.pulseSpeed + orb.pulsePhase);
        float alpha = orb.baseAlpha * verticalFade * pulse;
        orb.element.style.opacity = Mathf.Clamp01(alpha);

        // Set coordinates directly
        orb.element.style.left = Length.Percent(currentX * 100f);
        orb.element.style.top = Length.Percent(orb.posY * 100f);

        // Wrap orb back to below screen once it rises past top
        if (orb.posY < -0.10f)
        {
            orb.posY = 1.08f + Random.Range(0f, 0.06f);
            orb.baseX = Random.Range(0.03f, 0.97f);
            orb.color = OrbColors[Random.Range(0, OrbColors.Length)];
            orb.element.style.unityBackgroundImageTintColor = orb.color;
        }
    }
}

#endregion

private void LoadFramesIfNeeded()
{
    if (logoAnimationFrames != null && logoAnimationFrames.Length > 0) return;

#if UNITY_EDITOR
    if (!string.IsNullOrEmpty(framesFolderPath) && Directory.Exists(framesFolderPath))
    {

```

```

        string[] filePaths = Directory.GetFiles(framesFolderPath, "frame_*.png");
        System.Array.Sort(filePaths);

        List<Texture2D> loadedList = new List<Texture2D>();
        foreach (string path in filePaths)
        {
            Texture2D tex = AssetDatabase.LoadAssetAtPath<Texture2D>(path);
            if (tex != null)
            {
                loadedList.Add(tex);
            }
        }
        logoAnimationFrames = loadedList.ToArray();
        Debug.Log($"[MainMenuUIToolkitController] Loaded {logoAnimationFrames.Length} logo
animation frames from {framesFolderPath}");
    }
#endif
}

/// <summary>
/// Frame-by-frame logo animation loop for UI Toolkit.
/// </summary>
private void OnLogoTapped(ClickEvent evt)
{
    TriggerWingFlap();
}

/// <summary>
/// Flaps the wings and pauses when fully extended. Can be triggered on tap or when the ga
me initially starts.
/// </summary>
public void TriggerWingFlap()
{
    if (frameAnimationCoroutine != null)
    {
        StopCoroutine(frameAnimationCoroutine);
    }
    frameAnimationCoroutine = StartCoroutine(PlayFlapSequenceRoutine());
}

/// <summary>
/// Flaps the wings for one majestic cycle from the extended pose, lingers at the highest
point, and pauses back at fully extended.
/// </summary>
private IEnumerator PlayFlapSequenceRoutine()
{
    if (logoAnimationFrames == null || logoAnimationFrames.Length == 0)
    {
        LoadFramesIfNeeded();
    }

    if (logoAnimationFrames == null || logoAnimationFrames.Length == 0 || titleLogo == nul
l)
    {
        yield break;
    }

    float frameDuration = 1f / Mathf.Max(1f, animationFPS);
    int startFrame = fullyExtendedFrameIndex;
    int maxFrames = logoAnimationFrames.Length;

    // Play full wing flap stroke (rising to apex, lingering at highest wings, descending
back to extended)
    for (int i = 1; i <= 16; i++)
    {
        int currentFrame = (startFrame + i) % maxFrames;
        if (currentFrame < logoAnimationFrames.Length && logoAnimationFrames[currentFrame]
!= null)
        {
            titleLogo.style.backgroundImage = new StyleBackground(logoAnimationFrames[curr
entFrame]);
        }

        // When wings are at their highest (apex frames 16-18, especially frame 17):
        if (currentFrame == 17)
        {

```

```

        yield return new WaitForSecondsRealtime(highestWingsHoldDuration);
    }
    else if (currentFrame == 16 || currentFrame == 18)
    {
        yield return new WaitForSecondsRealtime(frameDuration * 1.8f);
    }
    else
    {
        yield return new WaitForSecondsRealtime(frameDuration);
    }
}

// Return to and pause on the fully extended wings pose (only that logo animation pauses)
if (fullyExtendedFrameIndex < logoAnimationFrames.Length && logoAnimationFrames[fullyExtendedFrameIndex] != null)
{
    titleLogo.style.backgroundImage = new StyleBackground(logoAnimationFrames[fullyExtendedFrameIndex]);
}

private IEnumerator PlayLogoFrameAnimation()
{
    if (logoAnimationFrames == null || logoAnimationFrames.Length == 0)
    {
        LoadFramesIfNeeded();
    }

    if (logoAnimationFrames == null || logoAnimationFrames.Length == 0 || titleLogo == null)
    {
        yield break;
    }

    // Initial launch flap: play majestic stroke from extended pose up to highest wings (linger), then return to extended
    yield return StartCoroutine(PlayFlapSequenceRoutine());

    // If continuous loop is requested without pause, continue cycling
    if (!pauseWhenWingsFullyExtended && loopAnimation)
    {
        int frameIndex = fullyExtendedFrameIndex;
        float frameDuration = 1f / Mathf.Max(1f, animationFPS);

        while (true)
        {
            if (titleLogo != null && frameIndex < logoAnimationFrames.Length && logoAnimationFrames[frameIndex] != null)
            {
                titleLogo.style.backgroundImage = new StyleBackground(logoAnimationFrames[frameIndex]);
            }

            if (frameIndex == 17 || frameIndex == 2 || frameIndex == 39 || frameIndex == 69 || frameIndex == 86)
            {
                yield return new WaitForSecondsRealtime(highestWingsHoldDuration);
            }
            else if (frameIndex == 16 || frameIndex == 18)
            {
                yield return new WaitForSecondsRealtime(frameDuration * 1.8f);
            }
            else
            {
                yield return new WaitForSecondsRealtime(frameDuration);
            }

            frameIndex = (frameIndex + 1) % logoAnimationFrames.Length;
        }
    }
}

/// <summary>
/// Gentle pulse for ambient background glow.
/// </summary>

```

```

private IEnumerator AnimateGlowPulse()
{
    float time = 0f;
    while (true)
    {
        time += Time.unscaledDeltaTime;
        if (ambientGlow != null)
        {
            float glowAlpha = 0.7f + Mathf.Sin(time * 2.0f) * 0.25f;
            ambientGlow.style.opacity = glowAlpha;
        }
        yield return null;
    }
}

private void RegisterCallbacks()
{
    if (titleLogo != null)
    {
        titleLogo.RegisterCallback<ClickEvent>(OnLogoTapped);
    }
    // titleContainer ignores picking to avoid intercepting button clicks
    if (btnPlay != null) btnPlay.clicked += OnPlayClicked;
    if (btnContinue != null) btnContinue.clicked += OnContinueClicked;
    if (btnNewGame != null) btnNewGame.clicked += OnNewGameClicked;
    if (btnLoadGame != null) btnLoadGame.clicked += OnLoadGameMenuClicked;
    if (btnSettings != null) btnSettings.clicked += () => OpenModal(modalSettings);
    if (btnQuit != null) btnQuit.clicked += OnQuitClicked;

    if (btnCloseSettings != null) btnCloseSettings.clicked += () => CloseModal(modalSettings);
    if (btnCloseLoadGame != null) btnCloseLoadGame.clicked += () => CloseModal(modalLoadGame);

    root.Query<Button>().ForEach(button =>
    {
        button.RegisterCallback<MouseEnterEvent>(OnButtonHover);
        button.RegisterCallback<ClickEvent>(OnButtonClickSFX);
    });

    if (sliderMaster != null)
    {
        sliderMaster.RegisterValueChangedCallback(evt =>
        {
            AudioListener.volume = evt.newValue;
            if (AudioManager.Instance != null) AudioManager.Instance.masterVolume = evt.newValue;
            PlayerPrefs.SetFloat("MasterVolume", evt.newValue);
        });
    }

    if (sliderMusic != null)
    {
        sliderMusic.RegisterValueChangedCallback(evt =>
        {
            if (AudioManager.Instance != null) AudioManager.Instance.musicVolume = evt.newValue;
            PlayerPrefs.SetFloat("MusicVolume", evt.newValue);
        });
    }

    if (sliderSFX != null)
    {
        sliderSFX.RegisterValueChangedCallback(evt =>
        {
            if (AudioManager.Instance != null) AudioManager.Instance.sfxVolume = evt.newValue;
            PlayerPrefs.SetFloat("SFXVolume", evt.newValue);
        });
    }

    if (toggleFullscreen != null)
    {
        toggleFullscreen.RegisterValueChangedCallback(evt =>
        {
            Screen.fullScreen = evt.newValue;
        });
    }
}

```

```

        PlayerPrefs.SetInt("Fullscreen", evt.newValue ? 1 : 0);
    });
}

if (toggleScreenShake != null)
{
    toggleScreenShake.RegisterValueChangedCallback(evt =>
    {
        PlayerPrefs.SetInt("ScreenShake", evt.newValue ? 1 : 0);
    });
}
}

private void UnregisterCallbacks()
{
    if (titleLabel != null) titleLabel.UnregisterCallback<ClickEvent>(OnLogoTapped);

    if (btnPlay != null) btnPlay.clicked -= OnPlayClicked;
    if (btnContinue != null) btnContinue.clicked -= OnContinueClicked;
    if (btnNewGame != null) btnNewGame.clicked -= OnNewGameClicked;
    if (btnLoadGame != null) btnLoadGame.clicked -= OnLoadGameMenuClicked;
    if (btnCloseSettings != null) btnCloseSettings.clicked -= () => CloseModal(modalSettings);
    if (btnCloseLoadGame != null) btnCloseLoadGame.clicked -= () => CloseModal(modalLoadGame);
    if (btnQuit != null) btnQuit.clicked -= OnQuitClicked;
}

private void LoadSavedSettings()
{
    float master = PlayerPrefs.GetFloat("MasterVolume", 1.0f);
    float music = PlayerPrefs.GetFloat("MusicVolume", 0.8f);
    float sfx = PlayerPrefs.GetFloat("SFXVolume", 0.9f);
    bool fullscreen = PlayerPrefs.GetInt("Fullscreen", Screen.fullScreen ? 1 : 0) == 1;
    bool screenShake = PlayerPrefs.GetInt("ScreenShake", 1) == 1;

    if (sliderMaster != null) sliderMaster.value = master;
    if (sliderMusic != null) sliderMusic.value = music;
    if (sliderSFX != null) sliderSFX.value = sfx;
    if (toggleFullscreen != null) toggleFullscreen.value = fullscreen;
    if (toggleScreenShake != null) toggleScreenShake.value = screenShake;

    AudioManager.volume = master;
    if (AudioManager.Instance != null)
    {
        AudioManager.Instance.masterVolume = master;
        AudioManager.Instance.musicVolume = music;
        AudioManager.Instance.sfxVolume = sfx;
    }
}

private void OpenModal(VisualElement modal)
{
    if (modal == null) return;
    Debug.Log($"[MainMenuUIToolkitController] OpenModal invoked for {modal.name}.");
    PlaySFX(modalOpenClip);
    modal.RemoveFromClassList("panel--hidden");
    modal.AddToClassList("panel--visible");
}

private void CloseModal(VisualElement modal)
{
    if (modal == null) return;
    PlaySFX(clickClip);
    modal.RemoveFromClassList("panel--visible");
    modal.AddToClassList("panel--hidden");
}

private void OnPlayClicked()
{
    PlaySFX(clickClip);
    isPlaying = true;
    Time.timeScale = 1f; // Unpause game on play
    EnablePlayerGameplay(true);

    if (HUDManager.Instance != null)

```

```

        {
            HUDManager.Instance.UpdateVisibility();
        }
        if (SpawnOfChaos.Minigames.HUDOrbPanel.Instance != null)
        {
            SpawnOfChaos.Minigames.HUDOrbPanel.Instance.UpdateVisibility();
        }

        // Trigger Level Gameplay Background Music
        LevelMusicPlayer lmp = FindFirstObjectByType<LevelMusicPlayer>();
        if (lmp != null)
        {
            lmp.StartLevelMusic();
        }

        string targetScene = string.IsNullOrEmpty(gameSceneName) ? "TutorialScene" : gameScene
Name;
        string activeScene = SceneManager.GetActiveScene().name;
        if (activeScene.Equals(targetScene, System.StringComparison.OrdinalIgnoreCase))
        {
            if (root != null)
            {
                root.style.display = DisplayStyle.None;
            }
            gameObject.SetActive(false);
            return;
        }

        ExecuteGameLaunch(targetScene);
    }

    private void RefreshContinueButtonState()
    {
        if (btnContinue == null) return;
        bool hasSave = SaveSlotManager.HasAnySave();
        // Keep button interactive; adjust opacity to clearly show save state
        btnContinue.SetEnabled(true);
        btnContinue.style.opacity = hasSave ? 1.0f : 0.65f;
    }

    private void OnContinueClicked()
    {
        Debug.Log("[MainMenuUIToolkitController] OnContinueClicked invoked.");
        PlaySFX(clickClip);
        int recentIdx = SaveSlotManager.GetMostRecentSlotIndex();
        SaveSlotData slot = SaveSlotManager.GetSlot(recentIdx);
        if (slot != null && !slot.isEmpty)
        {
            Debug.Log($"[MainMenuUIToolkitController] Resuming save slot {recentIdx}...");
            LoadGameSlot(slot);
        }
        else
        {
            Debug.Log("[MainMenuUIToolkitController] No prior saves found; falling back to New
Game...");
            OnNewGameClicked();
        }
    }

    private void OnNewGameClicked()
    {
        Debug.Log("[MainMenuUIToolkitController] OnNewGameClicked invoked.");
        PlaySFX(clickClip);
        int emptySlot = SaveSlotManager.FindFirstEmptySlotIndex();
        StartNewGameInSlot(emptySlot);
    }

    private void OnLoadGameMenuClicked()
    {
        Debug.Log("[MainMenuUIToolkitController] OnLoadGameMenuClicked invoked.");
        PlaySFX(modalOpenClip);
        PopulateSlotsGrid();
        OpenModal(modalLoadGame);
    }

```

```

public void PopulateSlotsGrid()
{
    if (slotsGrid == null) return;
    slotsGrid.Clear();

    List<SaveSlotData> slots = SaveSlotManager.GetAllSlots();
    for (int i = 0; i < slots.Count; i++)
    {
        SaveSlotData data = slots[i];
        int slotNumber = i + 1;

        VisualElement card = new VisualElement();
        card.AddToClassList("slot-card");

        if (data.isEmpty)
        {
            card.AddToClassList("slot-card--empty");

            VisualElement header = new VisualElement();
            header.AddToClassList("slot-card-header");
            Label badge = new Label($"SLOT {slotNumber:D2}");
            badge.AddToClassList("slot-badge");
            Label emptyLabel = new Label("UNCLAIMED VESSEL");
            emptyLabel.AddToClassList("slot-date");
            header.Add(badge);
            header.Add(emptyLabel);
            card.Add(header);

            Label loc = new Label("Empty Memory Vessel");
            loc.AddToClassList("slot-location");
            card.Add(loc);

            VisualElement details = new VisualElement();
            details.AddToClassList("slot-details-row");
            Label detailText = new Label("No drifter soul anchored to this timeline.");
            details.Add(detailText);
            card.Add(details);

            VisualElement actions = new VisualElement();
            actions.AddToClassList("slot-actions-row");
            Button btnNew = new Button(() => {
                CloseModal(modalLoadGame);
                StartNewGameInSlot(slotNumber);
            }) { text = "+ NEW RUN" };
            btnNew.AddToClassList("slot-btn");
            btnNew.AddToClassList("slot-btn--load");
            actions.Add(btnNew);
            card.Add(actions);
        }
        else
        {
            VisualElement header = new VisualElement();
            header.AddToClassList("slot-card-header");
            Label badge = new Label($"SLOT {slotNumber:D2}");
            badge.AddToClassList("slot-badge");
            Label date = new Label(string.IsNullOrEmpty(data.saveTimestamp) ? "Recorded So
ul" : data.saveTimestamp);
            date.AddToClassList("slot-date");
            header.Add(badge);
            header.Add(date);
            card.Add(header);

            Label loc = new Label(data.locationName);
            loc.AddToClassList("slot-location");
            card.Add(loc);

            VisualElement details = new VisualElement();
            details.AddToClassList("slot-details-row");
            Label stats = new Label($"LVL {data.playerLevel} | {data.currentWeapon} |
{data.coins} ??? | {data.GetFormattedPlaytime()}");
            details.Add(stats);
            card.Add(details);

            VisualElement actions = new VisualElement();
            actions.AddToClassList("slot-actions-row");

```

```

        Button btnDel = new Button(() => {
            DeleteGameSlot(slotNumber);
        }) { text = "DELETE" };
        btnDel.AddToClassList("slot-btn");
        btnDel.AddToClassList("slot-btn--del");
        actions.Add(btnDel);

        Button btnLoad = new Button(() => {
            CloseModal(modalLoadGame);
            LoadGameSlot(data);
        }) { text = "RESUME" };
        btnLoad.AddToClassList("slot-btn");
        btnLoad.AddToClassList("slot-btn--load");
        actions.Add(btnLoad);

        card.Add(actions);
    }

    slotsGrid.Add(card);
}

private void StartNewGameInSlot(int slotIndex)
{
    SaveSlotManager.ActiveSlotIndex = slotIndex;
    SaveSlotData startingSlot = new SaveSlotData
    {
        slotIndex = slotIndex,
        isEmpty = false,
        locationName = "Primordial Void & Tutorial",
        sceneName = "TutorialScene",
        playerLevel = 1,
        coins = 0,
        currentWeapon = "DarkSpear",
        playTimeSeconds = 0f
    };
    SaveSlotManager.SaveSlot(slotIndex, startingSlot);

    ExecuteGameLaunch(startingSlot.sceneName);
}

private void LoadGameSlot(SaveSlotData data)
{
    SaveSlotManager.ActiveSlotIndex = data.slotIndex;
    SaveSlotManager.LastPlayedSlotIndex = data.slotIndex;
    if (data.locationName.Contains("Cherry Blossom") || data.sceneName == "SampleScene")
    {
        PlayerPrefs.SetInt(SampleSceneToriiIntroSequence.PREF_INTRO_COMPLETED, 0);
        PlayerPrefs.Save();
    }
    ExecuteGameLaunch(data.sceneName);
}

private void DeleteGameSlot(int slotIndex)
{
    PlaySFX(clickClip);
    SaveSlotManager.DeleteSlot(slotIndex);
    PopulateSlotsGrid();
    RefreshContinueButtonState();
}

private void ExecuteGameLaunch(string sceneToLoad)
{
    isPlaying = true;
    Time.timeScale = 1f;
    EnablePlayerGameplay(true);

    string currentScene = SceneManager.GetActiveScene().name;
    if (currentScene.Equals(sceneToLoad, System.StringComparison.OrdinalIgnoreCase))
    {
        Debug.Log($"[MainMenu] Already in target scene '{sceneToLoad}'. Activating gamepla
y immediately.");
        // Hide Main Menu UI completely first
        if (root != null) root.style.display = DisplayStyle.None;
        gameObject.SetActive(false);
    }
}

```

```

        GameObject existingPlayer = GameObject.FindGameObjectWithTag("Player") ?? GameObject.Find("Player");
        if (existingPlayer != null)
        {
            existingPlayer.transform.position = SampleSceneToriiIntroSequence.GetToriiGateSpawnPosition();
        }

        if (HUDManager.Instance != null) HUDManager.Instance.UpdateVisibility();
        if (SpawnOfChaos.Minigames.HUDOrbPanel.Instance != null) SpawnOfChaos.Minigames.HUDOrbPanel.Instance.UpdateVisibility();

        LevelMusicPlayer lmp2 = FindFirstObjectByType<LevelMusicPlayer>();
        if (lmp2 != null) lmp2.StartLevelMusic();

        // Trigger Torii intro sequence in gameplay if entering SampleScene
        if (sceneToLoad == "SampleScene")
        {
            bool introDone = PlayerPrefs.GetInt(SampleSceneToriiIntroSequence.PREF_INTRO_COMPLETED, 0) == 1;
            if (!introDone)
            {
                SampleSceneToriiIntroSequence.EnsureExists();
                if (SampleSceneToriiIntroSequence.Instance != null)
                {
                    SampleSceneToriiIntroSequence.Instance.ForcePlayIntro();
                }
            }
        }

        return;
    }

    if (sceneToLoad == "TutorialScene")
    {
        GameObject existingPlayer = GameObject.FindGameObjectWithTag("Player") ?? GameObject.Find("Player");
        if (existingPlayer != null && !existingPlayer.name.Contains("BasePlayer"))
        {
            Debug.Log("[MainMenu] Clearing existing non-BasePlayer before launching TutorialScene.");
            Destroy(existingPlayer);
        }
        PlayerSpawnPointManager.targetSpawnPointName = "DefaultSpawnPoint";
    }

    if (HUDManager.Instance != null) HUDManager.Instance.UpdateVisibility();
    if (SpawnOfChaos.Minigames.HUDOrbPanel.Instance != null) SpawnOfChaos.Minigames.HUDOrbPanel.Instance.UpdateVisibility();

    LevelMusicPlayer lmp = FindFirstObjectByType<LevelMusicPlayer>();
    if (lmp != null) lmp.StartLevelMusic();

    if (root != null) root.style.display = DisplayStyle.None;
    gameObject.SetActive(false);

    ArcaneLoadingScreen.LoadScene(sceneToLoad);
}

private void PlayTitleMusic()
{
    EnsureTitleMusicAssigned();
    if (titleMusic != null && AudioManager.Instance != null)
    {
        AudioManager.Instance.PlayBGM(titleMusic, true);
    }
}

private void EnsureTitleMusicAssigned()
{
    if (titleMusic == null)
    {
        titleMusic = Resources.Load<AudioClip>("Audio/the-spawn-of-chaos");
    }
}
#endif UNITY_EDITOR

```

```

        titleMusic = UnityEditor.AssetDatabase.LoadAssetAtPath<AudioClip>("Assets/Audio/Spawn-of-chaos.mp3");
    }
#endif
    }

    private void OnQuitClicked()
    {
        Debug.Log("[MainMenuUIToolkitController] OnQuitClicked invoked.");
        PlaySFX(clickClip);
#if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
#else
        Application.Quit();
#endif
    }

    private void OnButtonHover(MouseEnterEvent evt)
    {
        PlaySFX(hoverClip);
    }

    private void OnButtonClickSFX(ClickEvent evt)
    {
        PlaySFX(clickClip);
    }

    private void PlaySFX(AudioClip clip)
    {
        if (clip != null && AudioManager.Instance != null)
        {
            AudioManager.Instance.PlaySFX(clip);
        }
    }
}

```

FILE 35: Assets/Scenes/scripts/Nyxaris Chat Panel/NyxarisFrameAnimator.cs
PURPOSE: Animates Nyxaris portrait frames.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
#if UNITY_EDITOR
using UnityEditor;
#endif

/// <summary>
/// Dedicated frame-by-frame animation sequencer for Nyxaris companion portrait.
/// Plays dynamic sprite sequences from Assets/Scenes/animations/frames/Nyxaris/
/// and freezes on the final frame when completing dialogue/response poses.
/// </summary>
public class NyxarisFrameAnimator : MonoBehaviour
{
    [Header("Target Display Image")]
    public Image targetImage;

    [Header("Animation Settings")]
    public float defaultFPS = 24f;
    public string defaultIdleAnimation = "nyxarisnuetral-411247bb";

    [System.Serializable]
    public class NamedFrameSequence
    {
        public string animationKey;
        public List<Sprite> frames = new List<Sprite>();
    }

    [Header("Pre-loaded Frame Sequences")]
    public List<NamedFrameSequence> sequences = new List<NamedFrameSequence>();

    private Dictionary<string, List<Sprite>> sequenceDict = new Dictionary<string, List<Sprite>>
    >>(System.StringComparer.OrdinalIgnoreCase);
    private Coroutine playRoutine;

```

```

public bool IsPlaying { get; private set; }
public bool IsFrozenOnLastFrame { get; private set; }
public string CurrentAnimationKey { get; private set; }

void Awake()
{
    if (targetImage == null)
    {
        targetImage = GetComponent<Image>();
    }
    BuildSequenceDictionary();
}

void Start()
{
    if (sequences.Count == 0)
    {
        AutoScanAndIndexAllNyxarisFrames();
    }
    BuildSequenceDictionary();
}

public void BuildSequenceDictionary()
{
    sequenceDict.Clear();
    foreach (var seq in sequences)
    {
        if (!string.IsNullOrEmpty(seq.animationKey) && seq.frames != null && seq.frames.Count > 0)
        {
            sequenceDict[seq.animationKey] = seq.frames;

            // Also index simplified alias (e.g. "explaining" -> "nyxarisexplaining-30d282
            string clean = CleanKey(seq.animationKey);
            if (!sequenceDict.ContainsKey(clean))
            {
                sequenceDict[clean] = seq.frames;
            }
        }
    }
}

private string CleanKey(string raw)
{
    if (string.IsNullOrEmpty(raw)) return "";
    string s = raw.ToLower();
    if (s.StartsWith("nyxaris")) s = s.Substring(7);
    int dashIdx = s.IndexOf('-');
    if (dashIdx > 0) s = s.Substring(0, dashIdx);
    return s.Replace("_", "").Replace(" ", "").Replace("mp4", "").Trim();
}

private string pendingAnimation = null;
private float pendingFps = 20f;

void OnEnable()
{
    if (!string.IsNullOrEmpty(pendingAnimation))
    {
        string anim = pendingAnimation;
        float fps = pendingFps;
        pendingAnimation = null;
        PlayAnimation(anim, fps);
    }
}

/// <summary>
/// Plays the matching frame sequence and holds/freezes on the last frame indefinitely.
/// </summary>
public void PlayAnimation(string animKey, float fps = 20f)
{
    if (string.IsNullOrEmpty(animKey)) return;

    if (!gameObject.activeInHierarchy)

```

```

    {
        pendingAnimation = animKey;
        pendingFps = fps;
        return;
    }

    List<Sprite> frames = FindFrames(animKey);
    if (frames == null || frames.Count == 0)
    {
        frames = FindFrames("neutral") ?? FindFrames("nueutral");
    }

    if (frames == null || frames.Count == 0) return;

    if (targetImage != null && frames[0] != null)
    {
        targetImage.sprite = frames[0];
        targetImage.enabled = true;
        targetImage.color = Color.white;
    }

    if (playRoutine != null)
    {
        StopCoroutine(playRoutine);
    }

    CurrentAnimationKey = animKey;
    playRoutine = StartCoroutine(PlayFrameSequenceRoutine(frames, fps > 0 ? fps : defaultFPS));
}

private IEnumerator PlayFrameSequenceRoutine(List<Sprite> frames, float fps)
{
    IsPlaying = true;
    IsFrozenOnLastFrame = false;
    float frameDelay = 1f / Mathf.Max(1f, fps);

    for (int i = 0; i < frames.Count; i++)
    {
        if (targetImage != null && frames[i] != null)
        {
            targetImage.sprite = frames[i];
            targetImage.enabled = true;
        }
        yield return new WaitForSecondsRealtime(frameDelay);
    }

    // Freeze on final frame
    if (frames.Count > 0 && targetImage != null)
    {
        targetImage.sprite = frames[frames.Count - 1];
    }

    IsPlaying = false;
    IsFrozenOnLastFrame = true;
}

/// <summary>
/// Finds frames using direct or fuzzy alias matching across all 34 folders.
/// </summary>
public List<Sprite> FindFrames(string query)
{
    if (string.IsNullOrEmpty(query)) return null;
    if (sequenceDict.Count == 0 || sequences.Count == 0)
    {
        AutoScanAndIndexAllNyxarisFrames();
        BuildSequenceDictionary();
    }

    string q = query.ToLower().Trim();

    // High priority semantic alias matching
    if (q.Contains("stern") || q.Contains("warn") || q.Contains("grave") || q.Contains("serious"))
    {
        if (sequenceDict.TryGetValue("nyxarissternorimportantwarning-534901f6", out var wL

```

```

ist)) return wList;
    if (sequenceDict.TryGetValue("sternorimportantwarning", out var wList2)) return wList2;
    if (sequenceDict.TryGetValue("nyxarisworriedupsetthinkingmp4-bd2148fc", out var wList3)) return wList3;
    }
    else if (q.Contains("angr") || q.Contains("rage") || q.Contains("piss") || q.Contains("fury"))
    {
        if (sequenceDict.TryGetValue("nyxarisangry-e56db4b1", out var aList)) return aList;
        ;
        if (sequenceDict.TryGetValue("nyxarismildanger-3d4eb3ea", out var aList2)) return aList2;
        if (sequenceDict.TryGetValue("nyxarispissed-7e387d67", out var aList3)) return aList3;
        if (sequenceDict.TryGetValue("angry", out var aList4)) return aList4;
    }
    else if (q.Contains("spread") || q.Contains("armsspread") || q.Contains("gift"))
    {
        if (sequenceDict.TryGetValue("nyxarisexcitedarmsspreadexplaining-cb776e55", out var sList)) return sList;
        if (sequenceDict.TryGetValue("excitedarmsspreadexplaining", out var sList2)) return sList2;
        if (sequenceDict.TryGetValue("nyxarisexplaining0-c6c6f20e", out var sList3)) return sList3;
    }
    else if (q.Contains("excit") || q.Contains("power") || q.Contains("wrath"))
    {
        if (sequenceDict.TryGetValue("nyxarisexcited-f2ab5508", out var exList)) return exList;
        if (sequenceDict.TryGetValue("nyxarisconfidently0-146ef255", out var exList2)) return exList2;
        if (sequenceDict.TryGetValue("nyxarishappytosay-e0fd84be", out var exList3)) return exList3;
        if (sequenceDict.TryGetValue("excited", out var exList4)) return exList4;
    }
    else if (q.Contains("confiden") || q.Contains("smug") || q.Contains("proud"))
    {
        if (sequenceDict.TryGetValue("nyxarisconfidently-64b9a921", out var cList)) return cList;
        if (sequenceDict.TryGetValue("nyxarisconfidently0-146ef255", out var cList2)) return cList2;
        if (sequenceDict.TryGetValue("confidently", out var cList3)) return cList3;
    }
    else if (q.Contains("happ") || q.Contains("smile") || q.Contains("glad"))
    {
        if (sequenceDict.TryGetValue("nyxarishappy-f4e5a565", out var hList)) return hList;
        ;
        if (sequenceDict.TryGetValue("nyxarishappytosay-e0fd84be", out var hList2)) return hList2;
        if (sequenceDict.TryGetValue("happy", out var hList3)) return hList3;
    }
    else if (q.Contains("explain") || q.Contains("talk"))
    {
        if (sequenceDict.TryGetValue("nyxarisexplaining0-c6c6f20e", out var exp0)) return exp0;
        if (sequenceDict.TryGetValue("nyxarisexplaining1-a4a19986", out var exp1)) return exp1;
        if (sequenceDict.TryGetValue("nyxarisexcitedarmsspreadexplaining-cb776e55", out var expSpread)) return expSpread;
        if (sequenceDict.TryGetValue("nyxaristalkingeyesclosed-a238b88a", out var talkClosed)) return talkClosed;
        if (sequenceDict.TryGetValue("explaining", out var expAlias)) return expAlias;
    }
    else if (q.Contains("think"))
    {
        if (sequenceDict.TryGetValue("nyxaristhinking-616901a8", out var tList)) return tList;
        if (sequenceDict.TryGetValue("nyxariscutelythinking-08686484", out var tList2)) return tList2;
        if (sequenceDict.TryGetValue("nyxarishappythinking-bd1f19af", out var tList3)) return tList3;
        if (sequenceDict.TryGetValue("nyxarisworriedupsetthinkingmp4-bd2148fc", out var tList4)) return tList4;
        if (sequenceDict.TryGetValue("thinking", out var tList5)) return tList5;
    }
}

```

```

        else if (q.Contains("neutr") || q.Contains("nueutr"))
        {
            if (sequenceDict.TryGetValue("nyxarisnueutral-411247bb", out var nList)) return nList;
            if (sequenceDict.TryGetValue("nyxarisnueutralstare-14b61402", out var nList2)) return nList2;
            if (sequenceDict.TryGetValue("nueutral", out var nList3)) return nList3;
        }

        // 1. Direct match
        if (sequenceDict.TryGetValue(q, out var directList) && directList.Count > 0)
            return directList;

        // 2. Clean alias match
        string cleanQuery = CleanKey(q);
        if (cleanQuery == "neutral" || cleanQuery == "nueutral")
        {
            if (sequenceDict.TryGetValue("nueutral", out var nList)) return nList;
            if (sequenceDict.TryGetValue("neutral", out var nList2)) return nList2;
        }
        else if (cleanQuery == "explaining" || cleanQuery.Contains("explain"))
        {
            if (sequenceDict.TryGetValue("explaining0", out var exp0)) return exp0;
            if (sequenceDict.TryGetValue("explaining1", out var exp1)) return exp1;
            if (sequenceDict.TryGetValue("excitedarmsspreadexplaining", out var expSpread)) return expSpread;
        }

        if (sequenceDict.TryGetValue(cleanQuery, out var cleanList) && cleanList.Count > 0)
            return cleanList;

        // 3. Partial keyword contains
        foreach (var kvp in sequenceDict)
        {
            if (kvp.Key.ToLower().Contains(cleanQuery) || cleanQuery.Contains(CleanKey(kvp.Key)))
            {
                if (kvp.Value != null && kvp.Value.Count > 0)
                    return kvp.Value;
            }
        }

        return null;
    }

    [ContextMenu("Scan & Index All Nyxaris Frames")]
    public void AutoScanAndIndexAllNyxarisFrames()
    {
#if UNITY_EDITOR
        string rootPath = "Assets/Scenes/animations/frames/Nyxaris";
        if (!AssetDatabase.IsValidFolder(rootPath))
        {
            Debug.LogWarning($"[NyxarisFrameAnimator] Folder not found at {rootPath}");
            return;
        }

        sequences.Clear();
        string[] subDirs = System.IO.Directory.GetDirectories(rootPath);

        foreach (string dir in subDirs)
        {
            string folderName = System.IO.Path.GetFileName(dir);
            string unityPath = rootPath + "/" + folderName;

            string[] guids = AssetDatabase.FindAssets("t:Sprite", new[] { unityPath });
            if (guids == null || guids.Length == 0) continue;

            List<Sprite> frameList = new List<Sprite>();
            // Load and sort frames numerically (frame_001, frame_002, etc.)
            List<string> assetPaths = new List<string>();
            foreach (string guid in guids)
            {
                string p = AssetDatabase.GUIDToAssetPath(guid);
                if (p.EndsWith(".png") || p.EndsWith(".jpg"))
                {
                    assetPaths.Add(p);
                }
            }
        }
    }

```

```

    }
}
assetPaths.Sort(System.StringComparer.OrdinalIgnoreCase);

foreach (string p in assetPaths)
{
    Sprite s = AssetDatabase.LoadAssetAtPath<Sprite>(p);
    if (s != null && !frameList.Contains(s))
    {
        frameList.Add(s);
    }
}

if (frameList.Count > 0)
{
    NamedFrameSequence seq = new NamedFrameSequence
    {
        animationKey = folderName,
        frames = frameList
    };
    sequences.Add(seq);
}
}

BuildSequenceDictionary();
Debug.Log($"[NyxarisFrameAnimator] Successfully indexed {sequences.Count} animation fr
ame sequences for Nyxaris!");
EditorUtility.SetDirty(this);
#endif
}
}

```

FILE 36: Assets/Scenes/scripts/Nyxaris Chat Panel/NyxarisUIStyler.cs
PURPOSE: Builds and styles Nyxaris dialogue UI.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using TMPro;

/// <summary>
/// Faithful port of the Game Dev OS nyxaris.html web design into Unity UI.
/// Layout: full-width dialogue bar at bottom with 3px cyan top border,
/// portrait on the right side above the panel, white input + cyan send button.
/// </summary>
[ExecuteAlways]
[RequireComponent(typeof(CanvasGroup))]
public class NyxarisUIStyler : MonoBehaviour
{
    [Header("UI References ??? Core")]
    [Tooltip("Image on LowerPanel")]
    public Image dialoguePanelImage;
    [Tooltip("Image on Portrait")]
    public Image portraitImage;
    [Tooltip("Character name text (inside LowerPanel)")]
    public TMP_Text nameText;
    [Tooltip("Phase / status text (inside LowerPanel)")]
    public TMP_Text phaseText;
    [Tooltip("Main dialogue response text (inside LowerPanel)")]
    public TMP_Text dialogueText;
    [Tooltip("Image on MessageInput")]
    public Image inputFieldImage;
    [Tooltip("Image on SendButton")]
    public Image sendButtonImage;
    [Tooltip("TMP Text inside SendButton")]
    public TMP_Text sendButtonText;

    [Header("UI References ??? Extended (Game Dev OS)")]
    [Tooltip("Text showing sprite_key value below controls")]
    public TMP_Text spriteKeyText;
    [Tooltip("Loading text shown during API calls")]
    public TMP_Text loadingText;
    [Tooltip("Optional mode dropdown (idle/combat/story)")]

```

[illegible]

```

}

private void SanitizeLayoutValues()
{
    panelHeight = 250f;
    portraitWidth = 1200f; // 2x larger
    portraitHeight = 900f; // 2x larger
    portraitOffsetX = -40f; // Right edge offset
    portraitOffsetY = panelHeight - 2f; // Base touches top cyan divider of panel (Y = 248
)
    dialogueLeftPad = 36f;
    dialogueRightPad = -36f;
    dialogueTopPad = -42f;
    dialogueBottomPad = 92f;
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// MASTER APPLY
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
[ContextMenu("Apply Game Dev OS Chat UI")]
public void ApplyStyling()
{
    SanitizeLayoutValues();
    AutoResolveReferences();
    GenerateDynamicSprites();

    if (canvasGroup == null) canvasGroup = GetComponent<CanvasGroup>();

    // Hide obsolete letterbox cutscene bars
    Transform img0 = transform.Find("Image");
    if (img0 != null && img0 != dialoguePanelImage?.transform) img0.gameObject.SetActive(f
else);
    Transform img1 = transform.Find("Image (1)");
    if (img1 != null && img1 != dialoguePanelImage?.transform) img1.gameObject.SetActive(f
else);

    // Canvas scaler ??? 1920??1080 reference with balanced 0.5 match so UI never clips on
any aspect ratio
    Canvas nearestCanvas = GetComponentInParent<Canvas>();
    Canvas rootCanvas = nearestCanvas != null ? nearestCanvas.rootCanvas : null;
    if (rootCanvas != null)
    {
        CanvasScaler scaler = rootCanvas.GetComponent<CanvasScaler>();
        if (scaler == null) scaler = rootCanvas.gameObject.AddComponent<CanvasScaler>();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(1920f, 1080f);
        scaler.screenMatchMode = CanvasScaler.ScreenMatchMode.MatchWidthOrHeight;
        scaler.matchWidthOrHeight = 0.5f;
    }

    // MainInterface RectTransform fill
    RectTransform mainRt = transform as RectTransform;
    if (mainRt != null)
    {
        mainRt.anchorMin = Vector2.zero;
        mainRt.anchorMax = Vector2.one;
        mainRt.offsetMin = Vector2.zero;
        mainRt.offsetMax = Vector2.zero;
    }

    // UIspace ??? full-screen with subtle dark overlay (matches body bg #0f0f1a)
    if (dialoguePanelImage != null && dialoguePanelImage.transform.parent != null)
    {
        RectTransform uiSpace = dialoguePanelImage.transform.parent as RectTransform;
        if (uiSpace != null && uiSpace.name == "UIspace")
        {
            uiSpace.anchorMin = Vector2.zero;
            uiSpace.anchorMax = Vector2.one;
            uiSpace.offsetMin = Vector2.zero;
            uiSpace.offsetMax = Vector2.zero;

            Image bgImg = uiSpace.GetComponent<Image>();
            if (bgImg == null) bgImg = uiSpace.gameObject.AddComponent<Image>();
            bgImg.color = bodyBgColor;

```

```

    }
}

StyleLowerPanel();
StylePortrait();
StyleTextElements();
StyleInputField();
StyleSendButton();
StyleExtendedElements();

AttachPolishComponents();
}

// ?????????????????????????????????????????????????????????????????????????????????????
// DIALOGUE BOX ??? Fits camera horizontally across bottom
// ?????????????????????????????????????????????????????????????????????????????????????
// ?????????????????????????????????????????????????????????????????????????????????????
private void StyleLowerPanel()
{
    if (dialoguePanelImage == null) return;

    dialoguePanelImage.sprite = _roundedBoxSprite;
    dialoguePanelImage.type = Image.Type.Sliced;
    dialoguePanelImage.color = panelBgColor;

    // Solid full-width bottom dialogue bar with 250px height
    RectTransform rt = dialoguePanelImage.rectTransform;
    rt.anchorMin = new Vector2(0f, 0f);
    rt.anchorMax = new Vector2(1f, 0f);
    rt.pivot = new Vector2(0.5f, 0f);
    rt.sizeDelta = new Vector2(0f, panelHeight);
    rt.anchoredPosition = Vector2.zero;

    // Hide any old border
    Transform oldLine = dialoguePanelImage.transform.Find("TopBorderLine");
    if (oldLine != null) oldLine.gameObject.SetActive(false);

    // ????? 3px cyan divider at top (border-top: 3px solid #00d4ff) ?????
    Transform dividerLine = dialoguePanelImage.transform.Find("DividerLine");
    if (dividerLine == null)
    {
        var go = new GameObject("DividerLine", typeof(Image));
        go.transform.SetParent(dialoguePanelImage.transform, false);
        dividerLine = go.transform;
        dividerLine.SetAsFirstSibling();
    }

    Image dividerImg = dividerLine.GetComponent<Image>();
    dividerImg.color = accentCyan;

    RectTransform drt = dividerImg.rectTransform;
    drt.anchorMin = new Vector2(0f, 1f);
    drt.anchorMax = new Vector2(1f, 1f);
    drt.pivot = new Vector2(0.5f, 1f);
    drt.sizeDelta = new Vector2(0f, 3f);
    drt.anchoredPosition = Vector2.zero;
}

// ?????????????????????????????????????????????????????????????????????????????????????
// PORTRAIT ??? Tall, 2x large character resting on top of bar
// ?????????????????????????????????????????????????????????????????????????????????????
private void StylePortrait()
{
    if (portraitImage == null) AutoResolveReferences();
    if (portraitImage == null) return;

    // Ensure the portrait image is enabled and active
    portraitImage.enabled = true;
    portraitImage.gameObject.SetActive(true);

    // Resolve target UISpace parent
    Transform uiSpace = null;
    if (dialoguePanelImage != null && dialoguePanelImage.transform.parent != null)

```

[illegible]

```

        dialogueText.lineSpacing = 3f;

        // Hide speech bubble background/outline if present
        Transform container = dialogueText.transform.Find("SpeechBubbleBg");
        if (container != null)
        {
            container.gameObject.SetActive(false);
        }

        // Position: fills the panel area, with padding to avoid the portrait on right
        RectTransform txtRt = dialogueText.rectTransform;
        txtRt.anchorMin = Vector2.zero;
        txtRt.anchorMax = Vector2.one;
        txtRt.offsetMin = new Vector2(dialogueLeftPad, dialogueBottomPad);
        txtRt.offsetMax = new Vector2(dialogueRightPad, dialogueTopPad);
    }

    // ????? Name text: "Nyxaris" ??? bold, cyan, top-left of panel ?????
    if (nameText != null)
    {
        if (dialoguePanelImage != null && nameText.transform.parent != dialoguePanelImage.
transform)
        {
            try { nameText.transform.SetParent(dialoguePanelImage.transform, false); }
            catch (System.Exception) { }
        }

        nameText.color = accentCyan;
        nameText.fontSize = 26f;
        nameText.fontStyle = FontStyles.Bold;

        RectTransform nr = nameText.rectTransform;
        nr.anchorMin = new Vector2(0f, 1f);
        nr.anchorMax = new Vector2(0f, 1f);
        nr.pivot = new Vector2(0f, 1f);
        nr.anchoredPosition = new Vector2(24f, -10f);
        nr.sizeDelta = new Vector2(200f, 32f);
    }

    // ????? Phase text: magenta, small, to the right of name ?????
    if (phaseText != null)
    {
        phaseText.color = accentMagenta;
        phaseText.fontSize = 17f;
        phaseText.fontStyle = FontStyles.Normal;
        phaseText.alignment = TextAlignmentOptions.Left;

        RectTransform phr = phaseText.rectTransform;
        phr.anchorMin = new Vector2(0f, 1f);
        phr.anchorMax = new Vector2(0f, 1f);
        phr.pivot = new Vector2(0f, 1f);
        phr.anchoredPosition = new Vector2(270f, -18f);
        phr.sizeDelta = new Vector2(300f, 26f);

        // No badge background on phase text
        Image badgeBg = phaseText.GetComponent<Image>();
        if (badgeBg != null) badgeBg.enabled = false;
    }
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// INPUT FIELD ??? white bg, rounded, dark text (web: default input)
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
private void StyleInputField()
{
    if (inputFieldImage == null) return;

    inputFieldImage.sprite = _roundedBoxSprite;
    inputFieldImage.type = Image.Type.Sliced;
    inputFieldImage.color = inputBgColor;

    // Position: bottom row of the panel, elevated 22px with 52px height
    RectTransform ir = inputFieldImage.rectTransform;
    ir.anchorMin = new Vector2(0f, 0f);

```

```

ir.anchorMax = new Vector2(0.88f, 0f);
ir.pivot = new Vector2(0f, 0f);
ir.sizeDelta = new Vector2(-20f, 52f);
ir.anchoredPosition = new Vector2(36f, 22f);

// Hide any glow border
Transform glowBorder = inputFieldImage.transform.Find("InputGlowBorder");
if (glowBorder != null) glowBorder.gameObject.SetActive(false);

TMP_InputField inputField = inputFieldImage.GetComponent<TMP_InputField>();
if (inputField != null)
{
    // Expand text viewport / Text Area so the whole box is clickable and typeable
    Transform textArea = inputFieldImage.transform.Find("Text Area");
    if (textArea != null)
    {
        RectTransform taRt = textArea as RectTransform;
        if (taRt != null)
        {
            taRt.anchorMin = Vector2.zero;
            taRt.anchorMax = Vector2.one;
            taRt.offsetMin = new Vector2(16f, 4f);
            taRt.offsetMax = new Vector2(-16f, -4f);
        }
    }

    if (inputField.placeholder != null)
    {
        var ph = inputField.placeholder as TMP_Text;
        if (ph != null)
        {
            ph.color = new Color(0.5f, 0.5f, 0.5f, 0.7f);
            ph.fontSize = 20f;
            ph.text = "Talk to Nyxaris???";
            ph.alignment = TextAlignmentOptions.MidlineLeft;
        }
    }

    if (inputField.textComponent != null)
    {
        inputField.textComponent.color = inputTextColor;
        inputField.textComponent.fontSize = 20f;
        inputField.textComponent.alignment = TextAlignmentOptions.MidlineLeft;
    }
}
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// SEND BUTTON ??? cyan bg #00d4ff, dark bold text, rounded
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
private void StyleSendButton()
{
    if (sendButtonImage == null) return;

    sendButtonImage.sprite = _roundedBoxSprite;
    sendButtonImage.type = Image.Type.Sliced;
    sendButtonImage.color = accentCyan;

    // Position: right of the input field, same row, elevated 22px with 52px height
    RectTransform br = sendButtonImage.rectTransform;
    br.anchorMin = new Vector2(0.88f, 0f);
    br.anchorMax = new Vector2(1f, 0f);
    br.pivot = new Vector2(0f, 0f);
    br.sizeDelta = new Vector2(-48f, 52f);
    br.anchoredPosition = new Vector2(12f, 22f);

    if (sendButtonText != null)
    {
        sendButtonText.color = buttonTextColor;
        sendButtonText.fontSize = 20f;
        sendButtonText.fontStyle = FontStyles.Bold;
        sendButtonText.text = "send";
        sendButtonText.alignment = TextAlignmentOptions.Center;
    }
}

```

[illegible]

```

        // Hover effect on send button
        if (sendButtonImage != null && sendButtonImage.GetComponent<NyxarisBounceEffect>() ==
null)
        {
            var bounce = sendButtonImage.gameObject.AddComponent<NyxarisBounceEffect>();
            bounce.normalColor = accentCyan;
            bounce.hoverColor = buttonHoverColor;
            bounce.targetImage = sendButtonImage;
            bounce.bounceScale = 1.03f;
        }

        // Subtle hover effect on input field
        if (inputFieldImage != null && inputFieldImage.GetComponent<NyxarisBounceEffect>() ==
null)
        {
            var bounce = inputFieldImage.gameObject.AddComponent<NyxarisBounceEffect>();
            bounce.normalColor = inputBgColor;
            bounce.hoverColor = new Color(0.95f, 0.95f, 0.98f, 1f);
            bounce.targetImage = inputFieldImage;
            bounce.bounceScale = 1.01f;
        }
    }

    // ?????????????????????????????????????????????????????????????????????????????????????
    // PUBLIC API ??? for NyxarisManager to call
    // ?????????????????????????????????????????????????????????????????????????????????????
    ?????????????????????????????????????????????????????????????????????????????????????

    /// <summary>Show the loading indicator.</summary>
    public void ShowLoading()
    {
        if (loadingText != null) loadingText.gameObject.SetActive(true);
    }

    /// <summary>Hide the loading indicator.</summary>
    public void HideLoading()
    {
        if (loadingText != null) loadingText.gameObject.SetActive(false);
    }

    /// <summary>Update the sprite_key display text.</summary>
    public void SetSpriteKeyDisplay(string spriteKey)
    {
        if (spriteKeyText != null)
            spriteKeyText.text = $"sprite_key: {spriteKey}";
    }

    /// <summary>Get the currently selected mode from the dropdown.</summary>
    public string GetSelectedMode()
    {
        if (modeDropdown == null || modeDropdown.options.Count == 0) return "idle";
        return modeDropdown.options[modeDropdown.value].text;
    }

    /// <summary>Display interactive glowing button to launch recommended minigame directly.</
summary>
    public void ShowMinigameShortcut(string minigameName)
    {
        if (string.IsNullOrEmpty(minigameName))
        {
            HideMinigameShortcut();
            return;
        }

        if (minigameShortcutButton == null)
        {
            CreateMinigameShortcutObject();
        }

        if (minigameShortcutButton != null)
        {
            minigameShortcutButton.SetActive(true);
            if (minigameShortcutText != null)
            {

```

```

        minigameShortcutText.text = $"??? PLAY {minigameName.ToUpper()}";
    }

    Button btn = minigameShortcutButton.GetComponent<Button>();
    if (btn != null)
    {
        btn.onClick.RemoveAllListeners();
        btn.onClick.AddListener(() => LaunchRecommendedMinigame(minigameName));
    }
}

public void HideMinigameShortcut()
{
    if (minigameShortcutButton != null)
    {
        minigameShortcutButton.SetActive(false);
    }
}

private void LaunchRecommendedMinigame(string name)
{
    if (NyxarisManager.Instance != null)
    {
        NyxarisManager.Instance.HideInterface();
    }

    string clean = name.ToLower();
    if (clean.Contains("surge"))
    {
        SpawnOfChaos.Minigames.VoidSurgeArcadeUI.ShowMinigame();
    }
    else if (clean.Contains("orb") || clean.Contains("splash"))
    {
        SpawnOfChaos.Minigames.OrbSplashArcadeUI.ShowMinigame();
    }
    else if (clean.Contains("runner") || clean.Contains("shadow"))
    {
        SpawnOfChaos.Minigames.ShadowRunnerArcadeUI.ShowMinigame();
    }
    else if (clean.Contains("sky") || clean.Contains("bound"))
    {
        SpawnOfChaos.Minigames.SkyboundArcadeUI.ShowMinigame();
    }
    else
    {
        SpawnOfChaos.Minigames.MinigameHubUI.OpenHub();
    }
}

private void CreateMinigameShortcutObject()
{
    if (dialoguePanelImage == null) return;

    GameObject go = new GameObject("MinigameShortcutBtn", typeof(Image), typeof(Button));
    go.transform.SetParent(dialoguePanelImage.transform, false);

    Image img = go.GetComponent<Image>();
    img.sprite = _roundedBoxSprite;
    img.type = Image.Type.Sliced;
    img.color = new Color(0.85f, 0.15f, 0.95f, 1f); // Glowing Neon Magenta

    RectTransform rt = go.GetComponent<RectTransform>();
    rt.anchorMin = new Vector2(0f, 1f);
    rt.anchorMax = new Vector2(0f, 1f);
    rt.pivot = new Vector2(0f, 1f);
    rt.anchoredPosition = new Vector2(580f, -12f);
    rt.sizeDelta = new Vector2(210f, 34f);

    GameObject txtGo = new GameObject("Label", typeof(TMP_Text));
    txtGo.transform.SetParent(go.transform, false);
    TMP_Text tmp = txtGo.GetComponent<TMP_Text>();
    tmp.color = Color.white;
    tmp.fontSize = 15f;
    tmp.fontStyle = FontStyles.Bold;
    tmp.alignment = TextAlignmentOptions.Center;
}

```

```

    RectTransform trt = tmp.rectTransform;
    trt.anchorMin = Vector2.zero;
    trt.anchorMax = Vector2.one;
    trt.offsetMin = Vector2.zero;
    trt.offsetMax = Vector2.zero;

    minigameShortcutButton = go;
    minigameShortcutText = tmp;
}

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// OPEN / CLOSE ANIMATIONS
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????

public void AnimateOpen()
{
    if (!gameObject.activeInHierarchy) gameObject.SetActive(true);
    if (!gameObject.activeInHierarchy) return;

    if (openCloseCoroutine != null) StopCoroutine(openCloseCoroutine);
    openCloseCoroutine = StartCoroutine(DoOpenAnimation());
}

public void AnimateClose()
{
    if (!gameObject.activeInHierarchy) return;
    if (NyxarisManager.Instance != null && NyxarisManager.Instance.IsCinematicPlaying) return;

    if (openCloseCoroutine != null) StopCoroutine(openCloseCoroutine);
    openCloseCoroutine = StartCoroutine(DoCloseAnimation());
}

private IEnumerator DoOpenAnimation()
{
    canvasGroup.alpha = 0f;
    transform.localScale = new Vector3(0.99f, 0.99f, 1f);

    float duration = 0.25f;
    float elapsed = 0f;

    while (elapsed < duration)
    {
        elapsed += Time.unscaledDeltaTime;
        float t = elapsed / duration;
        float eased = 1f - Mathf.Pow(1f - t, 2f);

        transform.localScale = Vector3.Lerp(new Vector3(0.99f, 0.99f, 1f), Vector3.one, eased);
        canvasGroup.alpha = eased;
        yield return null;
    }

    transform.localScale = Vector3.one;
    canvasGroup.alpha = 1f;
}

private IEnumerator DoCloseAnimation()
{
    if (NyxarisManager.Instance != null && NyxarisManager.Instance.IsCinematicPlaying) yield break;
    float duration = 0.15f;
    float elapsed = 0f;
    Vector3 startScale = transform.localScale;

    while (elapsed < duration)
    {
        elapsed += Time.unscaledDeltaTime;
        float t = elapsed / duration;
        float eased = Mathf.SmoothStep(0f, 1f, t);

        transform.localScale = Vector3.Lerp(startScale, new Vector3(0.98f, 0.98f, 1f), eased);
    }
}

```

[illegible]

[illegible]

```

        Transform phTr = lp.Find("PhaseText");
        if (phTr != null) phaseText = phTr.GetComponent<TMP_Text>();

        Transform skTr = lp.Find("SpriteKeyText");
        if (skTr != null) spriteKeyText = skTr.GetComponent<TMP_Text>();

        Transform ldTr = lp.Find("LoadingText");
        if (ldTr != null) loadingText = ldTr.GetComponent<TMP_Text>();

        Transform inpTr = lp.Find("MessageInput") ?? lp.Find("InputField");
        if (inpTr != null) inputFieldImage = inpTr.GetComponent<Image>();

        Transform btnTr = lp.Find("SendButton") ?? lp.Find("Send");
        if (btnTr != null)
        {
            sendButtonImage = btnTr.GetComponent<Image>();
            sendButtonText = btnTr.GetComponentInChildren<TMP_Text>();
        }

        Transform mdTr = lp.Find("ModeDropdown") ?? lp.Find("Mode");
        if (mdTr != null) modeDropdown = mdTr.GetComponent<TMP_Dropdown>();
    }

    Transform portTr = uiSpace.Find("Portrait") ?? (lp != null ? lp.Find("Portrait") : null);
    if (portTr != null) portraitImage = portTr.GetComponent<Image>();

    if (portraitImage == null)
    {
        GameObject portGO = new GameObject("Portrait", typeof(RectTransform), typeof(CanvasRenderer), typeof(Image));
        portGO.transform.SetParent(uiSpace, false);
        portraitImage = portGO.GetComponent<Image>();
    }

    if (NyxarisManager.Instance != null && portraitImage != null)
    {
        NyxarisManager.Instance.portrait = portraitImage;
    }
}

public void SetActiveDot(int activeIndex, int totalDots = 3)
{
}

public void SetCinematicDialogueMode(bool isCinematic)
{
    // 1. Make root canvas and background completely transparent so BasePlayer and level are 100% visible!
    Image rootBg = GetComponent<Image>();
    if (rootBg != null)
    {
        rootBg.color = isCinematic ? new Color(0f, 0f, 0f, 0f) : new Color(0.145f, 0.145f, 0.22f, 1f);
        rootBg.raycastTarget = !isCinematic;
    }

    if (inputFieldImage != null)
    {
        inputFieldImage.gameObject.SetActive(!isCinematic);
    }
    if (sendButtonImage != null)
    {
        sendButtonImage.gameObject.SetActive(!isCinematic);
    }

    // Make full-screen UISpace background transparent during cinematic dialogue so level is visible!
    Transform uiSpace = dialoguePanelImage != null ? dialoguePanelImage.transform.parent : transform.Find("UISpace");
    if (uiSpace != null)
    {
        Image bg = uiSpace.GetComponent<Image>();
        if (bg != null)
        {
            bg.color = isCinematic ? new Color(0f, 0f, 0f, 0f) : bodyBgColor;
        }
    }
}

```



```

        baseScale = transform.localScale;
        if (targetImage != null) targetImage.color = normalColor;
    }

    public void OnPointerEnter(PointerEventData eventData)
    {
        if (targetImage != null) targetImage.color = hoverColor;
        TriggerScaleAnimation(baseScale * bounceScale, 0.16f);
    }

    public void OnPointerExit(PointerEventData eventData)
    {
        if (targetImage != null) targetImage.color = normalColor;
        TriggerScaleAnimation(baseScale, 0.16f);
    }

    public void OnPointerDown(PointerEventData eventData)
    {
        TriggerScaleAnimation(baseScale * 0.97f, 0.08f);
    }

    public void OnPointerUp(PointerEventData eventData)
    {
        TriggerScaleAnimation(baseScale * bounceScale, 0.12f);
    }

    private void TriggerScaleAnimation(Vector3 targetScale, float duration)
    {
        if (animCoroutine != null) StopCoroutine(animCoroutine);
        if (gameObject.activeInHierarchy)
        {
            animCoroutine = StartCoroutine(DoScale(targetScale, duration));
        }
    }

    private IEnumerator DoScale(Vector3 targetScale, float duration)
    {
        Vector3 startScale = transform.localScale;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.unscaledDeltaTime;
            float t = elapsed / duration;
            float tSmooth = Mathf.SmoothStep(0f, 1f, t);
            transform.localScale = Vector3.Lerp(startScale, targetScale, tSmooth);
            yield return null;
        }

        transform.localScale = targetScale;
    }
}

```

FILE 37: Assets/Scenes/scripts/Nyxaris Chat Panel/NyxarisManager.cs
 PURPOSE: Handles Nyxaris chat and API communication.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Networking;
using UnityEngine.UI;
using TMPro;
using UnityEngine.SceneManagement;
#if UNITY_EDITOR
using UnityEditor;
#endif

public class NyxarisManager : MonoBehaviour
{
    public static NyxarisManager Instance { get; private set; }

    [Header("UI Structure")]

```

```

public GameObject mainInterfacePanel;

[Header("UI Elements")]
public TMP_Text dialogueText;
public TMP_InputField messageInput;
public Image portrait;
public NyxarisFrameAnimator frameAnimator;

[Header("Direct Expression Sprites (Game Dev OS Fallback)")]
public Sprite neutralSprite;
public Sprite explainingSprite;
public Sprite cuteSprite;

[Header("Sprite Lists")]
public Sprite[] neutralSprites;
public Sprite[] explainingSprites;
public Sprite[] thinkingSprites;
public Sprite[] angrySprites;
public Sprite[] cuteSprites;
public Sprite[] motherlySprites;
public Sprite[] kindSprites;

[Header("Current Game State Connection")]
[Tooltip("Options: idle, combat, story")]
public string currentMode = "idle";
[Range(0f, 1f)]
public float currentTrust = 0.5f;

[Header("API Server Config")]
public string apiHost = "http://127.0.0.1:5000/nyxaris";
public string apiHostFallback = "http://127.0.0.1:5001/nyxaris";

[System.Serializable]
public class NyxarisRequest
{
    public string message;
    public string mode;
    public float trust;
    public string level;
    public float player_hp;
    public float player_max_hp;
    public float player_mana;
    public float player_max_mana;
}

[System.Serializable]
public class NyxarisResponse
{
    public string response;
    public string emotion;
    public string animation;
    public string sprite_key;
    public string suggested_minigame;
    public float new_trust;
}

public bool IsCinematicPlaying => cinematicCoroutine != null;
public static bool IsChatActive
{
    get
    {
        if (Instance == null) return false;
        if (Instance.IsCinematicPlaying) return true;
        NyxarisUIStyler styler = Instance.GetStyler();
        if (styler != null && styler.gameObject.activeInHierarchy)
        {
            CanvasGroup cg = styler.GetComponent<CanvasGroup>();
            if (cg != null) return cg.alpha > 0.05f;
        }
        Transform uiSpace = Instance.transform.Find("UISpace");
        if (uiSpace != null && uiSpace.gameObject.activeSelf) return true;
        if (Instance.mainInterfacePanel != null && Instance.mainInterfacePanel != Instance
            .gameObject)
        {
            CanvasGroup cg = Instance.mainInterfacePanel.GetComponent<CanvasGroup>();
            if (cg != null) return cg.alpha > 0.05f;
        }
    }
}

```

```

        return Instance.mainInterfacePanel.activeSelf;
    }
    return false;
}

public static bool IsTyping
{
    get
    {
        if (IsChatActive) return true;
        if (UnityEngine.EventSystems.EventSystem.current != null && UnityEngine.EventSystem
ms.EventSystem.current.currentSelectedGameObject != null)
        {
            var go = UnityEngine.EventSystems.EventSystem.current.currentSelectedGameObjec
t;
            if (go.GetComponent<TMPPro.TMP_InputField>() != null || go.GetComponent<UnityEn
gine.UI.InputField>() != null)
                return true;
        }
        return false;
    }
}

private Coroutine currentTypewriterCoroutine;

void Awake()
{
    Instance = this;
    AutoLoadExpressionSprites();
    EnsureCanvasScaling();

    if (mainInterfacePanel == null)
    {
        mainInterfacePanel = gameObject;
    }

    if (mainInterfacePanel != null)
    {
        mainInterfacePanel.SetActive(false);
    }
}

void Start()
{
    AutoLoadExpressionSprites();
    EnsureCanvasScaling();
    SetupPortraitAndAnimator();

    NyxarisUIStyler styler = GetStyler();
    if (styler != null)
    {
        styler.ApplyStyling();
    }

    if (messageInput != null)
    {
        messageInput.onSubmit.AddListener((text) => SendInputMessage());
    }

    if (!IsCinematicPlaying)
    {
        CanvasGroup cg = GetComponent<CanvasGroup>();
        if (cg != null) { cg.alpha = 0f; cg.blocksRaycasts = false; cg.interactable = fals
e; }

        if (mainInterfacePanel != null)
        {
            mainInterfacePanel.SetActive(false);
        }
    }
}

private void EnsureCanvasScaling()
{
    Canvas rootCanvas = null;
    if (mainInterfacePanel != null)

```

```

    {
        Canvas c = mainInterfacePanel.GetComponentInParent<Canvas>();
        if (c != null) rootCanvas = c.rootCanvas;
    }
    if (rootCanvas == null)
    {
        Canvas[] all = FindObjectsByType<Canvas>(FindObjectsSortMode.None);
        if (all != null && all.Length > 0) rootCanvas = all[0].rootCanvas;
    }

    if (rootCanvas != null)
    {
        CanvasScaler scaler = rootCanvas.GetComponent<CanvasScaler>();
        if (scaler == null) scaler = rootCanvas.gameObject.AddComponent<CanvasScaler>();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(1920f, 1080f);
        scaler.screenMatchMode = CanvasScaler.ScreenMatchMode.MatchWidthOrHeight;
        scaler.matchWidthOrHeight = 1.0f;
    }
}

public void SetupPortraitAndAnimator()
{
    // Cleanup any stray untextured overlay
    GameObject stray = GameObject.Find("NyxarisPortraitOverlay");
    if (stray != null)
    {
        Destroy(stray);
    }

    NyxarisUIStyler styler = GetStyler();
    if (styler != null)
    {
        styler.ApplyStyling();
        if (styler.portraitImage != null) portrait = styler.portraitImage;
        if (styler.dialogueText != null) dialogueText = styler.dialogueText;
    }

    if (dialogueText == null && mainInterfacePanel != null)
    {
        Transform dt = mainInterfacePanel.transform.Find("UIspace/LowerPanel/DialogueText"
) ??
            mainInterfacePanel.transform.Find("LowerPanel/DialogueText") ??
            mainInterfacePanel.transform.Find("DialogueText");
        if (dt != null) dialogueText = dt.GetComponent<TMP_Text>();
    }
    if (dialogueText == null)
    {
        TMP_Text[] texts = GetComponentsInChildren<TMP_Text>(true);
        foreach (var t in texts)
        {
            if (t.gameObject.name.ToLower().Contains("dialogue"))
            {
                dialogueText = t;
                break;
            }
        }
        if (dialogueText == null && texts.Length > 0) dialogueText = texts[0];
    }

    if (portrait == null && mainInterfacePanel != null)
    {
        Transform p = mainInterfacePanel.transform.Find("UIspace/Portrait") ??
            mainInterfacePanel.transform.Find("Portrait");
        if (p != null) portrait = p.GetComponent<Image>();
    }
    if (portrait == null)
    {
        Image[] images = GetComponentsInChildren<Image>(true);
        foreach (var img in images)
        {
            if (img.gameObject.name.ToLower().Contains("portrait"))
            {
                portrait = img;
                break;
            }
        }
    }
}

```

```

    }
    if (portrait == null && images.Length > 0) portrait = images[0];
}

if (messageInput == null)
{
    messageInput = GetComponentInChildren<TMP_InputField>(true);
}

if (portrait != null)
{
    portrait.color = Color.white;
    portrait.preserveAspect = true;

    frameAnimator = portrait.GetComponent<NyxarisFrameAnimator>();
    if (frameAnimator == null)
    {
        frameAnimator = portrait.gameObject.AddComponent<NyxarisFrameAnimator>();
    }
    frameAnimator.targetImage = portrait;
    frameAnimator.PlayAnimation("nuetral", 24f);
}

}

public void EnsureDefaultPortrait()
{
    if (portrait == null) SetupPortraitAndAnimator();
    if (portrait == null) return;

    portrait.enabled = true;
    portrait.gameObject.SetActive(true);
    portrait.color = Color.white;
    portrait.preserveAspect = true;

    if (frameAnimator != null)
    {
        frameAnimator.targetImage = portrait;
        if (!frameAnimator.IsPlaying && !frameAnimator.IsFrozenOnLastFrame)
        {
            frameAnimator.PlayAnimation("nuetral", 24f);
        }
    }
    else if (portrait.sprite == null)
    {
        AutoLoadExpressionSprites();
        Sprite def = neutralSprite ??
            (neutralSprites != null && neutralSprites.Length > 0 ? neutralSprites[
0] : null) ??
            explainingSprite ?? cuteSprite;
        if (def != null) portrait.sprite = def;
    }
}

public void ShowInterface()
{
    if (HUDManager.IsInMainMenu())
    {
        HideInterface();
        return;
    }

    if (mainInterfacePanel != null)
    {
        mainInterfacePanel.SetActive(true);
        EnsureCanvasScaling();

        NyxarisUIStyler styler = GetStyler();
        if (styler != null)
        {
            styler.ApplyStyling();
            styler.AnimateOpen();
        }

        EnsureDefaultPortrait();

        if (messageInput != null)

```

```

        {
            messageInput.text = "";
            messageInput.ActivateInputField();
        }
        HUDManager.Instance?.UpdateVisibility();
        SpawnOfChaos.Minigames.HUDOrbPanel.Instance?.UpdateVisibility();
    }
}

public void HideInterface()
{
    if (mainInterfacePanel != null)
    {
        NyxarisUIStyler styler = GetStyler();
        if (styler != null)
        {
            styler.AnimateClose();
        }
        else
        {
            mainInterfacePanel.SetActive(false);
        }
    }
    HUDManager.Instance?.UpdateVisibility();
    SpawnOfChaos.Minigames.HUDOrbPanel.Instance?.UpdateVisibility();
}

void Update()
{
    if (HUDManager.IsInMainMenu())
    {
        if (mainInterfacePanel != null && mainInterfacePanel.activeSelf)
        {
            HideInterface();
        }
        return;
    }

    if (mainInterfacePanel != null && mainInterfacePanel.activeSelf)
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            HideInterface();
        }

        if ((Input.GetKeyDown(KeyCode.Return) || Input.GetKeyDown(KeyCode.KeypadEnter)))
        {
            if (messageInput != null && !string.IsNullOrEmpty(messageInput.text))
            {
                SendInputMessage();
            }
        }
    }

    if (Input.GetKeyDown(KeyCode.Tab))
    {
        if (mainInterfacePanel != null)
        {
            CanvasGroup cg = mainInterfacePanel.GetComponent<CanvasGroup>();
            bool isVisible = mainInterfacePanel.activeSelf && (cg == null || cg.alpha > 0.01f);

            if (isVisible)
                HideInterface();
            else
                ShowInterface();
        }
    }
}

private NyxarisUIStyler GetStyler()
{
    if (mainInterfacePanel == null) return null;
    return mainInterfacePanel.GetComponent<NyxarisUIStyler>() ??
        mainInterfacePanel.GetComponentInChildren<NyxarisUIStyler>(true);
}

```

```

// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????
// MESSAGING & BRAIN INTEGRATION
// ?????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????

```

```

public void SendInputMessage()
{
    string msg = messageInput.text;
    if (string.IsNullOrEmpty(msg)) return;

    StartCoroutine(SendRequest(msg));
    messageInput.text = "";
    messageInput.ActivateInputField();
}

IEnumerator SendRequest(string msg)
{
    NyxarisUIStyler styler = GetStyler();
    string mode = currentMode;
    if (styler != null)
    {
        string dropdownMode = styler.GetSelectedMode();
        if (!string.IsNullOrEmpty(dropdownMode)) mode = dropdownMode;
    }

    // Live Player Stats Injection
    float hp = 100f, maxHp = 100f, mana = 100f, maxMana = 100f;
    GameObject player = GameObject.FindGameObjectWithTag("Player");
    if (player != null)
    {
        Health h = player.GetComponent<Health>();
        if (h != null)
        {
            hp = h.CurrentHealth;
            maxHp = h.MaxHealth;
        }
        MageCombat mc = player.GetComponent<MageCombat>();
        if (mc != null)
        {
            mana = mc.currentMana;
            maxMana = mc.maxMana;
        }
    }

    NyxarisRequest requestData = new NyxarisRequest
    {
        message = msg,
        mode = mode,
        trust = currentTrust,
        level = SceneManager.GetActiveScene().name,
        player_hp = hp,
        player_max_hp = maxHp,
        player_mana = mana,
        player_max_mana = maxMana
    };
    string json = JsonUtility.ToJson(requestData);

    if (styler != null) styler.ShowLoading();

    bool success = false;
    NyxarisResponse response = null;

    // 1. Try Primary API Host (5000)
    yield return TryPostRequest(apiHost, json, (res) => { response = res; success = true;
});

    // 2. Try Fallback Host (5001) if not successful
    if (!success)
    {
        yield return TryPostRequest(apiHostFallback, json, (res) => { response = res; succ
ess = true; });
    }

    if (styler != null) styler.HideLoading();

```

```

// 3. Built-in Local Heuristic Mind Fallback
if (!success || response == null || string.IsNullOrEmpty(response.response))
{
    response = GetLocalFastResponse(msg, hp, maxHp, mana, maxMana);
}

// Apply Trust Delta
if (response.new_trust > 0f)
{
    currentTrust = Mathf.Clamp01(response.new_trust);
}

if (styler != null && !string.IsNullOrEmpty(response.sprite_key))
    styler.SetSpriteKeyDisplay(response.sprite_key);

// Show/Hide Minigame Quick-Launch Button
if (styler != null)
{
    if (!string.IsNullOrEmpty(response.suggested_minigame))
        styler.ShowMinigameShortcut(response.suggested_minigame);
    else
        styler.HideMinigameShortcut();
}

// Play Frame Animation (smooth 20 FPS, freeze on last frame)
PlayAnimation(response.animation, response.emotion);

if (currentTypewriterCoroutine != null) StopCoroutine(currentTypewriterCoroutine);
currentTypewriterCoroutine = StartCoroutine(TypeText(response.response));

// Track casual conversation exchanges with Nyxaris (prompts ad communion after 4 mess
ages, never in cutscenes)
if (AdManager.Instance != null && mode != "story")
{
    AdManager.Instance.RecordNyxarisCasualExchange();
}

private IEnumerator TryPostRequest(string url, string json, System.Action<NyxarisResponse>
onParsed)
{
    using (UnityWebRequest req = new UnityWebRequest(url, "POST"))
    {
        byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(json);
        req.uploadHandler = new UploadHandlerRaw(bodyRaw);
        req.downloadHandler = new DownloadHandlerBuffer();
        req.SetRequestHeader("Content-Type", "application/json");
        req.timeout = 15; // Generous timeout for local AI model generation

        yield return req.SendWebRequest();

        if (req.result == UnityWebRequest.Result.Success && !string.IsNullOrEmpty(req.down
loadHandler.text))
        {
            try
            {
                NyxarisResponse parsed = JsonUtility.FromJson<NyxarisResponse>(req.downloa
dHandler.text);
                if (parsed != null && !string.IsNullOrEmpty(parsed.response))
                {
                    onParsed?.Invoke(parsed);
                }
            }
            catch (System.Exception ex)
            {
                Debug.LogWarning("[NyxarisManager] Parse error from " + url + ": " + ex.Me
ssage);
            }
        }
    }
}

public void PlayAnimation(string animKey, string fallbackEmotion = "")
{
    if (frameAnimator != null)
    {

```

[illegible]

```

{
    string[] responses = {
        "You are battered, mortal! Do not throw your life away so recklessly. Take a breath or train in the Void Surge.",
        "Your wounds are severe! Step back and gather health orbs before you collapse!",
        "Look at you, barely standing! A fallen warrior is of no use to my mission. Recover immediately!"
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "cutely_upset";
    anim = "nyxariscutelyupset-72bf7d95";
    suggestedMg = "VoidSurge";
    trustDelta = 0.04f;
}
// 2. Low Mana Reaction
else if (manaPct < 0.30f && (textLower.Contains("mana") || textLower.Contains("spell") || textLower.Contains("mp") || textLower.Contains("magic") || textLower.Contains("empty") || textLower.Contains("cast")))
{
    string[] responses = {
        "Your mana is dangerously depleted. Refresh your arcane flow before engaging the next clan guardian.",
        "You cannot conjure spells on empty reserves! Collect mana orbs or meditate for a moment.",
        "The void essence within you runs thin. Recharging your magic is essential right now."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "warning";
    anim = "nyxarissternorimportantwarning-534901f6";
    suggestedMg = "OrbSplash";
    trustDelta = 0.03f;
}
// 3. Minigame & Training Inquiries
else if (textLower.Contains("minigame") || textLower.Contains("arcade") || textLower.Contains("train") || textLower.Contains("practice") || textLower.Contains("void surge") || textLower.Contains("orb splash") || textLower.Contains("shadow runner") || textLower.Contains("skybound") || textLower.Contains("play"))
{
    string[] games = { "VoidSurge", "OrbSplash", "ShadowRunner", "SkyboundBox" };
    suggestedMg = games[Random.Range(0, games.Length)];
    string[] responses = {
        $"Sharpen your instincts in {suggestedMg}! Gathering void essence now will make your strikes lethal.",
        $"A true warrior trains constantly. Test your reflexes in {suggestedMg} and return stronger!",
        $"Looking to hone your arcane mastery? Step into {suggestedMg} and collect divine rewards."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "excited";
    anim = excitedPool[Random.Range(0, excitedPool.Length)];
    trustDelta = 0.05f;
}
// 4. Identity & Lore of Nyxaris
else if (textLower.Contains("who are you") || textLower.Contains("what are you") || textLower.Contains("your name") || textLower.Contains("goddess") || textLower.Contains("about yourself"))
{
    string[] responses = {
        "I am Nyxaris, Goddess of the Dark Multiverse. Bound across timelines to restore cosmic balance??and uncover the truth behind my followers' demise.",
        "You stand before Nyxaris. Though my mortal form is diminished in this realm, the primordial void still answers my command.",
        "I am the sovereign of shadows and forgotten realms. Together, we are going to unravel the conspiracy consuming this forest."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "confidently";
    anim = explainingPool[Random.Range(0, explainingPool.Length)];
    trustDelta = 0.03f;
}
// 5. Followers Massacre & Mystery Lore
else if (textLower.Contains("who") || textLower.Contains("killer") || textLower.Contains("massacre") || textLower.Contains("follower") || textLower.Contains("culprit") || textLow

```

```

er.Contains("murder") || textLower.Contains("died") || textLower.Contains("who did this"))
{
    string[] responses = {
        "My followers were slaughtered in cold blood across this timeline. Clues point
        to a warrior hidden among the mountain clans???or something far darker impersonating them."
    ,
        "Someone orchestrated the massacre to spark war between the Strawhat and Samur
        ai clans. We must expose the imposter before more blood is spilled.",
        "The killer leaves a trail of deception. Investigate both d??j??s in the mount
        ains???the evidence will lead us to the culprit."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "explaining";
    anim = explainingPool[Random.Range(0, explainingPool.Length)];
    trustDelta = 0.03f;
}
// 6. Strawhat Clan Lore
else if (textLower.Contains("strawhat") || textLower.Contains("dojo 1") || textLower.C
ontains("dojo1") || textLower.Contains("straw"))
{
    string[] responses = {
        "The Strawhat Clan claims innocence, insisting the killer is a shape-shifter i
        n the Samurai Clan. Do not lower your guard in their d??j???",
        "Their warriors fight with swift straw blades. Challenge their altar guardian
        and demand the truth about the killings.",
        "The Strawhat masters know more than they let on. Watch their movements carefu
        lly when you step past their gates."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "thinking";
    anim = thinkingPool[Random.Range(0, thinkingPool.Length)];
}
// 7. Samurai Clan Lore
else if (textLower.Contains("samurai") || textLower.Contains("dojo 2") || textLower.Co
ntains("dojo2") || textLower.Contains("master"))
{
    string[] responses = {
        "Rumors say the Samurai Clan's Master was sighted alive, despite dying two yea
        rs ago... Be vigilant; things are not as they appear.",
        "The Samurai Clan blames the Strawhats, but this resurrected Master suggests d
        ark illusions are at play.",
        "Face the Samurai Clan guardian. We must determine if their fallen Master has
        truly returned from the grave."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "warning";
    anim = "nyxarissternorimportantwarning-534901f6";
}
// 8. Cave & Tsuchigumo Reveal
else if (textLower.Contains("cave") || textLower.Contains("spider") || textLower.Conta
ins("tsuchigumo") || textLower.Contains("web"))
{
    string[] responses = {
        "The giant spiders multiplying in the regional cave serve Tsuchigumo???the tru
        e shape-shifting culprit behind the massacre!",
        "Tsuchigumo weaves webs of discord, impersonating both clans to fuel their hat
        red. Descend the cave and crush this beast!",
        "The bottom of the mountain cave holds the final answer. Steel yourself, morta
        l???Tsuchigumo will not surrender easily."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "pissed";
    anim = "nyxarispissed-7e387d67";
    trustDelta = 0.05f;
}
// 9. Guidance, Navigation & "What should I do?"
else if (textLower.Contains("what should i do") || textLower.Contains("what now") || t
extLower.Contains("where do i go") || textLower.Contains("where to go") || textLower.Contain
s("next") || textLower.Contains("lost") || textLower.Contains("guide me") || textLower.Conta
ins("direction"))
{
    string[] responses = {
        "Head upward through the mountain path. We must visit both D??j?? 1 (Strawhat)
        and D??j?? 2 (Samurai) to gather clues before entering the cave.",
        "Explore the Cherry Blossom Village and test your blade against clan warriors.
        When you are ready, the cave depths await.",
    }
}

```

```

        "Keep advancing along the stone path. Every enemy defeated brings us closer to
        uncovering Tsuchigumo's nest."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "explaining";
    anim = explainingPool[Random.Range(0, explainingPool.Length)];
}
// 10. Flirting / High Affinity
else if (textLower.Contains("cute") || textLower.Contains("love") || textLower.Contains("pretty") || textLower.Contains("beautiful") || textLower.Contains("marry") || textLower.Contains("kiss") || textLower.Contains("gorgeous") || textLower.Contains("sweet"))
{
    if (currentTrust > 0.65f)
    {
        string[] responses = {
            "H-hush, mortal! A goddess does not get swayed by simple sweet-talking... Though, I suppose your company isn't entirely dreadful.",
            "Flattery from you is surprisingly pleasant... Not that I'm getting attached or anything! Keep your eyes on the road.",
            "You truly are bold to speak to a deity like that. Just make sure you stay alive so I can keep hearing it."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "in_love";
        anim = affectionPool[Random.Range(0, affectionPool.Length)];
        trustDelta = 0.08f;
    }
    else
    {
        string[] responses = {
            "Flattery will not distract me from our mission, mortal. Focus on the investigation at hand!",
            "Do not think cheap compliments will earn you divine favor so easily. Prove your worth in battle first!",
            "A goddess has no time for idle flirtation. Keep your blade sharp and your mind focused."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "cutely_annoyed";
        anim = tsundereTeasePool[Random.Range(0, tsundereTeasePool.Length)];
    }
}
// 11. Playful, Teasing, or Humorous
else if (textLower.Contains("tease") || textLower.Contains("annoying") || textLower.Contains("bossy") || textLower.Contains("funny") || textLower.Contains("joke") || textLower.Contains("short") || textLower.Contains("horns") || textLower.Contains("lazy") || textLower.Contains("food") || textLower.Contains("eat") || textLower.Contains("hungry") || textLower.Contains("baka"))
{
    string[] responses = {
        "Who are you calling bossy?! I am guiding you so you don't wander off a cliff, ungrateful mortal!",
        "Keep making remarks like that and I might just let the next spider have a nibble of your cloak!",
        "My mana reserves require constant replenishment... which totally includes delicious festival treats, obviously!"
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "cutely_annoyed";
    anim = tsundereTeasePool[Random.Range(0, tsundereTeasePool.Length)];
    trustDelta = 0.03f;
}
// 12. Gratitude, Agreement & Affirmations
else if (textLower.Contains("thank") || textLower.Contains("thanks") || textLower.Contains("ok") || textLower.Contains("okay") || textLower.Contains("alright") || textLower.Contains("got it") || textLower.Contains("understood") || textLower.Contains("yes") || textLower.Contains("yeah") || textLower.Contains("sure") || textLower.Contains("will do"))
{
    string[] responses = {
        "Good. As long as we understand each other, nothing in this forest can stand in our way.",
        "I expect nothing less from my companion. Let us proceed with haste.",
        "Very well. Lead onward, and strike true when the moment comes."
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "happy_to_say";
}

```

```

        anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
        trustDelta = 0.02f;
    }
    // 13. General Readiness & "How are you?"
    else if (textLower.Contains("ready") || textLower.Contains("how are you") || textLower
.Contains("how r u") || textLower.Contains("what's up") || textLower.Contains("whats up") ||
textLower.Contains("how do you feel") || textLower.Contains("are you okay") || textLower.Co
ntains("you ready"))
    {
        if (currentTrust > 0.6f)
        {
            string[] responses = {
                "I am primed for battle and eager to see what we uncover next. How are you
holding up?",
                "My arcane senses are tingling with anticipation. Whenever you are ready t
o move, I am with you.",
                "Feeling stronger by your side, mortal. Let us see what secrets this mount
ain still hides."
            };
            respText = responses[Random.Range(0, responses.Length)];
            emotion = "happy_to_say";
            anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
            trustDelta = 0.03f;
        }
        else
        {
            string[] responses = {
                "My senses are focused on the investigation. Make sure your reflexes are j
ust as sharp.",
                "I am ready when you are. Do not let your guard down for a single moment."
            },
            "Standing by. Speak your intent or lead the way toward our next objective."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "neutral";
        anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
    }
}
// 14. Greetings & Casual Status
else if (textLower.Contains("hello") || textLower.Contains("hi") || textLower.Contains
("hey") || textLower.Contains("nyxaris") || textLower.Contains("greetings") || textLower.Con
tains("yo"))
{
    if (hpPct < 0.4f)
    {
        respText = "I am with you, mortal. But you look exhausted???rest a moment befo
re charging into danger.";
        emotion = "worried_upset";
        anim = "nyxarisworriedupsetthinkingmp4-bd2148fc";
        suggestedMg = "OrbSplash";
    }
    else if (currentTrust > 0.6f)
    {
        string[] responses = {
            "Greetings! It is good to see you standing tall. What shall we investigate
next?",
            "Ah, there you are. I was wondering when you would seek my counsel again."
        },
        "Hello, companion. Ready to turn this mountain upside down?"
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "happy_to_say";
    anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
    trustDelta = 0.03f;
}
else
{
    string[] responses = {
        "I am with you, mortal. Speak your mind or ask for guidance on our investi
gation.",
        "Greetings. What observations do you have to share from your journey?",
        "I am listening. What direction shall we take?"
    };
    respText = responses[Random.Range(0, responses.Length)];
    emotion = "neutral";
}

```

```

        anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
    }
}
// 15. Dynamic Conversational Fallback
else
{
    if (activeScene.Contains("Dojo1"))
    {
        string[] responses = {
            "We are at the Strawhat D??j??. Stay alert and watch for any hidden clues
near their altar.",
            "The Strawhat warriors are sizing you up. Speak with their master or chall
enge their champions.",
            "Look around this d??j?? carefully. The killer may have left traces of the
ir presence."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "explaining";
        anim = explainingPool[Random.Range(0, explainingPool.Length)];
    }
    else if (activeScene.Contains("Dojo2"))
    {
        string[] responses = {
            "This d??j?? reeks of deception. The Master who stands before you is a fal
se illusion!",
            "Keep your distance from the Samurai guards until we verify who is command
ing them.",
            "Something is unnatural about this place. Be ready to draw your weapon at
a moment's notice."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "thinking";
        anim = thinkingPool[Random.Range(0, thinkingPool.Length)];
    }
    else if (activeScene.Contains("Cave"))
    {
        string[] responses = {
            "We stand at the threshold of truth. Tsuchigumo awaits below in the webs.
Prepare yourself for battle!",
            "The webs grow thicker here. Watch the shadows above as we descend into th
e lair.",
            "Tsuchigumo's venomous presence is heavy in the air. Let us cleanse this c
ave together!"
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "angry";
        anim = "nyxarisangry-e56db4b1";
    }
    else
    {
        string[] responses = {
            "I hear you, mortal. Keep moving through the Cherry Blossom Forest???the c
lues we need are waiting ahead.",
            "Interesting thought. Let us press onward to the mountain d??j??s and see
what we can find.",
            "Indeed. Stay vigilant and keep your blade ready. Every step brings us clo
ser to the truth.",
            "A curious remark. Keep your focus on our quest, and let us unveil what li
es in the cave."
        };
        respText = responses[Random.Range(0, responses.Length)];
        emotion = "explaining";
        anim = casualTalkPool[Random.Range(0, casualTalkPool.Length)];
    }
}

return new NyxarisResponse
{
    response = respText,
    emotion = emotion,
    animation = anim,
    sprite_key = emotion,
    suggested_minigame = suggestedMg,
    new_trust = Mathf.Clamp01(currentTrust + trustDelta)
};
}

```

```

private void SetEmotionFallback(string emotion)
{
    Sprite selectedSprite = neutralSprite;
    string e = emotion.ToLower();
    if (e.Contains("explain")) selectedSprite = explainingSprite ?? neutralSprite;
    else if (e.Contains("annoy") || e.Contains("cute") || e.Contains("love")) selectedSprite = cuteSprite ?? neutralSprite;

    if (portrait != null && selectedSprite != null)
    {
        portrait.sprite = selectedSprite;
    }
}

IEnumerator TypeText(string text)
{
    NyxarisUIStyler styler = GetStyler();
    if (styler != null) styler.BouncePortrait();

    if (dialogueText == null) yield break;

    dialogueText.text = "";
    foreach (char c in text)
    {
        dialogueText.text += c;
        yield return new WaitForSecondsRealtime(0.008f); // Realtime typewriter
    }
    dialogueText.text = text;
}

public void AutoLoadExpressionSprites()
{
    if (neutralSprite == null) neutralSprite = Resources.Load<Sprite>("NyxarisExpressions/neutral");
    if (explainingSprite == null) explainingSprite = Resources.Load<Sprite>("NyxarisExpressions/explaining");
    if (cuteSprite == null) cuteSprite = Resources.Load<Sprite>("NyxarisExpressions/cutely-annoyed");
}

[System.Serializable]
public class CinematicLine
{
    public string animationKey;
    public string text;

    public CinematicLine(string anim, string t)
    {
        animationKey = anim;
        text = t;
    }
}

public static NyxarisManager EnsureInstanceInScene()
{
    if (Instance != null) return Instance;

    NyxarisManager existing = Object.FindAnyObjectByType<NyxarisManager>(FindObjectsInactive.Include);
    if (existing != null)
    {
        Instance = existing;
        existing.SetupPortraitAndAnimator();
        return existing;
    }

    GameObject prefab = Resources.Load<GameObject>("Prefabs/MainInterface") ?? Resources.Load<GameObject>("MainInterface");

#if UNITY_EDITOR
    if (prefab == null)
    {
        prefab = UnityEditor.AssetDatabase.LoadAssetAtPath<GameObject>("Assets/Prefabs/MainInterface.prefab");
    }
#endif
}

```

```

    if (prefab != null)
    {
        GameObject go = Object.Instantiate(prefab);
        go.name = "MainInterface";
        NyxarisManager mgr = go.GetComponentInChildren<NyxarisManager>(true);
        if (mgr == null)
        {
            mgr = go.AddComponent<NyxarisManager>();
        }
        mgr.mainInterfacePanel = go;
        mgr.SetupPortraitAndAnimator();
        Transform uiSpace = go.transform.Find("UISpace");
        if (uiSpace != null) uiSpace.gameObject.SetActive(false);
        CanvasGroup cg = go.GetComponent<CanvasGroup>();
        if (cg != null) cg.alpha = 0f;
        Instance = mgr;
        return mgr;
    }
    return null;
}

private Coroutine cinematicCoroutine;

/// <summary>
/// Starts a one-way cinematic dialogue sequence where player presses [E] to advance.
/// Input typing is disabled, and animated character portraits react to each line.
/// </summary>
public void StartCinematicStoryDialogue(CinematicLine[] lines, System.Action onComplete)
{
    if (HUDManager.IsInMainMenu())
    {
        Debug.LogWarning("[NyxarisManager] Blocked StartCinematicStoryDialogue: Main Menu
is active!");
        return;
    }

    // 1. Stop any closing animations or coroutines immediately and enable cinematic dialo
gue mode
    NyxarisUIStyler styler = GetStyler();
    if (styler != null)
    {
        styler.StopAllCoroutines();
        styler.SetCinematicDialogueMode(true);
    }

    // 2. Ensure GameObject is fully active and scale is 1
    gameObject.SetActive(true);
    transform.localScale = Vector3.one;
    if (mainInterfacePanel == null) mainInterfacePanel = gameObject;
    mainInterfacePanel.SetActive(true);

    // 3. Ensure Canvas is ScreenSpaceOverlay with top sorting order
    Canvas c = mainInterfacePanel.GetComponent<Canvas>() ?? mainInterfacePanel.GetComponen
tInParent<Canvas>();
    if (c != null)
    {
        c.renderMode = RenderMode.ScreenSpaceOverlay;
        c.overrideSorting = true;
        c.sortingOrder = 500;
    }

    // 4. Force all CanvasGroups to 100% opaque
    CanvasGroup[] cgs = mainInterfacePanel.GetComponentsInChildren<CanvasGroup>(true);
    foreach (var cg in cgs)
    {
        cg.alpha = 1f;
        cg.interactable = true;
        cg.blocksRaycasts = true;
    }

    // 5. Ensure RectTransform fills screen
    RectTransform rt = mainInterfacePanel.GetComponent<RectTransform>();
    if (rt != null)
    {
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
    }
}

```

```

        rt.offsetMin = Vector2.zero;
        rt.offsetMax = Vector2.zero;
    }

    // 6. Ensure UIspace and LowerPanel are active
    Transform uiSpace = mainInterfacePanel.transform.Find("UIspace") ?? mainInterfacePanel
    .transform;
    uiSpace.gameObject.SetActive(true);
    Transform lowerPanel = uiSpace.Find("LowerPanel");
    if (lowerPanel != null) lowerPanel.gameObject.SetActive(true);

    EnsureCanvasScaling();
    SetupPortraitAndAnimator();

    if (dialogueText != null)
    {
        dialogueText.gameObject.SetActive(true);
        dialogueText.enabled = true;
        dialogueText.color = Color.white;
        dialogueText.alpha = 1f;
    }

    if (portrait != null)
    {
        portrait.gameObject.SetActive(true);
        portrait.enabled = true;
        portrait.color = Color.white;
    }

    if (cinematicCoroutine != null) StopCoroutine(cinematicCoroutine);
    cinematicCoroutine = StartCoroutine(CinematicStoryRoutine(lines, onComplete));
}

private IEnumerator CinematicStoryRoutine(CinematicLine[] lines, System.Action onComplete)
{
    try
    {
        // Initial debounce so combat clicks don't skip line 1
        yield return new WaitForSecondsRealtime(0.25f);

        CanvasGroup[] cgs = GetComponentsInChildren<CanvasGroup>(true);
        foreach (var cg in cgs) { cg.alpha = 1f; cg.interactable = true; cg.blocksRaycasts
        = true; }

        SetupPortraitAndAnimator();          if (mainInterfacePanel != null)
        {
            mainInterfacePanel.SetActive(true);
            Canvas c = mainInterfacePanel.GetComponentInParent<Canvas>();
            if (c != null)
            {
                c.sortingOrder = 1000;
                c.renderMode = RenderMode.ScreenSpaceOverlay;
            }
        }

        EnsureCanvasScaling();
        EnsureDefaultPortrait();

        // Hide message input & buttons during cinematic dialogue
        NyxarisUIStyler cineStyler = GetStyler();
        if (cineStyler != null) cineStyler.SetCinematicDialogueMode(true);
        if (messageInput != null) messageInput.gameObject.SetActive(false);

        Transform uiSpace = mainInterfacePanel != null ? mainInterfacePanel.transform.Find
        ("UIspace") : null;
        if (uiSpace != null)
        {
            Transform sendBtn = uiSpace.Find("SendButton") ?? uiSpace.Find("LowerPanel/Sen
            dButton") ?? uiSpace.Find("SubmitButton");
            if (sendBtn != null) sendBtn.gameObject.SetActive(false);
            Transform inGo = uiSpace.Find("MessageInput") ?? uiSpace.Find("LowerPanel/Mess
            ageInput");
            if (inGo != null) inGo.gameObject.SetActive(false);
        }

        // Lock player movement during conversation
    }
}

```

```

move.ExternalMovementLock = true;

for (int i = 0; i < lines.Length; i++)
{
    var line = lines[i];

    // 1. Play Portrait Expression Animation
    if (frameAnimator != null && !string.IsNullOrEmpty(line.animationKey))
    {
        frameAnimator.PlayAnimation(line.animationKey, 24f);
    }
    else
    {
        SetEmotionFallback(line.animationKey);
    }

    // 2. Typewriter line
    if (dialogueText != null) dialogueText.text = "";
    string fullText = line.text;
    int charIndex = 0;
    float charInterval = 0.007f;

    while (charIndex < fullText.Length)
    {
        if (Input.GetKeyDown(KeyCode.E) || Input.GetKeyDown(KeyCode.Space) || Input
t.GetKeyDown(KeyCode.Return) || Input.GetMouseButtonDown(0))
        {
            if (dialogueText != null) dialogueText.text = fullText;
            yield return new WaitForSecondsRealtime(0.12f);
            break;
        }

        if (dialogueText != null) dialogueText.text += fullText[charIndex];
        charIndex++;
        yield return new WaitForSecondsRealtime(charInterval);
    }

    if (dialogueText != null)
    {
        dialogueText.text = fullText + "\n\n<color=#D47BFF><size=70%>??? Press [E]
to continue</size></color>";
    }

    // 3. Wait for Player to press [E] or any key to advance
    yield return new WaitForSecondsRealtime(0.1f);
    float lineWaitTimer = 0f;
    while (lineWaitTimer < 8f)
    {
        lineWaitTimer += Time.unscaledDeltaTime;
        if (Input.GetKeyDown(KeyCode.E) || Input.GetKeyDown(KeyCode.Space) || Inpu
t.GetKeyDown(KeyCode.Return) || Input.GetMouseButtonDown(0))
        {
            break;
        }
        yield return null;
    }
    yield return new WaitForSecondsRealtime(0.08f);
}

}

finally
{
    cinematicCoroutine = null;
    NyxarisUIStyler cineStyler = GetStyler();
    if (cineStyler != null) cineStyler.SetCinematicDialogueMode(false);
    HideInterface();
    move.ExternalMovementLock = false;
    onComplete?.Invoke();
}

}

void OnDisable()
{
    move.ExternalMovementLock = false;
}

```

```

    void OnDestroy()
    {
        move.ExternalMovementLock = false;
    }
}

```

FILE 38: Assets/Scenes/scripts/TutorialCaveWaveManager.cs
PURPOSE: Controls tutorial enemy waves.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using SpawnOfChaos.Systems;

/// <summary>
/// TutorialCaveWaveManager - Manages the 3 combat wave trials inside the Tutorial Cavern Aren
a.
/// Stage 1: Melee 2-hit combo (J key) vs Training Samurai Mobs.
/// Stage 2: Unlocks Magic Projectiles (K key) vs flying Nightmare Orbs.
/// Stage 3: Shift Dash invulnerability (LeftShift) vs hazard barriers & Samurai Mobs.
/// Opens cavern exit gate leading to High Mountain Peak Nyxaris Shrine upon completion.
/// </summary>
public class TutorialCaveWaveManager : MonoBehaviour
{
    public static TutorialCaveWaveManager Instance { get; private set; }

    [Header("Wave Progression State")]
    public int currentWave = 0; // 0 = Inactive, 1 = Melee, 2 = Magic, 3 = Dash, 4 = Complete
    public bool isArenaActive = false;

    [Header("Spawn Points & Portals")]
    public Transform waveSpawnPointLeft;
    public Transform waveSpawnPointRight;
    public GameObject cavernExitGate;
    public GameObject magicPedestalFX;

    [Header("Tutorial UI Banners")]
    public string wave1Message = "?????? COMBAT TRIAL 1: Press 'J' for Melee Slash! Tap 'J' tw
ice for 2-Hit Combo!";
    public string wave2Message = "??? MAGIC UNLOCKED! Press 'K' to fire Magic Projectiles at f
lying targets!";
    public string wave3Message = "??? DASH DODGE: Press 'Shift' while moving for Invulnerable
Dash!";

    private List<GameObject> activeWaveMobs = new List<GameObject>();
    private bool waveTransitioning = false;

    void Awake()
    {
        if (Instance == null) Instance = this;
    }

    void Start()
    {
        if (cavernExitGate != null)
        {
            cavernExitGate.SetActive(true); // Lock exit until waves complete
        }
    }

    public void TriggerCavernArena()
    {
        if (isArenaActive || currentWave > 0) return;
        isArenaActive = true;
        StartCoroutine(StartWave1_Melee());
    }

    void Update()
    {
        if (!isArenaActive || waveTransitioning) return;

        // Check if all active mobs in wave are defeated
        activeWaveMobs.RemoveAll(mob => mob == null || !mob.activeInHierarchy);
    }
}

```

```

        if (activeWaveMobs.Count == 0)
        {
            if (currentWave == 1)
            {
                StartCoroutine(TransitionToWave2_Magic());
            }
            else if (currentWave == 2)
            {
                StartCoroutine(TransitionToWave3_Dash());
            }
            else if (currentWave == 3)
            {
                StartCoroutine(CompleteCavernArena());
            }
        }
    }

private IEnumerator StartWave1_Melee()
{
    waveTransitioning = true;
    currentWave = 1;
    ShowTutorialBanner(wave1Message);
    yield return new WaitForSeconds(1.5f);

    // Spawn 2 Samurai Mobs
    Vector3 posL = waveSpawnPointLeft != null ? waveSpawnPointLeft.position : transform.position + new Vector3(-4f, 0f, 0f);
    Vector3 posR = waveSpawnPointRight != null ? waveSpawnPointRight.position : transform.position + new Vector3(4f, 0f, 0f);

    GameObject mob1 = SpawnSamuraiMob("TutorialSamurai_1", posL);
    GameObject mob2 = SpawnSamuraiMob("TutorialSamurai_2", posR);

    if (mob1 != null) activeWaveMobs.Add(mob1);
    if (mob2 != null) activeWaveMobs.Add(mob2);

    waveTransitioning = false;
}

private IEnumerator TransitionToWave2_Magic()
{
    waveTransitioning = true;
    currentWave = 2;

    // Unlock Magic Projectile capability on MageCombat
    if (MageCombat.Instance != null)
    {
        MageCombat.Instance.UnlockProjectile();
    }

    if (magicPedestalFX != null)
    {
        magicPedestalFX.SetActive(true);
    }

    ShowTutorialBanner(wave2Message);
    yield return new WaitForSeconds(2.0f);

    // Spawn 3 Nightmare Orbs
    Vector3 centerPos = transform.position + Vector3.up * 3f;
    for (int i = 0; i < 3; i++)
    {
        Vector3 orbPos = centerPos + new Vector3((i - 1) * 3.5f, Random.Range(-0.5f, 1f), 0f);
        GameObject orb = SpawnNightmareOrb($"TutorialOrb_{i+1}", orbPos);
        if (orb != null) activeWaveMobs.Add(orb);
    }

    waveTransitioning = false;
}

private IEnumerator TransitionToWave3_Dash()
{
    waveTransitioning = true;
    currentWave = 3;

```

```

        ShowTutorialBanner(wave3Message);
        yield return new WaitForSeconds(2.0f);

        // Spawn 2 Samurai Mobs with higher aggressiveness
        Vector3 posL = waveSpawnPointLeft != null ? waveSpawnPointLeft.position : transform.position + new Vector3(-5f, 0f, 0f);
        Vector3 posR = waveSpawnPointRight != null ? waveSpawnPointRight.position : transform.position + new Vector3(5f, 0f, 0f);

        GameObject mob1 = SpawnSamuraiMob("TutorialSamurai_Dash1", posL);
        GameObject mob2 = SpawnSamuraiMob("TutorialSamurai_Dash2", posR);

        if (mob1 != null) activeWaveMobs.Add(mob1);
        if (mob2 != null) activeWaveMobs.Add(mob2);

        waveTransitioning = false;
    }

    private IEnumerator CompleteCavernArena()
    {
        waveTransitioning = true;
        currentWave = 4;

        ShowTutorialBanner("???? CAVERN ARENA CLEAR! Proceed upward to the High Mountain Peak Shrine!");
        yield return new WaitForSeconds(1.5f);

        // Unlock cavern exit gate
        if (cavernExitGate != null)
        {
            cavernExitGate.SetActive(false);
        }

        // Spawn loot rewards
        OrbSpawner.SpawnLootCluster(transform.position + Vector3.up * 1f, 10);
        waveTransitioning = false;
    }

    private GameObject SpawnSamuraiMob(string name, Vector3 pos)
    {
        GameObject mob = new GameObject(name);
        mob.transform.position = pos;
        mob.tag = "enemy";

        SpriteRenderer sr = mob.AddComponent<SpriteRenderer>();
        sr.color = new Color(0.9f, 0.25f, 0.25f, 1f); // Red samurai tint

        BoxCollider2D col = mob.AddComponent<BoxCollider2D>();
        col.size = new Vector2(1.2f, 2.8f);

        Rigidbody2D rb = mob.AddComponent<Rigidbody2D>();
        rb.gravityScale = 2.5f;
        rb.constraints = RigidbodyConstraints2D.FreezeRotation;

        NormalMaleSamuraiAI ai = mob.AddComponent<NormalMaleSamuraiAI>();

        return mob;
    }

    private GameObject SpawnNightmareOrb(string name, Vector3 pos)
    {
        GameObject orb = new GameObject(name);
        orb.transform.position = pos;
        orb.tag = "enemy";

        SpriteRenderer sr = orb.AddComponent<SpriteRenderer>();
        sr.color = new Color(0.6f, 0.1f, 0.9f, 1f); // Purple orb tint

        CircleCollider2D col = orb.AddComponent<CircleCollider2D>();
        col.radius = 0.8f;

        Rigidbody2D rb = orb.AddComponent<Rigidbody2D>();
        rb.gravityScale = 0f; // Flying orb

        NightmareOrbAI ai = orb.AddComponent<NightmareOrbAI>();
    }

```

```

        return orb;
    }

    private void ShowTutorialBanner(string text)
    {
        Debug.Log($"[TutorialCaveWaveManager] {text}");
        GameObject player = GameObject.FindGameObjectWithTag("Player");
        if (player != null)
        {
            Vector3 pPos = player.transform.position;
            FloatingDamageNumber.SpawnText(pPos + Vector3.up * 2.5f, text, new Color(1f, 0.85f
, 0.3f, 1f));
        }
    }
}

```

FILE 39: Assets/Scenes/scripts/Platforms/PurpleWater2D.cs
PURPOSE: Simulates interactive spring-based water.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace SpawnOfChaos.Props
{
    /// <summary>
    /// PurpleWater2D - Interactive 2D Spring-Mesh Pink Water System.
    /// Simulates real-time Hooke's Law spring node surface wave physics, velocity-based wake
    trails,
    /// entry/exit splash impulses, concentric pink glow rings, and fluid buoyancy drag on sub
    merged entities.
    /// </summary>
    [RequireComponent(typeof(MeshFilter))]
    [RequireComponent(typeof(MeshRenderer))]
    [RequireComponent(typeof(BoxCollider2D))]
    public class PurpleWater2D : MonoBehaviour
    {
        [Header("Water Body Dimensions")]
        public float width = 12f;
        public float depth = 4f;
        [Range(10, 120)]
        public int nodeCount = 50;

        [Header("Spring Surface Physics")]
        public float stiffness = 0.025f;
        public float damping = 0.07f;
        public float spread = 0.04f;

        [Header("Interaction & VFX")]
        public float splashImpulseMultiplier = 0.25f;
        public float wakeRippleMultiplier = 0.05f;

        [Header("Buoyancy & Fluid Drag")]
        public float buoyancyFactor = 14f;
        public float fluidLinearDamping = 3.5f;
        public float fluidAngularDamping = 2f;

        [Header("Color Palette")]
        public Color topSurfaceColor = new Color(1.0f, 0.18f, 0.58f, 0.95f); // Luminous pink
        public Color bottomDeepColor = new Color(0.10f, 0.015f, 0.16f, 0.98f); // Deep plum
        public Color foamGlowColor = new Color(1.0f, 0.66f, 0.92f, 1.0f); // Soft foam highlig
ht
        public Color causticColor = new Color(0.84f, 0.22f, 0.72f, 1.0f); // Moving inner glow
        public Color surfaceHighlightColor = new Color(1.0f, 0.88f, 0.96f, 1.0f); // Liquid sh
een

        public struct SpringNode
        {
            public float y;
            public float velocity;
            public float targetY;
        }

        private SpringNode[] nodes;
        private Mesh mesh;
    }
}

```

```

private MeshFilter meshFilter;
private MeshRenderer meshRenderer;
private BoxCollider2D boxCollider;
private Material waterMaterial;

// Tracks submerged rigidbodies to restore original damping on exit
private Dictionary<Rigidbody2D, float> originalLinearDampings = new Dictionary<Rigidbody2D, float>();
private Dictionary<Rigidbody2D, float> originalAngularDampings = new Dictionary<Rigidbody2D, float>();

void Awake()
{
    EnsureComponents();
    SetupMaterial();
    GenerateMeshAndNodes();
    SetupBoxCollider();
}

private void EnsureComponents()
{
    if (meshFilter == null) meshFilter = GetComponent<MeshFilter>() ?? gameObject.AddComponent<MeshFilter>();
    if (meshRenderer == null) meshRenderer = GetComponent<MeshRenderer>() ?? gameObject.AddComponent<MeshRenderer>();
    if (boxCollider == null) boxCollider = GetComponent<BoxCollider2D>() ?? gameObject.AddComponent<BoxCollider2D>();
}

private void SetupMaterial()
{
    EnsureComponents();
    Shader shader = Shader.Find("Sprites/RealisticPurpleWater");
    if (shader == null)
    {
        shader = Shader.Find("Sprites/Default");
    }

    waterMaterial = new Material(shader);
    waterMaterial.SetColor("_TopColor", topSurfaceColor);
    waterMaterial.SetColor("_BottomColor", bottomDeepColor);
    waterMaterial.SetColor("_FoamColor", foamGlowColor);
    waterMaterial.SetColor("_CausticColor", causticColor);
    waterMaterial.SetColor("_HighlightColor", surfaceHighlightColor);
    waterMaterial.SetFloat("_FoamHeight", 0.045f);
    waterMaterial.SetFloat("_CausticStrength", 0.16f);
    waterMaterial.SetFloat("_SurfaceSheen", 0.3f);

    meshRenderer.material = waterMaterial;
    meshRenderer.sortingLayerName = "Default";
    meshRenderer.sortingOrder = 5;
}

public void GenerateMeshAndNodes()
{
    EnsureComponents();
    nodes = new SpringNode[nodeCount];
    for (int i = 0; i < nodeCount; i++)
    {
        nodes[i] = new SpringNode { y = 0f, velocity = 0f, targetY = 0f };
    }

    mesh = new Mesh();
    mesh.name = "PurpleWaterMesh";

    Vector3[] vertices = new Vector3[nodeCount * 2];
    Vector2[] uvs = new Vector2[nodeCount * 2];
    int[] triangles = new int[(nodeCount - 1) * 6];

    float dx = width / (nodeCount - 1);
    float startX = -width * 0.5f;

    for (int i = 0; i < nodeCount; i++)
    {
        float x = startX + i * dx;
        float u = (float)i / (nodeCount - 1);
    }
}

```

```

        // Top surface vertex
        vertices[i] = new Vector3(x, 0f, 0f);
        uvs[i] = new Vector2(u, 1f);

        // Bottom vertex
        vertices[i + nodeCount] = new Vector3(x, -depth, 0f);
        uvs[i + nodeCount] = new Vector2(u, 0f);
    }

    int t = 0;
    for (int i = 0; i < nodeCount - 1; i++)
    {
        int topL = i;
        int topR = i + 1;
        int botL = i + nodeCount;
        int botR = i + nodeCount + 1;

        triangles[t++] = topL;
        triangles[t++] = topR;
        triangles[t++] = botL;

        triangles[t++] = topR;
        triangles[t++] = botR;
        triangles[t++] = botL;
    }

    mesh.vertices = vertices;
    mesh.uv = uvs;
    mesh.triangles = triangles;
    mesh.RecalculateBounds();

    meshFilter.mesh = mesh;
}

private void SetupBoxCollider()
{
    EnsureComponents();
    if (boxCollider != null)
    {
        boxCollider.isTrigger = true;
        boxCollider.size = new Vector2(width, depth);
        boxCollider.offset = new Vector2(0f, -depth * 0.5f);
    }
}

void Update()
{
    UpdateSpringNodes(Time.deltaTime);
    UpdateMeshVertices();
}

private float animTime = 0f;

private void UpdateSpringNodes(float deltaTime)
{
    if (nodes == null || nodes.Length == 0) return;
    animTime += deltaTime;

    // 1. Hooke's Law Spring Update with Multi-Harmonic Swells
    for (int i = 0; i < nodeCount; i++)
    {
        // Multi-frequency natural water swells
        float ambientSwell = Mathf.Sin(animTime * 2.2f + i * 0.15f) * 0.08f
            + Mathf.Sin(animTime * 3.6f - i * 0.09f) * 0.04f
            + Mathf.Cos(animTime * 1.4f + i * 0.05f) * 0.03f;

        nodes[i].targetY = ambientSwell;

        float dy = nodes[i].y - nodes[i].targetY;
        float accel = -stiffness * dy - damping * nodes[i].velocity;
        nodes[i].velocity += accel;
        nodes[i].velocity = Mathf.Clamp(nodes[i].velocity, -0.25f, 0.25f);
        nodes[i].y += nodes[i].velocity;
        nodes[i].y = Mathf.Clamp(nodes[i].y, -0.4f, 0.4f);
    }
}

```

```

// 2. Wave Propagation Spread Pass (8 iterations for smooth liquid propagation)
float[] leftDeltas = new float[nodeCount];
float[] rightDeltas = new float[nodeCount];

for (int pass = 0; pass < 8; pass++)
{
    for (int i = 0; i < nodeCount; i++)
    {
        if (i > 0)
        {
            leftDeltas[i] = spread * (nodes[i].y - nodes[i - 1].y);
            nodes[i - 1].velocity += leftDeltas[i];
        }
        if (i < nodeCount - 1)
        {
            rightDeltas[i] = spread * (nodes[i].y - nodes[i + 1].y);
            nodes[i + 1].velocity += rightDeltas[i];
        }
    }

    for (int i = 0; i < nodeCount; i++)
    {
        if (i > 0) nodes[i - 1].y += leftDeltas[i];
        if (i < nodeCount - 1) nodes[i + 1].y += rightDeltas[i];
    }
}

private void UpdateMeshVertices()
{
    if (mesh == null || nodes == null) return;

    Vector3[] vertices = mesh.vertices;

    // Apply Catmull-Rom Spline Curve Smoothing across adjacent spring nodes
    for (int i = 0; i < nodeCount; i++)
    {
        int p0 = Mathf.Clamp(i - 1, 0, nodeCount - 1);
        int p1 = i;
        int p2 = Mathf.Clamp(i + 1, 0, nodeCount - 1);
        int p3 = Mathf.Clamp(i + 2, 0, nodeCount - 1);

        // Smooth cubic spline filtering
        float smoothY = 0.5f * (
            (2f * nodes[p1].y) +
            (-nodes[p0].y + nodes[p2].y) * 0.5f +
            (2f * nodes[p0].y - 5f * nodes[p1].y + 4f * nodes[p2].y - nodes[p3].y) * 0
        .25f
        );

        vertices[i].y = smoothY;
    }

    mesh.vertices = vertices;
    mesh.RecalculateBounds();
}

public void SplashAtWorldPosition(Vector3 worldPos, float impulse)
{
    float clampedImpulse = Mathf.Clamp(impulse, -0.3f, 0.3f);
    int index = WorldPositionToNodeIndex(worldPos);
    if (index >= 0 && index < nodeCount)
    {
        nodes[index].velocity += clampedImpulse;

        // Splash nearby nodes proportionally
        if (index > 0) nodes[index - 1].velocity += clampedImpulse * 0.4f;
        if (index < nodeCount - 1) nodes[index + 1].velocity += clampedImpulse * 0.4f;
    }
}

private int WorldPositionToNodeIndex(Vector3 worldPos)
{
    float localX = transform.InverseTransformPoint(worldPos).x;
    float normalizedX = Mathf.InverseLerp(-width * 0.5f, width * 0.5f, localX);
}

```

```

        return Mathf.Clamp(Mathf.RoundToInt(normalizedX * (nodeCount - 1)), 0, nodeCount -
1);
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        Rigidbody2D rb = other.attachedRigidbody;
        if (rb == null) return;

        float impactSpeed = Mathf.Max(rb.linearVelocity.magnitude, 3.5f);
        float vy = rb.linearVelocity.y != 0 ? rb.linearVelocity.y : -3.5f;
        SplashAtWorldPosition(other.transform.position, vy * splashImpulseMultiplier);

        // Trigger Purple Droplet VFX & Concentric Glow Rings
        Vector3 splashPos = new Vector3(other.transform.position.x, transform.position.y,
transform.position.z);
        PurpleWaterSplashFX.SpawnSplash(splashPos, impactSpeed, topSurfaceColor, foamGlowC
olor);

        // Store original drag values
        if (!originalLinearDampings.ContainsKey(rb))
        {
            originalLinearDampings[rb] = rb.linearDamping;
            originalAngularDampings[rb] = rb.angularDamping;
        }

        rb.linearDamping = fluidLinearDamping;
        rb.angularDamping = fluidAngularDamping;
    }

    private void OnTriggerStay2D(Collider2D other)
    {
        Rigidbody2D rb = other.attachedRigidbody;
        if (rb == null) return;

        // Velocity-based wake trail ripples (gentle, subtle ripple)
        if (Mathf.Abs(rb.linearVelocity.x) > 0.1f)
        {
            float wakeImpulse = -Mathf.Sign(rb.linearVelocity.x) * Mathf.Min(Mathf.Abs(rb.
linearVelocity.x), 6f) * wakeRippleMultiplier;
            SplashAtWorldPosition(other.transform.position, wakeImpulse * Time.deltaTime *
3f);
        }

        // Upward Buoyancy Force
        float waterSurfaceY = transform.position.y;
        float subDepth = Mathf.Clamp(waterSurfaceY - rb.position.y, 0f, depth);
        if (subDepth > 0f)
        {
            float buoyancy = subDepth * buoyancyFactor;
            rb.AddForce(Vector2.up * buoyancy, ForceMode2D.Force);
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        Rigidbody2D rb = other.attachedRigidbody;
        if (rb == null) return;

        float impactSpeed = Mathf.Max(rb.linearVelocity.magnitude, 3.5f);
        float vy = rb.linearVelocity.y != 0 ? rb.linearVelocity.y : 3.5f;
        SplashAtWorldPosition(other.transform.position, vy * splashImpulseMultiplier);

        Vector3 exitSplashPos = new Vector3(other.transform.position.x, transform.position
.y, transform.position.z);
        PurpleWaterSplashFX.SpawnSplash(exitSplashPos, impactSpeed, topSurfaceColor, foamG
lowColor);

        // Restore original drag values
        if (originalLinearDampings.TryGetValue(rb, out float origLinear))
        {
            rb.linearDamping = origLinear;
            originalLinearDampings.Remove(rb);
        }
        if (originalAngularDampings.TryGetValue(rb, out float origAngular))
        {

```

```

        rb.angularDamping = origAngular;
        originalAngularDampings.Remove(rb);
    }
}
}

```

FILE 40: Assets/Scenes/scripts/Systems/TutorialLorePrologue.cs
PURPOSE: Runs the seven-slide opening lore sequence and narration.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
#if UNITY_EDITOR
using UnityEditor;
#endif

/// <summary>
/// TutorialLorePrologue - Atmospheric opening lore sequence for the Tutorial level.
/// Features:
/// - Living cosmic void backdrop with gentle procedural floating purple orbs & nebula glow.
/// - Prominent, regal Ancient Title typography with amethyst glow.
/// - Smooth cinematic slide Fade-In and Fade-Out (replaces typewriter effect).
/// - Floating magical text aura with breathing shimmer.
/// - Interactive advance ([Space], [Enter], Click) and skip ([Esc]).
/// - Live font style cycling ([F1]: Gothic / Papyrus / Serif).
/// - Asynchronous companion frame pre-warming in background.
/// </summary>
public class TutorialLorePrologue : MonoBehaviour
{
    public static TutorialLorePrologue Instance { get; private set; }
    public static bool IsPrologueActive { get; private set; }

    public enum AncientFontStyle
    {
        AncientGothic,    // Old English Text MT / Dark fantasy medieval tome
        AncientPapyrus,   // Papyrus / Weathered mythic scroll
        AncientSerif      // Georgia / Classical mythic inscriptional serif
    }

    [Header("Ancient Font Settings")]
    public AncientFontStyle fontStyle = AncientFontStyle.AncientGothic;
    public Font customFont;

    [Header("Audio (Assign custom music track when ready)")]
    public AudioSource audioSource;
    public AudioClip prologueMusicTrack;
    [Range(0f, 1f)] public float musicVolume = 0.8f;

    [Header("Voiceover Narration")]
    public AudioSource narrationSource;
    public AudioClip[] narrationClips = new AudioClip[7];
    [Range(0f, 1f)] public float narrationVolume = 1f;
    [Range(0f, 3f)] public float firstNarrationDelay = 0.7f;

    [Header("Cinematic Timing Settings")]
    public float slideFadeInDuration = 0.8f;
    public float slideFadeOutDuration = 0.45f;
    public float sceneAwakenDuration = 1.4f;

    // Narrative Slides (Atmospheric & Captivating)
    private readonly string[] loreSlides = new string[]
    {
        "Before time had a heartbeat, there was only the Void.\n\nAn infinite, formless stilln  

ess... commonly known as Chaos.",
        "For uncounted eons, darkness reigned alone.\n\nUntil a spark of radiant defiance igni  

ted within the silence: Aether, the First Light.",
        "They were absolute opposites, yet in their collision, balance was born.\n\nChaos gave  

substance to the dark; Aether gave warmth and form to reality.",
        "From their union sprang the multiverse, and the children destined to rule it.\n\nFore  

most among them was Nyxaris, Goddess of Darkness, Heir to Chaos.",
        "For an age, existence held steady between shadow and dawn.\n\nUntil the day the light  

fractured.",
    }
}

```

```

        "Without warning or whisper... Aether disappeared.\n\nWith her absence, the cosmic equilibrium unraveled.",
        "Bereft of the light that bound them, realm after realm began to fall into the abyss.\n\nNow, from the remnants of a collapsing multiverse, the Void stirs once more...",
        "<size=56><color=#F2B8FF><b>THE SPAWN OF CHAOS</b></color></size>\n\n<size=30><color=#D47BFF><i>??? Heir to the Void ???</i></color></size>"
    };

    // UI References
    private Canvas prologueCanvas;
    private CanvasGroup canvasGroup;
    private CanvasGroup slideCanvasGroup;
    private RectTransform slideContainerRT;
    private Text headerTextComp;
    private Text loreTextComp;
    private Text slideProgressTextComp;
    private Text promptTextComp;
    private Text skipTextComp;
    private Text fontHintTextComp;
    private Shadow loreShadow;
    private Outline loreOutline;
    private Image nebulaGlowImg;

    // Floating Purple Orbs System
    private class FloatingOrb
    {
        public RectTransform rt;
        public Image img;
        public float speed;
        public float xCenter;
        public float swayFreq;
        public float swayAmp;
        public float phase;
        public float baseAlpha;
        public Color baseColor;
    }
    private List<FloatingOrb> orbs = new List<FloatingOrb>();
    private Sprite softOrbSprite;

    // State
    private int currentSlideIndex = 0;
    private bool isTransitioning = false;
    private bool hasFinished = false;
    private Coroutine currentTransitionCoroutine;
    private float baseLoreY = -15f;
    private List<AudioSource> mutedSceneAudioSources = new List<AudioSource>();
    private List<float> mutedSceneAudioVolumes = new List<float>();

    void Awake()
    {
        Instance = this;
        IsPrologueActive = true;

        // Ensure Lumi (the light orb) NEVER exists in TutorialScene
        PurgeLumiInstances();

        // Immediately lock player movement and hide gameplay
        move.ExternalMovementLock = true;

        softOrbSprite = CreateSoftRadialGlowSprite();
        BuildPrologueUI();
    }

    private void PurgeLumiInstances()
    {
        if (LightOrbCompanion.Instance != null)
        {
            Destroy(LightOrbCompanion.Instance.gameObject);
        }
        var allLumi = Object.FindObjectsByType<LightOrbCompanion>(FindObjectsSortMode.None);
        foreach (var l in allLumi)
        {
            if (l != null) Destroy(l.gameObject);
        }
        var allRoots = UnityEngine.SceneManagement.SceneManager.GetActiveScene().GetRootGameObjects();
    }

```

```

        foreach (var rootGO in allRoots)
        {
            if (rootGO != null && (rootGO.name.Contains("Lumi") || rootGO.name.Contains("Light
OrbCompanion")))
            {
                Destroy(rootGO);
            }
        }
    }

    void Start()
    {
        PurgeLumiInstances();

        // Pre-warm FoxNyxaris frames asynchronously in background during lore display
        StartCoroutine(FoxNyxarisController.PreloadFramesRoutine());

        // Ensure Nyxaris Seal Breakdown sequence is active in TutorialScene
        NyxarisSealSequence.EnsureInstanceInScene();

        EnsureNarrationSource();
        EnsureNarrationClipsAssigned();
        SilenceSceneAudioDuringLore();

        // Play optional music track if provided
        if (prologueMusicTrack != null)
        {
            if (audioSource == null) audioSource = gameObject.AddComponent<AudioSource>();
            audioSource.clip = prologueMusicTrack;
            audioSource.volume = musicVolume;
            audioSource.loop = true;
            audioSource.Play();
        }

        // Begin with first slide fading in
        currentTransitionCoroutine = StartCoroutine(ShowSlideRoutine(0));
    }

    private void EnsureNarrationSource()
    {
        if (narrationSource == null)
        {
            narrationSource = GetComponent<AudioSource>();
            if (narrationSource == null)
            {
                narrationSource = gameObject.AddComponent<AudioSource>();
            }
        }

        narrationSource.loop = false;
        narrationSource.playOnAwake = false;
        narrationSource.volume = narrationVolume;

        ApplyNarrationReverb();
    }

    private void ApplyNarrationReverb()
    {
        if (narrationSource == null)
        {
            return;
        }

        try
        {
            var filterType = System.Type.GetType("UnityEngine.AudioReverbFilter, UnityEngine");

            if (filterType == null)
            {
                narrationSource.reverbZoneMix = 1.2f;
                return;
            }

            var filter = narrationSource.GetComponent(filterType);
            if (filter == null)
            {

```

```

        filter = narrationSource.gameObject.AddComponent(filterType);
    }

    var presetType = System.Type.GetType("UnityEngine.AudioReverbPreset, UnityEngine");
;
    if (presetType != null)
    {
        var preset = System.Enum.Parse(presetType, "Cathedral");
        var presetProp = filterType.GetProperty("reverbPreset");
        if (presetProp != null && presetProp.PropertyType == presetType)
        {
            presetProp.SetValue(filter, preset);
        }
    }

    SetFloatProperty(filter, "dryLevel", 0f);
    SetFloatProperty(filter, "room", -1000f);
    SetFloatProperty(filter, "roomHF", -1000f);
    SetFloatProperty(filter, "decayTime", 4.8f);
    SetFloatProperty(filter, "decayHFRatio", 1.3f);
    SetFloatProperty(filter, "reflections", 80f);
    SetFloatProperty(filter, "reflectionsDelay", 0.03f);
    SetFloatProperty(filter, "reverb", 2000f);
    SetFloatProperty(filter, "reverbDelay", 0.06f);
    SetFloatProperty(filter, "hfReference", 5000f);
    SetFloatProperty(filter, "lfReference", 250f);
    SetFloatProperty(filter, "diffusion", 100f);
    SetFloatProperty(filter, "density", 100f);

    var enabledProp = filterType.GetProperty("enabled");
    if (enabledProp != null && enabledProp.PropertyType == typeof(bool))
    {
        enabledProp.SetValue(filter, true);
    }
}
catch (System.Exception ex)
{
    Debug.LogWarning("[TutorialLorePrologue] Reverb setup unavailable in this Unity bu
ild: " + ex.Message);
    narrationSource.reverbZoneMix = 1.2f;
}

private static void SetFloatProperty(Component target, string propertyName, float value)
{
    if (target == null)
    {
        return;
    }

    var prop = target.GetType().GetProperty(propertyName);
    if (prop != null && prop.PropertyType == typeof(float))
    {
        prop.SetValue(target, value);
    }
}

private void EnsureNarrationClipsAssigned()
{
    bool hasAssignedClip = narrationClips != null && narrationClips.Length >= 7 && narrati
onClips[0] != null;
    if (hasAssignedClip)
    {
        return;
    }

    narrationClips = new AudioClip[7];
}

#if UNITY_EDITOR
string[] assetPaths = new[]
{
    "Assets/Audio/Voice/openingsequence/prologue_slide_01.wav",
    "Assets/Audio/Voice/openingsequence/prologue_slide_02.wav",
    "Assets/Audio/Voice/openingsequence/prologue_slide_03.wav",
    "Assets/Audio/Voice/openingsequence/prologue_slide_04.wav",
    "Assets/Audio/Voice/openingsequence/prologue_slide_05.wav",
}

```

```

        "Assets/Audio/Voice/openingsequence/prologue_slide_06.wav",
        "Assets/Audio/Voice/openingsequence/prologue_slide_07.wav"
    };

    for (int i = 0; i < assetPaths.Length; i++)
    {
        narrationClips[i] = AssetDatabase.LoadAssetAtPath<AudioClip>(assetPaths[i]);
        if (narrationClips[i] == null)
        {
            Debug.LogWarning($"[TutorialLorePrologue] Missing narration clip at path: {assetPaths[i]}");
        }
    }
#endif

    if (narrationClips[0] == null && narrationClips[1] == null && narrationClips[2] == null && narrationClips[3] == null && narrationClips[4] == null && narrationClips[5] == null && narrationClips[6] == null)
    {
        Debug.LogWarning("[TutorialLorePrologue] No prologue narration clips were assigned. Ensure the seven openingsequence wav files exist under Assets/Audio/Voice/openingsequence.");
    }
}

private void SilenceSceneAudioDuringLore()
{
    if (mutedSceneAudioSources.Count > 0)
    {
        return;
    }

    AudioSource[] allSources = FindObjectsByType<AudioSource>(FindObjectsSortMode.None);
    for (int i = 0; i < allSources.Length; i++)
    {
        var source = allSources[i];
        if (source == null || source == audioSource || source == narrationSource)
        {
            continue;
        }

        mutedSceneAudioSources.Add(source);
        mutedSceneAudioVolumes.Add(source.volume);
        source.mute = true;
        source.Stop();
    }
}

private void RestoreSceneAudioAfterLore()
{
    for (int i = 0; i < mutedSceneAudioSources.Count; i++)
    {
        var source = mutedSceneAudioSources[i];
        if (source != null)
        {
            source.mute = false;
            source.volume = mutedSceneAudioVolumes[i];
        }
    }

    mutedSceneAudioSources.Clear();
    mutedSceneAudioVolumes.Clear();
}

private void PlayNarrationForSlide(int slideIndex)
{
    if (narrationSource == null)
    {
        EnsureNarrationSource();
    }

    if (slideIndex < 0 || slideIndex >= 7 || narrationClips == null || slideIndex >= narrationClips.Length || narrationClips[slideIndex] == null)
    {
        if (narrationSource != null)
        {

```

```

        narrationSource.Stop();
    }
    Debug.LogWarning($"[TutorialLorePrologue] No narration clip available for slide {s
lideIndex}");
    return;
}

if (narrationSource.clip == narrationClips[slideIndex] && narrationSource.isPlaying)
{
    return;
}

narrationSource.Stop();
narrationSource.clip = narrationClips[slideIndex];
narrationSource.volume = narrationVolume;
narrationSource.Play();
}

private IEnumerator DelayNarrationForFirstSlide()
{
    if (firstNarrationDelay > 0f)
    {
        yield return new WaitForSeconds(firstNarrationDelay);
    }

    PlayNarrationForSlide(0);
}

void Update()
{
    if (hasFinished) return;

    UpdateFloatingOrbs();
    UpdateMagicalTextAtmosphere();

    // Press [F1] to cycle font styles in real-time
    if (Input.GetKeyDown(KeyCode.F1))
    {
        CycleFontStyle();
    }

    // Skip entire prologue with [Escape]
    if (Input.GetKeyDown(KeyCode.Escape))
    {
        SkipPrologue();
        return;
    }

    // Advance slide on [Space], [Enter], [E], or Left Click
    if (Input.GetKeyDown(KeyCode.Space) || Input.GetKeyDown(KeyCode.Return) || Input.GetKe
yDown(KeyCode.E) || Input.GetMouseButtonDown(0))
    {
        OnPlayerAdvanceInput();
    }
}

private void OnPlayerAdvanceInput()
{
    if (isTransitioning)
    {
        // Fast-forward transition to fully visible
        if (slideCanvasGroup != null && slideCanvasGroup.alpha < 0.95f)
        {
            if (currentTransitionCoroutine != null) StopCoroutine(currentTransitionCorouti
ne);
            slideCanvasGroup.alpha = 1f;
            if (slideContainerRT != null) slideContainerRT.anchoredPosition = new Vector2(
0f, 0f);
            UpdatePromptState(true);
            isTransitioning = false;
        }
        return;
    }

    currentSlideIndex++;
    if (currentSlideIndex < loreSlides.Length)

```

```

        {
            currentTransitionCoroutine = StartCoroutine(TransitionToNextSlideRoutine(currentSlideIndex));
        }
        else
        {
            CompletePrologue();
        }
    }

private IEnumerator ShowSlideRoutine(int index)
{
    isTransitioning = true;
    UpdatePromptState(false);

    bool isTitleCard = (index == loreSlides.Length - 1);

    // Populate texts
    if (isTitleCard)
    {
        loreTextComp.text = "<size=62><color=#F2B8FF><b>THE SPAWN OF CHAOS</b></color></size>\n\n<size=34><color=#D47BFF><i>??? Heir to the Void ???</i></color></size>";
        loreTextComp.fontSize = 58;
        headerTextComp.text = "??? The Multiverse Fractures ???";
        slideProgressTextComp.text = "??? TITLE CARD ???";
    }
    else
    {
        loreTextComp.text = loreSlides[index];
        loreTextComp.fontSize = 42;
        headerTextComp.text = "??? Chronicles of the Void ???";
        slideProgressTextComp.text = $"??? Part {ToRomanNumeral(index + 1)} ???";
    }

    if (index == 0)
    {
        if (narrationSource != null)
        {
            narrationSource.Stop();
            narrationSource.clip = null;
        }
        StartCoroutine(DelayNarrationForFirstSlide());
    }
    else if (index < 7)
    {
        PlayNarrationForSlide(index);
    }
    else if (narrationSource != null)
    {
        narrationSource.Stop();
    }

    // Fade In smoothly from alpha 0 to 1 with gentle upward floating rise
    float elapsed = 0f;
    Vector2 startPos = new Vector2(0f, -25f);
    Vector2 endPos = Vector2.zero;

    slideCanvasGroup.alpha = 0f;
    slideContainerRT.anchoredPosition = startPos;

    while (elapsed < slideFadeInDuration)
    {
        elapsed += Time.deltaTime;
        float t = Mathf.SmoothStep(0f, 1f, elapsed / slideFadeInDuration);
        slideCanvasGroup.alpha = t;
        slideContainerRT.anchoredPosition = Vector2.Lerp(startPos, endPos, t);
        yield return null;
    }

    slideCanvasGroup.alpha = 1f;
    slideContainerRT.anchoredPosition = endPos;
    isTransitioning = false;

    // Softly reveal continue prompt
    UpdatePromptState(true);
}

```

```

private IEnumerator TransitionToNextSlideRoutine(int targetIndex)
{
    isTransitioning = true;
    UpdatePromptState(false);

    // Fade Out current slide
    float elapsed = 0f;
    float startAlpha = slideCanvasGroup.alpha;
    Vector2 currentPos = slideContainerRT.anchoredPosition;
    Vector2 fadeOutTargetPos = new Vector2(0f, 15f);

    while (elapsed < slideFadeOutDuration)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / slideFadeOutDuration;
        slideCanvasGroup.alpha = Mathf.Lerp(startAlpha, 0f, t);
        slideContainerRT.anchoredPosition = Vector2.Lerp(currentPos, fadeOutTargetPos, t);
        yield return null;
    }

    slideCanvasGroup.alpha = 0f;

    // Fade In new slide
    yield return StartCoroutine(ShowSlideRoutine(targetIndex));
}

private void UpdateMagicalTextAtmosphere()
{
    // Gentle undulating float on the lore text
    if (loreTextComp != null)
    {
        RectTransform rt = loreTextComp.GetComponent<RectTransform>();
        if (rt != null)
        {
            float floatOffset = Mathf.Sin(Time.time * 1.6f) * 4.5f;
            rt.anchoredPosition = new Vector2(0f, baseLoreY + floatOffset);
        }
    }

    // Shimmering magical glow aura on outline & shadow
    float pulse = (Mathf.Sin(Time.time * 2.2f) + 1f) * 0.5f;
    if (loreShadow != null)
    {
        loreShadow.effectColor = Color.Lerp(new Color(0.35f, 0.08f, 0.60f, 0.70f), new Color(0.70f, 0.25f, 1.0f, 0.95f), pulse);
    }
    if (loreOutline != null)
    {
        loreOutline.effectColor = Color.Lerp(new Color(0.18f, 0.04f, 0.32f, 0.85f), new Color(0.45f, 0.15f, 0.70f, 0.95f), pulse);
    }

    // Soft breathing on central nebula glow
    if (nebulaGlowImg != null)
    {
        float nebulaPulse = (Mathf.Sin(Time.time * 0.8f) + 1f) * 0.5f;
        nebulaGlowImg.color = new Color(0.55f, 0.15f, 0.85f, Mathf.Lerp(0.12f, 0.24f, nebulaPulse));
    }
}

private void UpdateFloatingOrbs()
{
    float dt = Time.deltaTime;
    float time = Time.time;

    for (int i = 0; i < orbs.Count; i++)
    {
        FloatingOrb orb = orbs[i];
        Vector2 pos = orb.rt.anchoredPosition;

        // Move gently upward
        pos.y += orb.speed * dt;

        // Sinusoidal horizontal sway

```

```

        pos.x = orb.xCenter + Mathf.Sin(time * orb.swayFreq + orb.phase) * orb.swayAmp;

        // Wrap around when reaching top of screen
        if (pos.y > 620f)
        {
            pos.y = -620f;
            orb.xCenter = Random.Range(-950f, 950f);
            pos.x = orb.xCenter;
        }

        orb.rt.anchoredPosition = pos;

        // Gentle alpha breathing
        float pulse = 0.7f + 0.3f * Mathf.Sin(time * 1.8f + orb.phase);
        orb.img.color = new Color(orb.baseColor.r, orb.baseColor.g, orb.baseColor.b, orb.b
aseAlpha * pulse);
    }
}

private void UpdatePromptState(bool readyToAdvance)
{
    if (promptTextComp != null)
    {
        bool isTitleCard = (currentSlideIndex == loreSlides.Length - 1);
        if (readyToAdvance)
        {
            promptTextComp.text = isTitleCard
                ? "<color=#F0A0FF>??? Press [Space] or Click to Enter the Realm</color>"
                : "<color=#D47BFF>??? Press [Space] or Click to Continue</color>";
        }
        else
        {
            promptTextComp.text = "";
        }
    }
}

private void SkipPrologue()
{
    if (hasFinished) return;
    Debug.Log("<color=#D47BFF>[TutorialLorePrologue] Prologue skipped by player.</color>");
;
    CompletePrologue();
}

private void CompletePrologue()
{
    if (hasFinished) return;
    hasFinished = true;
    IsPrologueActive = false;

    if (currentTransitionCoroutine != null) StopCoroutine(currentTransitionCoroutine);
    StartCoroutine(FadeOutAndAwakenWorld());
}

private IEnumerator FadeOutAndAwakenWorld()
{
    Debug.Log("<color=#55FF88>[TutorialLorePrologue] Opening lore concluded. Transitioning
to Tutorial Scene...</color>");

    float elapsed = 0f;
    float startMusicVol = audioSource != null ? audioSource.volume : 0f;

    while (elapsed < sceneAwakenDuration)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / sceneAwakenDuration;
        if (canvasGroup != null) canvasGroup.alpha = 1f - t;
        if (audioSource != null) audioSource.volume = Mathf.Lerp(startMusicVol, 0f, t);
        yield return null;
    }

    if (canvasGroup != null) canvasGroup.alpha = 0f;
    if (audioSource != null) audioSource.Stop();
    if (narrationSource != null)
    {

```

```

        narrationSource.Stop();
        narrationSource.clip = null;
    }
    RestoreSceneAudioAfterLore();
    IsPrologueActive = false;

    // Release player movement lock
    move.ExternalMovementLock = false;

    // Cleanup prologue canvas
    if (prologueCanvas != null)
    {
        Destroy(prologueCanvas.gameObject);
    }

    Destroy(gameObject);
}

public void CycleFontStyle()
{
    int next = ((int)fontStyle + 1) % 3;
    fontStyle = (AncientFontStyle)next;
    Font f = GetActiveFont();
    ApplyFontToAllComponents(f);
    if (fontHintTextComp != null) fontHintTextComp.text = $"[F1] Font: {fontStyle}";
    Debug.Log($"<color=#D47BFF>[TutorialLorePrologue] Switched font style to: {fontStyle}
(Loaded: {f?.name})</color>");
}

private void ApplyFontToAllComponents(Font f)
{
    if (f == null) return;
    if (headerTextComp != null) headerTextComp.font = f;
    if (slideProgressTextComp != null) slideProgressTextComp.font = f;
    if (loreTextComp != null) loreTextComp.font = f;
    if (promptTextComp != null) promptTextComp.font = f;
    if (skipTextComp != null) skipTextComp.font = f;
    if (fontHintTextComp != null) fontHintTextComp.font = f;
}

public Font GetActiveFont()
{
    if (customFont != null) return customFont;

    Font f = null;
    switch (fontStyle)
    {
        case AncientFontStyle.AncientGothic:
            f = Resources.Load<Font>("Fonts/AncientGothic");
            if (f == null) f = Font.CreateDynamicFontFromOSFont("Old English Text MT", 42);
            break;
        case AncientFontStyle.AncientPapyrus:
            f = Resources.Load<Font>("Fonts/AncientPapyrus");
            if (f == null) f = Font.CreateDynamicFontFromOSFont("Papyrus", 42);
            break;
        case AncientFontStyle.AncientSerif:
            f = Resources.Load<Font>("Fonts/AncientSerif");
            if (f == null) f = Font.CreateDynamicFontFromOSFont("Georgia", 42);
            break;
    }

    if (f == null) f = Resources.Load<Font>("Fonts/AncientGothic");
    if (f == null) f = Resources.Load<Font>("Fonts/AncientPapyrus");
    if (f == null) f = Resources.Load<Font>("Fonts/AncientSerif");
    if (f == null) f = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");

    return f;
}

private string ToRomanNumeral(int number)
{
    string[] roman = { "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX", "X" };
    if (number >= 1 && number <= roman.Length) return roman[number - 1];
    return number.ToString();
}

```

#region Procedural UI Canvas Builder & Floating Orbs

```
private Sprite CreateSoftRadialGlowSprite()
{
    int size = 256;
    Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.wrapMode = TextureWrapMode.Clamp;
    tex.filterMode = FilterMode.Bilinear;
    Vector2 center = new Vector2((size - 1) * 0.5f, (size - 1) * 0.5f);
    float maxRadius = size * 0.49f;

    for (int y = 0; y < size; y++)
    {
        for (int x = 0; x < size; x++)
        {
            float dist = Vector2.Distance(new Vector2(x, y), center) / maxRadius;
            if (dist >= 1.0f)
            {
                tex.SetPixel(x, y, Color.clear);
                continue;
            }

            // Pure out-of-focus optical bokeh Gaussian blur with smooth zero-falloff edge
            float edgeFactor = Mathf.Clamp01(1.0f - dist);
            float smoothEdge = edgeFactor * edgeFactor * (3.0f - 2.0f * edgeFactor);
            // Ultra-diffuse wide Gaussian curve - soft, misty, dreamy ethereal glow
            float alpha = Mathf.Exp(-1.9f * dist * dist) * smoothEdge;
            tex.SetPixel(x, y, new Color(1f, 1f, 1f, alpha));
        }
    }
    tex.Apply();
    return Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f));
}
```

```
private void BuildPrologueUI()
{
    GameObject canvasGO = new GameObject("[Tutorial_LorePrologueCanvas]");
    prologueCanvas = canvasGO.AddComponent<Canvas>();
    prologueCanvas.renderMode = RenderMode.ScreenSpaceOverlay;
    prologueCanvas.sortingOrder = 9999;

    var scaler = canvasGO.AddComponent<CanvasScaler>();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(1920, 1080);
    scaler.matchWidthOrHeight = 0.5f;

    canvasGroup = canvasGO.AddComponent<CanvasGroup>();
    canvasGroup.alpha = 1.0f;

    // Solid Pitch Black Backdrop
    GameObject bgGO = new GameObject("PitchBlackBackdrop");
    bgGO.transform.SetParent(canvasGO.transform, false);
    Image bgImg = bgGO.AddComponent<Image>();
    bgImg.color = Color.black;
    RectTransform bgRT = bgGO.GetComponent<RectTransform>();
    bgRT.anchorMin = Vector2.zero;
    bgRT.anchorMax = Vector2.one;
    bgRT.sizeDelta = Vector2.zero;

    // Central Cosmic Nebula Glow
    GameObject nebulaGO = new GameObject("CosmicNebulaGlow");
    nebulaGO.transform.SetParent(bgGO.transform, false);
    nebulaGlowImg = nebulaGO.AddComponent<Image>();
    nebulaGlowImg.sprite = softOrbSprite;
    nebulaGlowImg.color = new Color(0.55f, 0.15f, 0.85f, 0.18f);
    RectTransform nebulaRT = nebulaGO.GetComponent<RectTransform>();
    nebulaRT.anchorMin = new Vector2(0.5f, 0.5f);
    nebulaRT.anchorMax = new Vector2(0.5f, 0.5f);
    nebulaRT.sizeDelta = new Vector2(1450f, 850f);

    // Floating Purple Orbs Container (Placed behind text)
    GameObject orbsContainer = new GameObject("FloatingOrbsContainer");
    orbsContainer.transform.SetParent(bgGO.transform, false);
    RectTransform orbsContainerRT = orbsContainer.AddComponent<RectTransform>();
    orbsContainerRT.anchorMin = Vector2.zero;
```

```

orbsContainerRT.anchorMax = Vector2.one;
orbsContainerRT.sizeDelta = Vector2.zero;

SpawnFloatingOrbs(orbsContainer.transform);

// Resolve Active Ancient Font
Font activeFont = GetActiveFont();

// Main Content Container
GameObject containerGO = new GameObject("PrologueContainer");
containerGO.transform.SetParent(bgGO.transform, false);
RectTransform containerRT = containerGO.AddComponent<RectTransform>();
containerRT.anchorMin = new Vector2(0.5f, 0.5f);
containerRT.anchorMax = new Vector2(0.5f, 0.5f);
containerRT.sizeDelta = new Vector2(1350f, 700f);

// Header Title (Large, Regal, Highly Visible with Amethyst Glow)
GameObject headerGO = new GameObject("HeaderTitle");
headerGO.transform.SetParent(containerGO.transform, false);
headerTextComp = headerGO.AddComponent<Text>();
headerTextComp.font = activeFont;
// In gothic scripts, Title Case is significantly more readable and stately than cramp
ed all-caps
headerTextComp.text = "??? Chronicles of the Void ???";
headerTextComp.fontSize = 46; // Significantly larger
headerTextComp.fontStyle = FontStyle.Bold;
headerTextComp.color = new Color(0.88f, 0.58f, 1f, 0.98f);
headerTextComp.alignment = TextAnchor.MiddleCenter;

var headerOutline = headerGO.AddComponent<Outline>();
headerOutline.effectColor = new Color(0.25f, 0.05f, 0.42f, 0.95f);
headerOutline.effectDistance = new Vector2(2f, -2f);

var headerShadow = headerGO.AddComponent<Shadow>();
headerShadow.effectColor = new Color(0.65f, 0.18f, 0.95f, 0.75f);
headerShadow.effectDistance = new Vector2(3.5f, -3.5f);

RectTransform headerRT = headerGO.GetComponent<RectTransform>();
headerRT.anchoredPosition = new Vector2(0f, 210f);
headerRT.sizeDelta = new Vector2(1250f, 75f);

// Slide Content Container (Controlled by slideCanvasGroup for smooth fade-in/out)
GameObject slideContainerGO = new GameObject("SlideContentContainer");
slideContainerGO.transform.SetParent(containerGO.transform, false);
slideContainerRT = slideContainerGO.AddComponent<RectTransform>();
slideContainerRT.anchorMin = new Vector2(0.5f, 0.5f);
slideContainerRT.anchorMax = new Vector2(0.5f, 0.5f);
slideContainerRT.sizeDelta = new Vector2(1250f, 400f);
slideContainerRT.anchoredPosition = Vector2.zero;

slideCanvasGroup = slideContainerGO.AddComponent<CanvasGroup>();
slideCanvasGroup.alpha = 0f;

// Slide Progress Indicator
GameObject progressGO = new GameObject("SlideProgress");
progressGO.transform.SetParent(slideContainerGO.transform, false);
slideProgressTextComp = progressGO.AddComponent<Text>();
slideProgressTextComp.font = activeFont;
slideProgressTextComp.text = "??? Part I ???";
slideProgressTextComp.fontSize = 28;
slideProgressTextComp.fontStyle = FontStyle.Italic;
slideProgressTextComp.color = new Color(0.78f, 0.68f, 0.92f, 0.85f);
slideProgressTextComp.alignment = TextAnchor.MiddleCenter;
RectTransform progressRT = progressGO.GetComponent<RectTransform>();
progressRT.anchoredPosition = new Vector2(0f, 135f);
progressRT.sizeDelta = new Vector2(500f, 45f);

// Main Lore Text (Large, Ethereal Ivory, Magical Pulsing Outline & Shadow)
GameObject loreGO = new GameObject("LoreText");
loreGO.transform.SetParent(slideContainerGO.transform, false);
loreTextComp = loreGO.AddComponent<Text>();
loreTextComp.font = activeFont;
loreTextComp.fontSize = 42;
loreTextComp.lineSpacing = 1.42f;
loreTextComp.color = new Color(0.97f, 0.95f, 1f, 1f); // Radiant ivory parchment
loreTextComp.alignment = TextAnchor.MiddleCenter;

```

```

loreTextComp.horizontalOverflow = HorizontalWrapMode.Wrap;
loreTextComp.verticalOverflow = VerticalWrapMode.Overflow;

loreOutline = loreGO.AddComponent<Outline>();
loreOutline.effectColor = new Color(0.20f, 0.05f, 0.35f, 0.90f);
loreOutline.effectDistance = new Vector2(2f, -2f);

loreShadow = loreGO.AddComponent<Shadow>();
loreShadow.effectColor = new Color(0.55f, 0.15f, 0.85f, 0.80f);
loreShadow.effectDistance = new Vector2(3f, -3f);

RectTransform loreRT = loreGO.GetComponent<RectTransform>();
loreRT.anchoredPosition = new Vector2(0f, baseLoreY);
loreRT.sizeDelta = new Vector2(1200f, 320f);

// Bottom Continue Prompt
GameObject promptGO = new GameObject("PromptText");
promptGO.transform.SetParent(containerGO.transform, false);
promptTextComp = promptGO.AddComponent<Text>();
promptTextComp.font = activeFont;
promptTextComp.fontSize = 24;
promptTextComp.fontStyle = FontStyle.Bold;
promptTextComp.color = new Color(0.90f, 0.52f, 1f, 0.95f);
promptTextComp.alignment = TextAnchor.MiddleCenter;
promptTextComp.supportRichText = true;

var promptShadow = promptGO.AddComponent<Shadow>();
promptShadow.effectColor = new Color(0.4f, 0.05f, 0.6f, 0.7f);
promptShadow.effectDistance = new Vector2(2f, -2f);

RectTransform promptRT = promptGO.GetComponent<RectTransform>();
promptRT.anchoredPosition = new Vector2(0f, -230f);
promptRT.sizeDelta = new Vector2(900f, 50f);

// Bottom-Right Corner Skip Indicator
GameObject skipGO = new GameObject("SkipText");
skipGO.transform.SetParent(bgGO.transform, false);
skipTextComp = skipGO.AddComponent<Text>();
skipTextComp.font = activeFont;
skipTextComp.text = "[Esc] Skip";
skipTextComp.fontSize = 18;
skipTextComp.color = new Color(0.55f, 0.55f, 0.65f, 0.65f);
skipTextComp.alignment = TextAnchor.LowerRight;
RectTransform skipRT = skipGO.GetComponent<RectTransform>();
skipRT.anchorMin = new Vector2(1f, 0f);
skipRT.anchorMax = new Vector2(1f, 0f);
skipRT.pivot = new Vector2(1f, 0f);
skipRT.anchoredPosition = new Vector2(-40f, 35f);
skipRT.sizeDelta = new Vector2(250f, 40f);

// Bottom-Left Realtime Font Cycling Hint
GameObject fontHintGO = new GameObject("FontHintText");
fontHintGO.transform.SetParent(bgGO.transform, false);
fontHintTextComp = fontHintGO.AddComponent<Text>();
fontHintTextComp.font = activeFont;
fontHintTextComp.text = $"[F1] Font: {fontStyle}";
fontHintTextComp.fontSize = 16;
fontHintTextComp.color = new Color(0.50f, 0.45f, 0.62f, 0.65f);
fontHintTextComp.alignment = TextAnchor.LowerLeft;
RectTransform hintRT = fontHintGO.GetComponent<RectTransform>();
hintRT.anchorMin = new Vector2(0f, 0f);
hintRT.anchorMax = new Vector2(0f, 0f);
hintRT.pivot = new Vector2(0f, 0f);
hintRT.anchoredPosition = new Vector2(40f, 35f);
hintRT.sizeDelta = new Vector2(350f, 40f);
}

private void SpawnFloatingOrbs(Transform parent)
{
    orbs.Clear();
    int orbCount = 38;

    Color[] palette = new Color[]
    {
        new Color(0.65f, 0.20f, 0.95f), // Radiant violet
        new Color(0.85f, 0.35f, 1.00f), // Neon purple
    }

```

```

        new Color(0.92f, 0.50f, 0.98f), // Ethereal magenta
        new Color(0.48f, 0.15f, 0.85f), // Deep amethyst
        new Color(0.78f, 0.45f, 0.90f) // Soft lilac
    };

    for (int i = 0; i < orbCount; i++)
    {
        GameObject orbGO = new GameObject($"PurpleOrb_{i}");
        orbGO.transform.SetParent(parent, false);

        Image img = orbGO.AddComponent<Image>();
        img.sprite = softOrbSprite;

        RectTransform rt = orbGO.GetComponent<RectTransform>();

        // Layered out-of-focus optical bokeh sizing and depth
        float layerRoll = Random.value;
        float size;
        float alpha;
        float speed;
        float swayAmp;

        if (layerRoll < 0.35f)
        {
            // Deep background blur
            size = Random.Range(45f, 80f);
            alpha = Random.Range(0.18f, 0.36f);
            speed = Random.Range(22f, 42f);
            swayAmp = Random.Range(12f, 25f);
        }
        else if (layerRoll < 0.75f)
        {
            // Midground dreamy bokeh
            size = Random.Range(85f, 140f);
            alpha = Random.Range(0.24f, 0.44f);
            speed = Random.Range(35f, 58f);
            swayAmp = Random.Range(20f, 38f);
        }
        else
        {
            // Large foreground out-of-focus bokeh blooms
            size = Random.Range(145f, 225f);
            alpha = Random.Range(0.28f, 0.50f);
            speed = Random.Range(48f, 75f);
            swayAmp = Random.Range(28f, 50f);
        }

        rt.sizeDelta = new Vector2(size, size);

        float initialX = Random.Range(-950f, 950f);
        float initialY = Random.Range(-620f, 620f);
        rt.anchoredPosition = new Vector2(initialX, initialY);

        Color chosenColor = palette[Random.Range(0, palette.Length)];
        img.color = new Color(chosenColor.r, chosenColor.g, chosenColor.b, alpha);

        FloatingOrb orb = new FloatingOrb
        {
            rt = rt,
            img = img,
            speed = speed,
            xCenter = initialX,
            swayFreq = Random.Range(0.5f, 1.2f),
            swayAmp = swayAmp,
            phase = Random.Range(0f, Mathf.PI * 2f),
            baseAlpha = alpha,
            baseColor = chosenColor
        };

        orbs.Add(orb);
    }
}

#endregion

#region Auto-Initialization Bootstrap

```

```

[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
private static void AutoInit()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
    SceneManager.sceneLoaded += OnSceneLoaded;
    CheckAndInstantiateForScene(SceneManager.GetActiveScene().name);
}

private static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    CheckAndInstantiateForScene(scene.name);
}

private static void CheckAndInstantiateForScene(string sceneName)
{
    if (sceneName == "TutorialScene")
    {
        if (FindFirstObjectByType<TutorialLorePrologue>() == null)
        {
            GameObject go = new GameObject("[TutorialLorePrologue]");
            go.AddComponent<TutorialLorePrologue>();
            Debug.Log("<color=#D47BFF>[TutorialLorePrologue] Initialized opening lore prologue for TutorialScene.</color>");
        }
    }
}

#endregion
}

```